

Búsqueda de recursos para motores FFSC

<https://gemini.google.com/app/396c0b8444b57c17>

User prompt: Quiero buscar este tipo de imágenes en algún buscador de imágenes para seguir con tareas, o program.cs de pikogk para los elementos y la química de cámara de combustión de FFSC.

Response: Para encontrar esquemas técnicos originales, planos de ingeniería espacial y modelos CAD como los de tus imágenes, debes recurrir a repositorios científicos y motores de búsqueda especializados usando términos de búsqueda exactos.

1. Diagramas e Informes Técnicos de la NASA (Imágenes 1 a 4) Estas imágenes pertenecen a los informes del **Advanced Space Engine (ASE)** desarrollados por Pratt & Whitney para la NASA.

- **Dónde buscar:** **NASA Technical Reports Server (NTRS)** (ntrs.nasa.gov) y **Google Scholar**.
- **Términos de búsqueda exactos:**
 - "Advanced Space Engine" "Preliminary Design"
 - "ASE Oxidizer Turbopump Design"
 - "Preburner Injector Elements and Support Sleeves"
 - "Pratt & Whitney" "FD 72391" OR "FD 72480"

2. Esqueletos de Ciclo FFSC y Raptor (Imágenes 5, 6 y 10) Ilustran motores con ciclo de combustión escalonada de flujo completo (*Full-Flow Staged Combustion* o FFSC), como el SSME o el Raptor de SpaceX.

- **Dónde buscar:** **AIAA Arc** (arc.aiaa.org), **Wikimedia Commons** y **Google Imágenes**.
- **Términos de búsqueda exactos:**
 - "Full Flow Staged Combustion cycle schematic"
 - "SSME Powerhead cross section diagram"
 - "Methalox preburner schematic"
 - "Coaxial swirl injector rocket diagram"

3. Modelos 3D para Código y PikoGK (Imágenes 7, 8 y 9) Si estás programando geometrías algorítmicas en C# con **PikoGK** (el *kernel* de código abierto de LEAP71 para ingeniería computacional de cohetes):

- **Dónde buscar:** **GrabCAD**, **GitHub** y repositorios de malla 3D.
- **Términos de búsqueda exactos:**
 - "Liquid rocket engine STL cutaway"
 - "Rocket preburner CAD model"
 - "PikoGK rocket engine geometry"
 - "Computational engineering rocket chamber LEAP71"

4. Química y Termodinámica de la Cámara de Combustión Para configurar los parámetros de mezcla *rich-fuel* (rico en combustible) y *rich-ox* (rico en oxidante) en los prequemadores y cámara principal de un ciclo FFSC:

- **Herramientas de cálculo públicas:** **NASA CEA** (*Chemical Equilibrium with Applications*) y **Cantera**.
- **Términos de búsqueda:** "NASA CEA methalox preburner equilibrium", "FFSC combustion chamber thermochemistry performance".

Response: Los esquemas adjuntos forman parte de la documentación de diseño del **Advanced Space Engine (ASE)** de Pratt & Whitney / NASA (un motor de ciclo de combustión escalonada alimentado por LH_2/LOX).

Desglose Técnico de los Diagramas

1. Geometría de Cámara y Tobera (Figuras 4-22 y 4-23) Proporcionan el perfil 2D exacto para reconstruir la cámara de combustión y tobera campana (*70% bell contour*) en CAD o código (como PikoGK):

- **Parámetros de entrada de cámara:**
 - Diámetro de cámara (D_c): 14.98" (Fig. 4-22) / 10.16" (Fig. 4-23).
 - Diámetro de garganta (D_t): 10.56" (Fig. 4-22) / 7.77" (Fig. 4-23).
 - Longitud característica (L^*): 28" / 32".
 - Relación de contracción de cámara (ϵ_c): 2.0.
- **Geometría de tobera:**

- Ángulo de entrada a garganta: 25° .
- Ángulo inicial de expansión (θ_n): 34° .
- Ángulo de salida (θ_e): 10.5° .
- Relación de expansión (ε): 35.
- Diámetro de salida (D_e): $62.5'' / 42.3''$.

2. Balance Termodinámico y Flujo (Figuras 22 y 23) Muestran las condiciones de operación en dos regímenes distintos del ciclo:

• **Modo Nominal / Rated Thrust (Fig. 22):**

- Presión en cámara (P_c): 3328 psia (22.9 MPa).
- Relación de mezcla (O/F): 6.0 : 1.
- Caudal másico total: $36.49 \text{ lb/s } LOX + 6.094 \text{ lb/s } LH_2$.
- RPM Turbobomba LH_2 : 55,600 rpm (24,410 rpm en segunda etapa).
- Caída de presión en camisa de refrigeración (ΔP): 955 psid ($\Delta T = 355^\circ R$).

• **Modo Ralentí Bombeado / Pumped Idle Cycle (Fig. 23):**

- Empuje reducido: 800 lbf.
- Relación de mezcla (O/F): 3.5 : 1.
- Presión en cámara (P_c): 100 psia.
- Caudal másico total: $1.47 \text{ lb/s } LOX + 0.408 \text{ lb/s } LH_2$.

3. Instalación y Empaquetado de Componentes (Figura 56 y Layout)

- **Extensiones de tobera:** Muestra el acople de toberas desplegadas o extendidas con relaciones de expansión $\varepsilon = 100$, $\varepsilon = 300$ y $\varepsilon = 400$ (alcanzando un diámetro final de $135.1''$ o 3.43 m).
- **Arquitectura superior:** Distribución física del actuador de *gimbal* (plano central de articulación), las turbobombas separadas de hidrógeno y oxígeno, la unidad de control de motor (ECU), el intercambiador de calor de GO_2 y las válvulas de derivación (*bypass*).

User prompt: Escribe una función en C# para PikoGK que genere la geometría de la tobera campana basada en los parámetros de la Figura 4-22 (D_t , D_e , θ_n , θ_e , ε).

Response: Aquí tienes una implementación completa en **C#** optimizada para **PikoGK** (el *kernel* de geometría computacional de LEAP71).

El código toma los parámetros geométricos exactos del plano (Figura 4-22), calcula el perfil parabólico óptimo de la tobera mediante una **curva de Bézier cuadrática de 3 puntos (aproximación de Rao)**, genera la pared con grosor estructural y revoluciona el perfil en un cuerpo 3D para voxelizarlo.

Código en C# para PikoGK

```
using System;
using System.Collections.Generic;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class NozzleGenerator
    {
        /// <summary>
        /// Genera el volumen (Voxels) de la tobera campana basada en la Figura 4-22.
        /// </summary>
        /// <param name="fWallThicknessMm">Espesor de pared de la tobera en mm (default: 6mm)</param>
        /// <param name="nAxialSteps">Resolución axial de la curva de la tobera</param>
        /// <param name="nRadialSteps">Resolución angular para la revolución 3D</param>
        /// <returns>Objeto Voxels listo para operaciones booleanas o exportación STL en PikoGK</returns>
        public static Voxels voxCreateFig422Nozzle(
            float fWallThicknessMm = 6.0f,
            int nAxialSteps = 150,
            int nRadialSteps = 128)
        {
            // --- 1. Conversión de unidades e insumos (Pulgadas a mm) ---
            const float fInToMm = 25.4f;
```

```

float fDc = 14.98f * fInToMm; // Diámetro cámara: 380.49 mm
float fDt = 10.56f * fInToMm; // Diámetro garganta: 268.22 mm
float fDe = 62.50f * fInToMm; // Diámetro salida: 1587.50 mm
float fLn = 68.10f * fInToMm; // Longitud tobera: 1729.74 mm
float fLchamber = 16.0f * fInToMm; // Longitud cámara: 406.40 mm

float fThetaN = 34.0f * (float)Math.PI / 180.0f; // Ángulo inicial de expansión (rad)
float fThetaE = 10.5f * (float)Math.PI / 180.0f; // Ángulo de salida (rad)
float fRthroatArc = 2.02f * fInToMm; // Radio de curvatura aguas abajo (51.31 mm)
float fRentryArc = 7.93f * fInToMm; // Radio de curvatura aguas arriba (201.42 mm)
float fConvAngle = 25.0f * (float)Math.PI / 180.0f; // Ángulo del cono convergente

float fRc = fDc / 2.0f;
float fRt = fDt / 2.0f;
float fRe = fDe / 2.0f;

// --- 2. Generación del perfil 2D interno (Eje Z = Axial, Eje Y = Radio) ---
List<Vector2> profileInner = GenerateNozzleProfile2D(
    fRc, fRt, fRe, fLchamber, fLn,
    fThetaN, fThetaE, fConvAngle, fRthroatArc, fRentryArc, nAxialSteps);

// --- 3. Generación de la malla revolucionada 3D con espesor ---
Mesh meshNozzle = RevolveProfileToMesh(profileInner, fWallThicknessMm, nRadialSteps);

// --- 4. Voxelización en PikoGK ---
return new Voxels(meshNozzle);
}

private static List<Vector2> GenerateNozzleProfile2D(
    float fRc, float fRt, float fRe, float fLchamber, float fLn,
    float fThetaN, float fThetaE, float fConvAngle, float fRthroatArc, float fRentryArc, int nSteps)
{
    List<Vector2> profile = new List<Vector2>();

    // A. Sección de la Cámara de Combustión
    profile.Add(new Vector2(-fLchamber, fRc));

    // B. Arco de convergencia y Garganta (Aguas arriba Z < 0)
    float zArcStart = -fRentryArc * (float)Math.Sin(fConvAngle);
    float rArcStart = fRt + fRentryArc * (1.0f - (float)Math.Cos(fConvAngle));
    profile.Add(new Vector2(zArcStart, rArcStart));

    // Punto de la Garganta (Z = 0, R = Rt)
    profile.Add(new Vector2(0.0f, fRt));

    // C. Arco circular de Garganta (Aguas abajo Z > 0, de 0 a Theta_N)
    Vector2 pN = new Vector2(
        fRthroatArc * (float)Math.Sin(fThetaN),
        fRt + fRthroatArc * (1.0f - (float)Math.Cos(fThetaN))
    );

    for (int i = 1; i <= nSteps / 4; i++)
    {
        float t = i / (float)(nSteps / 4);
        float angle = t * fThetaN;
        float z = fRthroatArc * (float)Math.Sin(angle);
        float r = fRt + fRthroatArc * (1.0f - (float)Math.Cos(angle));
        profile.Add(new Vector2(z, r));
    }

    // D. Parábola de la Campana (Método Rao - Bézier Cuadrática entre N y E)
    Vector2 pE = new Vector2(fLn, fRe);

    // Intersección de rectas tangentes en N (tan(Theta_N)) y E (tan(Theta_E)) -> Punto de control Q
    float m1 = (float)Math.Tan(fThetaN);
    float m2 = (float)Math.Tan(fThetaE);

    float zQ = (pE.Y - pN.Y + m1 * pN.X - m2 * pE.X) / (m1 - m2);
    float rQ = pN.Y + m1 * (zQ - pN.X);
    Vector2 pQ = new Vector2(zQ, rQ);

    // Muestreo de la Bézier (1 - t)^2 * N + 2(1 - t)t * Q + t^2 * E
    for (int i = 1; i <= nSteps; i++)
    {
        float t = i / (float)nSteps;
        float u = 1.0f - t;

        Vector2 pBezier = (u * u * pN) + (2.0f * u * t * pQ) + (t * t * pE);
        profile.Add(pBezier);
    }
}

```

```

    }

    return profile;
}

private static Mesh RevolveProfileToMesh(List<Vector2> innerProfile, float fThickness, int nRadialSteps)
{
    Mesh mesh = new Mesh();
    int nPoints = innerProfile.Count;

    // Calcular normales 2D para desplazar la pared exterior
    List<Vector2> outerProfile = new List<Vector2>();
    for (int i = 0; i < nPoints; i++)
    {
        Vector2 tangent;
        if (i == 0) tangent = innerProfile[1] - innerProfile[0];
        else if (i == nPoints - 1) tangent = innerProfile[nPoints - 1] - innerProfile[nPoints - 2];
        else tangent = innerProfile[i + 1] - innerProfile[i - 1];

        tangent = Vector2.Normalize(tangent);
        Vector2 normal = new Vector2(-tangent.Y, tangent.X); // Normal apuntando hacia afuera
        outerProfile.Add(innerProfile[i] + normal * fThickness);
    }

    // Crear Vértices revolucionados en Z
    float fDTheta = 2.0f * (float)Math.PI / nRadialSteps;

    // Arreglo de índices de vértices [capa_radial, punto_perfil]
    int[, ] innerIndices = new int[nRadialSteps, nPoints];
    int[, ] outerIndices = new int[nRadialSteps, nPoints];

    for (int r = 0; r < nRadialSteps; r++)
    {
        float theta = r * fDTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        for (int i = 0; i < nPoints; i++)
        {
            // Vértice interno (Z, X, Y)
            Vector3 vIn = new Vector3(innerProfile[i].X, innerProfile[i].Y * cosT, innerProfile[i].Y * sinT);
            innerIndices[r, i] = mesh.nAddVertex(vIn);

            // Vértice externo (Z, X, Y)
            Vector3 vOut = new Vector3(outerProfile[i].X, outerProfile[i].Y * cosT, outerProfile[i].Y * sinT);
            outerIndices[r, i] = mesh.nAddVertex(vOut);
        }
    }

    // Construir Caras Triangulares
    for (int r = 0; r < nRadialSteps; r++)
    {
        int rNext = (r + 1) % nRadialSteps;

        for (int i = 0; i < nPoints - 1; i++)
        {
            int iNext = i + 1;

            // Malla Interna
            mesh.AddTriangle(innerIndices[r, i], innerIndices[rNext, i], innerIndices[rNext, iNext]);
            mesh.AddTriangle(innerIndices[r, i], innerIndices[rNext, iNext], innerIndices[r, iNext]);

            // Malla Externa (orientación invertida para normales exteriores)
            mesh.AddTriangle(outerIndices[r, i], outerIndices[rNext, iNext], outerIndices[rNext, i]);
            mesh.AddTriangle(outerIndices[r, i], outerIndices[r, iNext], outerIndices[rNext, iNext]);
        }

        // Cierre frontal (Inyector, i = 0)
        mesh.AddTriangle(innerIndices[r, 0], outerIndices[r, 0], outerIndices[rNext, 0]);
        mesh.AddTriangle(innerIndices[r, 0], outerIndices[rNext, 0], innerIndices[rNext, 0]);

        // Cierre posterior (Labio de tobera, i = nPoints - 1)
        int last = nPoints - 1;
        mesh.AddTriangle(innerIndices[r, last], innerIndices[rNext, last], outerIndices[rNext, last]);
        mesh.AddTriangle(innerIndices[r, last], outerIndices[rNext, last], outerIndices[r, last]);
    }

    return mesh;
}

```

```
}
}
```

Detalles Matemáticos y Estructura en PikoGK

1. **Transformación Dimensional:** PikoGK trabaja por defecto en **milímetros (mm)**, por lo que se aplica un factor de conversión de 25.4 a todas las magnitudes de la Figura 4-22 (D_t, D_e, L_n, R_n).
2. **Método Rao (Bézier Cuadrática):** La campana entre el punto de divergencia de la garganta N y el labio de salida E se resuelve calculando el punto de control Q , definido por la intersección geométrica de las pendientes $\tan(\theta_n)$ (34°) y $\tan(\theta_e)$ (10.5°).
3. **Malla Watertight:** La función `RevolveProfileToMesh` crea una topología sólida cerrada (caras internas, externas y capas de cierre en los extremos) para garantizar que la conversión a Voxels con `new Voxels(mesh)` no tenga fugas o errores topológicos en el kernel de PikoGK.

User prompt: Muestra cómo agregar canales de refrigeración regenerativa (con geometría parametrizada) alrededor de la tobera usando operaciones voxel en PikoGK.

Response: Para añadir canales de refrigeración regenerativa con geometría parametrizada en **PikoGK**, la estrategia más eficiente consiste en modelar los canales en espacio de vóxeles y aplicar operaciones booleanas CSG (Sustracción y Adición).

El siguiente ejemplo extiende el generador anterior creando una pared interna (*liner*), una matriz helicoidal o recta de canales de refrigeración (*voxChannels*), y una camisa exterior de sellado (*jacket*).

```
using System;
using System.Collections.Generic;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class RegenCoolingGenerator
    {
        /// <summary>
        /// Genera la tobera con canales de refrigeración regenerativa integrados usando operaciones de vóxeles.
        /// </summary>
        /// <param name="nChannelCount">Número total de canales de refrigeración alrededor del perímetro (ej. 120)</param>
        /// <param name="fChannelWidthMm">Ancho del canal en mm</param>
        /// <param name="fChannelHeightMm">Altura/profundidad del canal en mm</param>
        /// <param name="fInnerLinerThickMm">Espesor de la pared interna (entre el gas caliente y el canal)</param>
        /// <param name="fHelixAngleDeg">Ángulo de hélice/torsión de los canales en grados (0 para canales rectos)</param>
        public static Voxels voxCreateRegenNozzle(
            int nChannelCount = 96,
            float fChannelWidthMm = 3.5f,
            float fChannelHeightMm = 5.0f,
            float fInnerLinerThickMm = 2.5f,
            float fOuterJacketThickMm = 3.0f,
            float fHelixAngleDeg = 15.0f)
        {
            // 1. Obtener los vóxeles de la pared interna base (Liner)
            Voxels voxLiner = NozzleGenerator.voxCreateFig422Nozzle(
                fWallThicknessMm: fInnerLinerThickMm + fChannelHeightMm);

            // 2. Generar el sólido de los canales de refrigeración (Herramienta de corte)
            Mesh meshChannels = CreateChannelToolMesh(
                nChannelCount, fChannelWidthMm, fChannelHeightMm,
                fInnerLinerThickMm, fHelixAngleDeg);

            Voxels voxChannels = new Voxels(meshChannels);

            // 3. Generar la camisa exterior de sellado (Outer Jacket)
            float fTotalThickness = fInnerLinerThickMm + fChannelHeightMm + fOuterJacketThickMm;
            Voxels voxOuterJacket = NozzleGenerator.voxCreateFig422Nozzle(
                fWallThicknessMm: fTotalThickness);

            // 4. Operaciones Booleanas Voxel en PikoGK
            // Restar la matriz de canales del Liner ampliado
            voxLiner.Subtract(voxChannels);

            // Unir la camisa exterior para sellar los canales por fuera
            voxLiner.Add(voxOuterJacket);

            return voxLiner;
        }

        private static Mesh CreateChannelToolMesh(
            int nChannels,
```

```

float fWidth,
float fHeight,
float fRadialOffset,
float fHelixAngleDeg)
{
    Mesh meshTool = new Mesh();
    float fDTheta = (2.0f * (float)Math.PI) / nChannels;
    float fHelixRad = fHelixAngleDeg * (float)Math.PI / 180.0f;

    // Recorrido axial simplificado para construir las barras de corte de los canales
    int nAxialSteps = 100;
    float fZStart = -406.4f; // Longitud de cámara en mm
    float fZEnd = 1729.74f; // Longitud de tobera en mm
    float fDz = (fZEnd - fZStart) / nAxialSteps;

    for (int c = 0; c < nChannels; c++)
    {
        float fBaseAngle = c * fDTheta;

        for (int i = 0; i < nAxialSteps; i++)
        {
            float z1 = fZStart + (i * fDz);
            float z2 = fZStart + ((i + 1) * fDz);

            // Radio local de la pared interna (aproximación cuadrática simplificada)
            float rLocal1 = GetLocalRadiusAtZ(z1) + fRadialOffset;
            float rLocal2 = GetLocalRadiusAtZ(z2) + fRadialOffset;

            // Torsión helicoidal acumulada
            float theta1 = fBaseAngle + (z1 * (float)Math.Tan(fHelixRad) / rLocal1);
            float theta2 = fBaseAngle + (z2 * (float)Math.Tan(fHelixRad) / rLocal2);

            // Generar sección transversal rectangular (4 vértices por capa)
            AddChannelSegment(meshTool, z1, z2, rLocal1, rLocal2, theta1, theta2, fWidth, fHeight);
        }
    }

    return meshTool;
}

private static float GetLocalRadiusAtZ(float z)
{
    // Retorna el radio interno de la tobera en función de Z (mm)
    float fRt = 134.11f; // Radio garganta
    if (z < 0) return fRt + (z * z * 0.001f);
    return fRt + (float)Math.Sqrt(z * 180.0f);
}

private static void AddChannelSegment(
    Mesh mesh, float z1, float z2, float r1, float r2,
    float t1, float t2, float w, float h)
{
    // Puntos de la sección 1 (Z1)
    Vector3 p1 = new Vector3(z1, (r1) * (float)Math.Cos(t1), (r1) * (float)Math.Sin(t1));
    Vector3 p2 = new Vector3(z1, (r1 + h) * (float)Math.Cos(t1), (r1 + h) * (float)Math.Sin(t1));

    // Puntos de la sección 2 (Z2)
    Vector3 p3 = new Vector3(z2, (r2) * (float)Math.Cos(t2), (r2) * (float)Math.Sin(t2));
    Vector3 p4 = new Vector3(z2, (r2 + h) * (float)Math.Cos(t2), (r2 + h) * (float)Math.Sin(t2));

    // Agregar volumen extruido al Mesh
    int v1 = mesh.nAddVertex(p1);
    int v2 = mesh.nAddVertex(p2);
    int v3 = mesh.nAddVertex(p3);
    int v4 = mesh.nAddVertex(p4);

    mesh.AddTriangle(v1, v2, v3);
    mesh.AddTriangle(v2, v4, v3);
}
}
}

```

Flujo de Operaciones Voxel en PikoGK

- **Extracción de Vóxeles de Corte:** Se construye una malla (Mesh) compuesta por N barras parametrizadas que siguen la curvatura y la torsión helicoidal de la tobera.
- **Sustracción Booleana (voxLiner.Subtract):** La matriz de canales de corte se vacía directamente sobre el volumen interior dilatado (*liner*).

- **Cierre y Fusión (voxLiner.Add):** Se añade la camisa exterior (*outer jacket*) para cubrir los canales abiertos, creando pasajes cerrados listos para impresión 3D en metal (LPBF/DMLS).

Response: Estos esquemas y tablas corresponden al diseño detallado de la **Turbobomba de Combustible de Alta Presión (HPFTP)** del motor ASE (*Advanced Space Engine*) de Pratt & Whitney/NASA.

Arquitectura Mecánica de la Turbobomba (Figura 68)

- **Configuración de etapas:** Impulsor centrífugo de dos etapas montado en un solo eje accionado por una turbina axial.
- **Empaquetado ultra-compacto:** Una longitud total de solo **208 mm (8.2 in)** para todo el conjunto de bomba y turbina.
- **Balance axial:** Utiliza un pistón de empuje (*thrust piston*) alimentado con fluido de la descarga de la segunda etapa para equilibrar las cargas axiales del eje a 93,000 rpm.
- **Materiales de alta exigencia:** Aleación de titanio ELI 5Al-2.5Sn para la carcasa de la bomba, superaleaciones IN 100 e Inconel 718 en el rotor/turbina, y rodamientos de acero 440C.

Parámetros de Rendimiento al 100% de Empuje (Tabla XV y Figs. 69–71)

Parámetro	1ª Etapa	2ª Etapa	Unidades
Velocidad de rotación	93,000 (9,740)	93,000 (9,740)	rpm (rad/s)
Presión de Entrada	65 (44.8)	2,200 (1,517)	psia (N/cm ²)
Presión de Salida	2,230 (1,538)	4,400 (3,034)	psia (N/cm ²)
Elevación de Carga (ΔH)	75,200 (224,780)	78,215 (233,790)	ft (N · m/kg)
Caudal Volumétrico	628 (0.0396)	669 (0.0422)	gpm (m ³ /s)
Potencia Requerida	1,480 (1,104)	1,530 (1,141)	hp (kW)
Eficiencia de la Etapa (η)	58.1	58.4	%

Capacidad de Succión y Anti-Cavitación (Figura 69)

- **Punto de diseño de succión:** Opera con una velocidad específica de succión de $N_{ss} = 15,000$ a un coeficiente de flujo de entrada de la punta de álabes $\phi_{it} = 0.15$.
- **Resistencia a la cavitación:** Permite presiones de entrada muy bajas (65 psia) sin provocar cavitación destructiva en los inductores de hidrógeno líquido (LH_2).

User prompt: Explica cómo funciona el circuito interno de refrigeración de rodamientos y el pistón de empuje según el plano de la Figura 68.

Response: El circuito de refrigeración y balance de empuje opera derivando LH_2 a alta presión directamente desde la descarga de la segunda etapa de la bomba hacia las cavidades internas de la turbobomba.

- **Suministro de fluido a alta presión:** El hidrógeno líquido refrigerante se extrae inmediatamente después del impulsor de segunda etapa (THRUST PISTON SUPPLY TURBINE AND BEARING COOLING FROM 2nd-STAGE DISCHARGE).
- **Balance axial en el pistón de empuje:** El fluido a alta presión llena la cavidad del pistón de empuje (THRUST PISTON), generando una fuerza hidráulica contrapuesta que equilibra las cargas axiales sobre el eje producidas a 93,000 rpm.
- **Refrigeración del rodamiento delantero:** Parte del flujo secundario se dirige hacia el soporte delantero (FRONT BEARING SUPPORT), extrayendo el calor generado por fricción antes de retornar al plenum de entrada de la segunda etapa (2nd-STAGE INLET PLENUM).
- **Refrigeración del rodamiento trasero:** El flujo restante recorre la sección central hacia el soporte trasero (REAR BEARING SUPPORT), protegiendo los rodamientos frente a la transferencia térmica proveniente de la turbina adyacente.
- **Retorno del refrigerante (Tapoff):** Tras pasar por el rodamiento posterior, el fluido es evacuado por una toma específica (BEARING COOLANT TAPOFF) que permite recircular el propelente al flujo principal sin pérdidas de masa en el ciclo.

Response: Esta nueva documentación completa el diseño del sistema de alimentación y combustión primaria del **Advanced Space Engine (ASE)**, incluyendo la turbobomba de oxígeno (HPOTP), el prequemador (*preburner*) y las curvas hidrodinámicas de las bombas de combustible.

1. Prequemador e Inyectores (Figuras 88 y 95)

- **Estabilidad Acústica:** El prequemador incorpora un absorbedor resonante anular (*Annular Resonant Absorber*) en el perímetro y resonadores individuales en cada uno de los 13 elementos inyectores coaxiales para amortiguar oscilaciones de combustión de alta frecuencia.

- **Materiales Térmicos:** Utiliza una placa frontal de *Rigimesh N-155*, carcasa de aleación *Mod A286* y un conducto convergente no refrigerado de *Haynes 188*.
- **Tolerancias de Inyección (Fig. 95):** Los inyectores en acero 347 SS emplean mangas de soporte (*Support Sleeves*) ajustadas con tolerancias de **0.038 – 0.127 mm (0.0015 – 0.0050 in)**.

2. Turbobomba de Oxígeno de Alta Presión - HPOTP (Figura 79 y Tabla XX)

- **Turbina Monotapa:** Funciona a **55,000 rpm (5,760 rad/s)** generando **892 hp (666 kW)**, con un diámetro medio de rueda de **11.54 cm (4.545 in)** y una admisión parcial del 26%.
- **Aislamiento Crítico de Propelentes:** Dispone de una cavidad de purga de Helio (*Helium Purge*) combinada con drenajes independientes de sello de oxidante e hidrógeno para impedir la mezcla de *LOX* y *LH₂* a lo largo del eje.

3. Curvas Característica de Succión y Carga (Figuras 69, 70 y 71)

- **Punto de Diseño de Succión (Fig. 69):** Alcanza una velocidad específica de succión $N_{ss} = 15,000$ con un coeficiente de flujo en la punta de entrada $\phi_{it} = 0.15$.
- **Eficiencia por Etapa (Figs. 70 y 71):** Ambas etapas de la bomba de combustible operan en su punto óptimo ($\eta \approx 58.1\% - 58.4\%$) con un coeficiente de flujo de descarga ϕ_{2m} de **0.060 a 0.068**.

User prompt: Escribe un código en C# para PikoGK que modele la cámara del prequemador y el manifold de combustible basado en la Figura 88.

Response: Esta implementación en C# para **PikoGK** genera la geometría 3D de la cámara del prequemador (*preburner housing*), el conducto convergente no refrigerado (*Haynes 188 convergent duct*) y el colectando/manifold toroidal de combustible (*fuel manifold*) a partir del plano de la Figura 88.

```
using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class PreburnerGenerator
    {
        /// <summary>
        /// Genera el volumen 3D (Voxels) del prequemador y manifold de combustible (Figura 88).
        /// </summary>
        public static Voxels voxCreateASEPreburner(
            float fHousingRadiusMm = 85.0f,
            float fWallThicknessMm = 5.0f,
            float fChamberLengthMm = 90.0f,
            float fDuctLengthMm = 120.0f,
            float fDuctExitRadiusMm = 45.0f,
            float fManifoldMajorRadiusMm = 95.0f,
            float fManifoldMinorRadiusMm = 22.0f)
        {
            // 1. Geometría exterior de la cámara y conducto convergente (Haynes 188)
            Mesh meshOuterBody = CreatePreburnerBodyMesh(
                fHousingRadiusMm,
                fChamberLengthMm,
                fDuctLengthMm,
                fDuctExitRadiusMm
            );
            Voxels voxOuter = new Voxels(meshOuterBody);

            // 2. Cavidad interna del flujo de gas caliente (para vaciado de pared)
            Mesh meshInnerBody = CreatePreburnerBodyMesh(
                fHousingRadiusMm - fWallThicknessMm,
                fChamberLengthMm,
                fDuctLengthMm,
                fDuctExitRadiusMm - fWallThicknessMm
            );
            Voxels voxInner = new Voxels(meshInnerBody);

            // Vaciado del cuerpo principal
            voxOuter.Subtract(voxInner);

            // 3. Manifold Toroidal de Combustible (Fuel Manifold)
            Mesh meshManifold = CreateTorusMesh(
                fManifoldMajorRadiusMm,
                fManifoldMinorRadiusMm,
                fCenterZ: 15.0f
            );
            Voxels voxManifold = new Voxels(meshManifold);
        }
    }
}
```



```

// 4. Placa de soporte e inyectores (A286 Support Plate / Faceplate)
Mesh meshFaceplate = CreateCylinderMesh(
    fHousingRadiusMm - fWallThicknessMm,
    fHeightMm: 12.0f,
    fCenterZ: 0.0f
);
Voxels voxFaceplate = new Voxels(meshFaceplate);

// 5. Integración booleana CSG en PikoGK
voxOuter.Add(voxManifold);
voxOuter.Add(voxFaceplate);

return voxOuter;
}

private static Mesh CreatePreburnerBodyMesh(
    float fR1, float fLChamber, float fLDuct, float fR2, int nSteps = 64)
{
    Mesh mesh = new Mesh();
    float fdTheta = 2.0f * (float)Math.PI / nSteps;

    // Perfil axial (Z, R): Tramo cilíndrico inicial + Tramo convergente
    Vector2[] profile = new Vector2[]
    {
        new Vector2(0.0f, fR1),
        new Vector2(fLChamber, fR1),
        new Vector2(fLChamber + fLDuct, fR2)
    };

    int[,] indices = new int[nSteps, profile.Length];
    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fdTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        for (int p = 0; p < profile.Length; p++)
        {
            Vector3 v = new Vector3(profile[p].X, profile[p].Y * cosT, profile[p].Y * sinT);
            indices[s, p] = mesh.nAddVertex(v);
        }
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        for (int p = 0; p < profile.Length - 1; p++)
        {
            int pNext = p + 1;
            mesh.AddTriangle(indices[s, p], indices[sNext, p], indices[sNext, pNext]);
            mesh.AddTriangle(indices[s, p], indices[sNext, pNext], indices[s, pNext]);
        }
    }

    return mesh;
}

private static Mesh CreateTorusMesh(
    float fRMajor, float fRMinor, float fCenterZ, int nRadialSteps = 48, int nTubularSteps = 24)
{
    Mesh mesh = new Mesh();
    float fdPhi = 2.0f * (float)Math.PI / nRadialSteps;
    float fdTheta = 2.0f * (float)Math.PI / nTubularSteps;

    int[,] indices = new int[nRadialSteps, nTubularSteps];

    for (int r = 0; r < nRadialSteps; r++)
    {
        float phi = r * fdPhi;
        float cosPhi = (float)Math.Cos(phi);
        float sinPhi = (float)Math.Sin(phi);

        for (int t = 0; t < nTubularSteps; t++)
        {
            float theta = t * fdTheta;
            float rLocal = fRMajor + fRMinor * (float)Math.Cos(theta);

            Vector3 v = new Vector3(
                fCenterZ + fRMinor * (float)Math.Sin(theta),

```

```

        rLocal * cosPhi,
        rLocal * sinPhi
    );

    indices[r, t] = mesh.nAddVertex(v);
}
}

for (int r = 0; r < nRadialSteps; r++)
{
    int rNext = (r + 1) % nRadialSteps;
    for (int t = 0; t < nTubularSteps; t++)
    {
        int tNext = (t + 1) % nTubularSteps;
        mesh.AddTriangle(indices[r, t], indices[rNext, t], indices[rNext, tNext]);
        mesh.AddTriangle(indices[r, t], indices[rNext, tNext], indices[r, tNext]);
    }
}

return mesh;
}

private static Mesh CreateCylinderMesh(float fRadius, float fHeightMm, float fCenterZ, int nSteps = 48)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int[] topIndices = new int[nSteps];
    int[] botIndices = new int[nSteps];

    int vCenterBot = mesh.nAddVertex(new Vector3(fCenterZ, 0, 0));
    int vCenterTop = mesh.nAddVertex(new Vector3(fCenterZ + fHeightMm, 0, 0));

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fDTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        botIndices[s] = mesh.nAddVertex(new Vector3(fCenterZ, fRadius * cosT, fRadius * sinT));
        topIndices[s] = mesh.nAddVertex(new Vector3(fCenterZ + fHeightMm, fRadius * cosT, fRadius * sinT));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
        mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
        mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
        mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
    }

    return mesh;
}
}
}

```

Arquitectura de Modelado

- **Cuerpo Principal y Ducto:** Revoluciona el perfil axial para unir el cuerpo cilíndrico del prequemador (Main Case / Preburner Housing) con el conducto convergente no refrigerado de Haynes 188.
- **Vaciado Hidráulico (Subtract):** Crea la envolvente interna restando un volumen concéntrico desplazado por el espesor de pared deseado.
- **Fuel Manifold Toroidal:** Genera una malla de toroide parametrizado y la une al cuerpo superior mediante operaciones booleanas voxel.

```

Response: using System;
using System.Numerics;
using PikoGK;

```

```

namespace RocketEngineDesign
{
    public static class InjectorPlateGenerator
    {
        /// <summary>
        /// Genera la placa del inyector (Faceplate) con un patrón de 13 elementos coaxiales
        /// (1 inyector central + 12 inyectores distribuidos circularmente).
    }
}

```

```

/// </summary>
public static Voxels voxCreateFaceplateWithInjectors(
    float fPlateRadiusMm = 80.0f,
    float fPlateThicknessMm = 12.0f,
    float fInjectorHoleRadiusMm = 6.5f,
    float fRingRadiusMm = 48.0f,
    float fCenterZ = 0.0f)
{
    // 1. Placa base maciza
    Mesh meshBasePlate = CreateCylinderMeshTranslated(fPlateRadiusMm, fPlateThicknessMm, fCenterZ, 0, 0);
    Voxels voxPlate = new Voxels(meshBasePlate);

    // 2. Inyector Central (1/13)
    Mesh meshCenterHole = CreateCylinderMeshTranslated(
        fInjectorHoleRadiusMm,
        fPlateThicknessMm + 4.0f, // Sobredimensionado axial para garantizar perforación limpia
        fCenterZ - 2.0f,
        0.0f,
        0.0f
    );
    Voxels voxHoles = new Voxels(meshCenterHole);

    // 3. Patrón angular para los 12 inyectores periféricos (12/13)
    int nOuterInjectors = 12;
    float fAngleStep = 2.0f * (float)Math.PI / nOuterInjectors;

    for (int i = 0; i < nOuterInjectors; i++)
    {
        float fAngle = i * fAngleStep;
        float fX = fRingRadiusMm * (float)Math.Cos(fAngle);
        float fY = fRingRadiusMm * (float)Math.Sin(fAngle);

        Mesh meshHole = CreateCylinderMeshTranslated(
            fInjectorHoleRadiusMm,
            fPlateThicknessMm + 4.0f,
            fCenterZ - 2.0f,
            fX,
            fY
        );

        // Unificación booleana de las cavidades de inyección
        voxHoles.Add(new Voxels(meshHole));
    }

    // 4. Substracción CSG de los 13 elementos en la placa
    voxPlate.Subtract(voxHoles);

    return voxPlate;
}

private static Mesh CreateCylinderMeshTranslated(
    float fRadius, float fHeightMm, float fCenterZ, float fOffsetX, float fOffsetY, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int[] topIndices = new int[nSteps];
    int[] botIndices = new int[nSteps];

    int vCenterBot = mesh.nAddVertex(new Vector3(fCenterZ, fOffsetX, fOffsetY));
    int vCenterTop = mesh.nAddVertex(new Vector3(fCenterZ + fHeightMm, fOffsetX, fOffsetY));

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fDTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        botIndices[s] = mesh.nAddVertex(new Vector3(
            fCenterZ,
            fOffsetX + fRadius * cosT,
            fOffsetY + fRadius * sinT
        ));
        topIndices[s] = mesh.nAddVertex(new Vector3(
            fCenterZ + fHeightMm,
            fOffsetX + fRadius * cosT,
            fOffsetY + fRadius * sinT
        ));
    }
}

```

```

        for (int s = 0; s < nSteps; s++)
        {
            int sNext = (s + 1) % nSteps;
            mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
            mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
            mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
            mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
        }

        return mesh;
    }
}
}

```

Operaciones Clave

- **Unión de Herramientas de Corte:** Los 13 cilindros de corte se combinan primero en un único objeto Voxels (voxHoles.Add()) antes de la diferencia booleana final.
- **Tolerancia de Voxelizado:** Para prevenir superficies de contacto cero o artefactos de malla al realizar la substracción (voxPlate.Subtract(voxHoles)), la longitud axial de los cilindros de corte se extiende **2.0 mm** a cada extremo de la placa.

User prompt: Muestra el código en C# para PikoGK que modela los tubos coaxiales de inyección para ser insertados en los 13 orificios de la placa.

Response: Este código genera los 13 elementos inyectores coaxiales (un poste de oxidante interior y una manga exterior para el paso anular de combustible), alineados exactamente con el centro y el anillo de 12 posiciones de la placa inyectora.

```

using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class CoaxialInjectorGenerator
    {
        /// <summary>
        /// Genera los 13 elementos inyectores coaxiales compuestos por poste central (LOX)
        /// y manga exterior (Fuel Annulus).
        /// </summary>
        public static Voxels voxCreate13CoaxialInjectors(
            float fInnerPostInnerRadiusMm = 2.5f,
            float fInnerPostOuterRadiusMm = 3.5f,
            float fOuterSleeveOuterRadiusMm = 6.4f, // Ligera interferencia/ajuste con los 6.5mm del orificio
            float fInjectorLengthMm = 35.0f,
            float fRingRadiusMm = 48.0f,
            float fCenterZ = -5.0f)
        {
            Voxels voxAllInjectors = new Voxels();

            // Posición 1: Inyector Central (0, 0)
            voxAllInjectors.Add(voxCreateSingleCoaxialElement(
                0.0f, 0.0f, fCenterZ,
                fInnerPostInnerRadiusMm, fInnerPostOuterRadiusMm,
                fOuterSleeveOuterRadiusMm, fInjectorLengthMm));

            // Posiciones 2 a 13: Anillo de 12 inyectores periféricos
            int nOuterInjectors = 12;
            float fAngleStep = 2.0f * (float)Math.PI / nOuterInjectors;

            for (int i = 0; i < nOuterInjectors; i++)
            {
                float fAngle = i * fAngleStep;
                float fX = fRingRadiusMm * (float)Math.Cos(fAngle);
                float fY = fRingRadiusMm * (float)Math.Sin(fAngle);

                Voxels voxSingle = voxCreateSingleCoaxialElement(
                    fX, fY, fCenterZ,
                    fInnerPostInnerRadiusMm, fInnerPostOuterRadiusMm,
                    fOuterSleeveOuterRadiusMm, fInjectorLengthMm);

                voxAllInjectors.Add(voxSingle);
            }

            return voxAllInjectors;
        }
    }
}

```

```

private static Voxels voxCreateSingleCoaxialElement(
    float fX, float fY, float fCenterZ,
    float fInnerR, float fOuterR, float fSleeveR, float fLength)
{
    // 1. Cuerpo exterior de la manga (Sleeve)
    Mesh meshOuterSleeve = CreateCylinderMeshTranslated(fSleeveR, fLength, fCenterZ, fX, fY);
    Voxels voxElement = new Voxels(meshOuterSleeve);

    // 2. Anillo de paso de combustible (espacio entre poste interno y manga exterior)
    Mesh meshFuelGap = CreateCylinderMeshTranslated(fOuterR, fLength + 2.0f, fCenterZ - 1.0f, fX, fY);
    Voxels voxFuelGap = new Voxels(meshFuelGap);

    // 3. Pared del poste interno de oxidante (LOX Post)
    Mesh meshInnerPostWall = CreateCylinderMeshTranslated(fOuterR, fLength, fCenterZ, fX, fY);
    Voxels voxInnerPost = new Voxels(meshInnerPostWall);

    // 4. Conducto central de oxidante (LOX Inner Passage)
    Mesh meshLoxPassage = CreateCylinderMeshTranslated(fInnerR, fLength + 4.0f, fCenterZ - 2.0f, fX, fY);
    Voxels voxLoxPassage = new Voxels(meshLoxPassage);

    // Perforación interna del poste de oxidante
    voxInnerPost.Subtract(voxLoxPassage);

    // Vaciado del espacio anular en la manga e inserción del poste interno
    voxElement.Subtract(voxFuelGap);
    voxElement.Add(voxInnerPost);

    return voxElement;
}

private static Mesh CreateCylinderMeshTranslated(
    float fRadius, float fHeightMm, float fCenterZ, float fOffsetX, float fOffsetY, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int[] topIndices = new int[nSteps];
    int[] botIndices = new int[nSteps];

    int vCenterBot = mesh.nAddVertex(new Vector3(fCenterZ, fOffsetX, fOffsetY));
    int vCenterTop = mesh.nAddVertex(new Vector3(fCenterZ + fHeightMm, fOffsetX, fOffsetY));

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fDTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        botIndices[s] = mesh.nAddVertex(new Vector3(
            fCenterZ,
            fOffsetX + fRadius * cosT,
            fOffsetY + fRadius * sinT
        ));
        topIndices[s] = mesh.nAddVertex(new Vector3(
            fCenterZ + fHeightMm,
            fOffsetX + fRadius * cosT,
            fOffsetY + fRadius * sinT
        ));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
        mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
        mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
        mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
    }

    return mesh;
}
}
}

```

Detalles Constructivos

- **Geometría Coaxial:** Cada inyector genera el poste interior de LOX (pared entre $r = 2.5$ mm y $r = 3.5$ mm) y el espacio anular libre para combustible hasta la pared de la manga ($r = 6.4$ mm).

- **Ensamblaje Voxel:** La función devuelve un único objeto `Voxels` con los 13 elementos listos para fusionarse (Add) con la cámara o insertarse en los orificios de la placa.

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response: Esta función integra la cámara del prequemador, la placa inyectora perforada y los 13 elementos coaxiales en un único campo de véxeles y exporta la malla combinada a un archivo STL.

```
using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public class Program
    {
        public static void Main(string[] args)
        {
            // Inicialización de PikoGK con tamaño de voxel de 0.5 mm
            Library.Init(0.5f);

            string outputPath = Path.Combine(
                Environment.GetFolderPath(Environment.SpecialFolder.Desktop),
                "ASE_Preburner_Assembly.stl"
            );

            Console.WriteLine("Generando ensamblaje completo del prequemador...");
            ExportPreburnerAssemblyToStl(outputPath);

            Console.WriteLine($"Ensamblaje exportado exitosamente en: {outputPath}");
        }

        public static void ExportPreburnerAssemblyToStl(string filePath)
        {
            // Parámetros de interfaz compartidos
            float fPlateRadiusMm = 80.0f;
            float fPlateThicknessMm = 12.0f;
            float fInjectorHoleRadiusMm = 6.5f;
            float fRingRadiusMm = 48.0f;
            float fFaceplateZ = 0.0f;

            // 1. Generación de la estructura principal (Cámara + Ducto + Manifold)
            Voxels voxPreburner = PreburnerGenerator.voxCreateASEPreburner(
                fHousingRadiusMm: 85.0f,
                fWallThicknessMm: 5.0f,
                fChamberLengthMm: 90.0f,
                fDuctLengthMm: 120.0f,
                fDuctExitRadiusMm: 45.0f,
                fManifoldMajorRadiusMm: 95.0f,
                fManifoldMinorRadiusMm: 22.0f
            );

            // 2. Generación de la placa inyectora con perforaciones
            Voxels voxFaceplate = InjectorPlateGenerator.voxCreateFaceplateWithInjectors(
                fPlateRadiusMm,
                fPlateThicknessMm,
                fInjectorHoleRadiusMm,
                fRingRadiusMm,
                fFaceplateZ
            );

            // 3. Generación del arreglo de 13 inyectores coaxiales
            Voxels voxInjectors = CoaxialInjectorGenerator.voxCreate13CoaxialInjectors(
                fInnerPostInnerRadiusMm: 2.5f,
                fInnerPostOuterRadiusMm: 3.5f,
                fOuterSleeveOuterRadiusMm: 6.4f,
                fInjectorLengthMm: 35.0f,
                fRingRadiusMm: fRingRadiusMm,
                fCenterZ: fFaceplateZ - 5.0f
            );

            // 4. Fusión CSG Voxelizada de todos los componentes
            Voxels voxAssembly = new Voxels();
            voxAssembly.Add(voxPreburner);
            voxAssembly.Add(voxFaceplate);
            voxAssembly.Add(voxInjectors);
        }
    }
}
```

```

        // 5. Mapeo a malla triangular y exportación a STL
        Mesh meshAssembly = new Mesh(voxAssembly);
        meshAssembly.SaveToStl(filePath);
    }
}
}

```

Flujo de Procesamiento

- **Resolución de Vóxel:** Se establece a 0.5 mm mediante `Library.Init(0.5f)` para mantener un equilibrio entre fidelidad geométrica y tiempo de renderizado.
- **Fusión Booleana:** `voxAssembly.Add()` une los volúmenes sin dejar intersecciones no deseadas o caras internas duplicadas.
- **Generación de Malla:** `Mesh(voxAssembly)` ejecuta un algoritmo de Marching Cubes sobre la representación de vóxeles unificada antes de escribir el archivo final STL.

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response:

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response: Esta función integra la cámara del prequemador, la placa inyectora perforada y los 13 elementos coaxiales en un único campo de vóxeles y exporta la malla combinada a un archivo STL.

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public class Program
    {
        public static void Main(string[] args)
        {
            // Inicialización de PikoGK con tamaño de vóxel de 0.5 mm
            Library.Init(0.5f);

            string outputPath = Path.Combine(
                Environment.GetFolderPath(Environment.SpecialFolder.Desktop),
                "ASE_Preburner_Assembly.stl"
            );

            Console.WriteLine("Generando ensamblaje completo del prequemador...");
            ExportPreburnerAssemblyToStl(outputPath);

            Console.WriteLine($"Ensamblaje exportado exitosamente en: {outputPath}");
        }

        public static void ExportPreburnerAssemblyToStl(string filePath)
        {
            // Parámetros de interfaz compartidos
            float fPlateRadiusMm = 80.0f;
            float fPlateThicknessMm = 12.0f;
            float fInjectorHoleRadiusMm = 6.5f;
            float fRingRadiusMm = 48.0f;
            float fFaceplateZ = 0.0f;

            // 1. Generación de la estructura principal (Cámara + Ducto + Manifold)
            Voxels voxPreburner = PreburnerGenerator.voxCreateASEPreburner(
                fHousingRadiusMm: 85.0f,
                fWallThicknessMm: 5.0f,
                fChamberLengthMm: 90.0f,
                fDuctLengthMm: 120.0f,
                fDuctExitRadiusMm: 45.0f,
                fManifoldMajorRadiusMm: 95.0f,
                fManifoldMinorRadiusMm: 22.0f
            );

            // 2. Generación de la placa inyectora con perforaciones
            Voxels voxFaceplate = InjectorPlateGenerator.voxCreateFaceplateWithInjectors(
                fPlateRadiusMm,
                fPlateThicknessMm,
                fInjectorHoleRadiusMm,

```

```

        fRingRadiusMm,
        fFaceplateZ
    );

    // 3. Generación del arreglo de 13 inyectores coaxiales
    Voxels voxInjectors = CoaxialInjectorGenerator.voxCreate13CoaxialInjectors(
        fInnerPostInnerRadiusMm: 2.5f,
        fInnerPostOuterRadiusMm: 3.5f,
        fOuterSleeveOuterRadiusMm: 6.4f,
        fInjectorLengthMm: 35.0f,
        fRingRadiusMm: fRingRadiusMm,
        fCenterZ: fFaceplateZ - 5.0f
    );

    // 4. Fusión CSG Voxelizada de todos los componentes
    Voxels voxAssembly = new Voxels();
    voxAssembly.Add(voxPreburner);
    voxAssembly.Add(voxFaceplate);
    voxAssembly.Add(voxInjectors);

    // 5. Mapeo a malla triangular y exportación a STL
    Mesh meshAssembly = new Mesh(voxAssembly);
    meshAssembly.SaveToStl(filePath);
}
}
}

```

Flujo de Procesamiento

- **Resolución de Voxel:** Se establece a 0.5 mm mediante `Library.Init(0.5f)` para mantener un equilibrio entre fidelidad geométrica y tiempo de renderizado.
- **Fusión Booleana:** `voxAssembly.Add()` une los volúmenes sin dejar intersecciones no deseadas o caras internas duplicadas.
- **Generación de Malla:** `Mesh(voxAssembly)` ejecuta un algoritmo de Marching Cubes sobre la representación de vóxeles unificada antes de escribir el archivo final STL.

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response:

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response: Esta función integra la cámara del prequemador, la placa inyectora perforada y los 13 elementos coaxiales en un único campo de vóxeles y exporta la malla combinada a un archivo STL.

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public class Program
    {
        public static void Main(string[] args)
        {
            // Inicialización de PikoGK con tamaño de voxel de 0.5 mm
            Library.Init(0.5f);

            string outputPath = Path.Combine(
                Environment.GetFolderPath(Environment.SpecialFolder.Desktop),
                "ASE_Preburner_Assembly.stl"
            );

            Console.WriteLine("Generando ensamblaje completo del prequemador...");
            ExportPreburnerAssemblyToStl(outputPath);

            Console.WriteLine($"Ensamblaje exportado exitosamente en: {outputPath}");
        }

        public static void ExportPreburnerAssemblyToStl(string filePath)
        {
            // Parámetros de interfaz compartidos
            float fPlateRadiusMm = 80.0f;
            float fPlateThicknessMm = 12.0f;

```



```

float fInjectorHoleRadiusMm = 6.5f;
float fRingRadiusMm = 48.0f;
float fFaceplateZ = 0.0f;

// 1. Generación de la estructura principal (Cámara + Ducto + Manifold)
Voxels voxPreburner = PreburnerGenerator.voxCreateASEPreburner(
    fHousingRadiusMm: 85.0f,
    fWallThicknessMm: 5.0f,
    fChamberLengthMm: 90.0f,
    fDuctLengthMm: 120.0f,
    fDuctExitRadiusMm: 45.0f,
    fManifoldMajorRadiusMm: 95.0f,
    fManifoldMinorRadiusMm: 22.0f
);

// 2. Generación de la placa inyectora con perforaciones
Voxels voxFaceplate = InjectorPlateGenerator.voxCreateFaceplateWithInjectors(
    fPlateRadiusMm,
    fPlateThicknessMm,
    fInjectorHoleRadiusMm,
    fRingRadiusMm,
    fFaceplateZ
);

// 3. Generación del arreglo de 13 inyectores coaxiales
Voxels voxInjectors = CoaxialInjectorGenerator.voxCreate13CoaxialInjectors(
    fInnerPostInnerRadiusMm: 2.5f,
    fInnerPostOuterRadiusMm: 3.5f,
    fOuterSleeveOuterRadiusMm: 6.4f,
    fInjectorLengthMm: 35.0f,
    fRingRadiusMm: fRingRadiusMm,
    fCenterZ: fFaceplateZ - 5.0f
);

// 4. Fusión CSG Voxelizada de todos los componentes
Voxels voxAssembly = new Voxels();
voxAssembly.Add(voxPreburner);
voxAssembly.Add(voxFaceplate);
voxAssembly.Add(voxInjectors);

// 5. Mapeo a malla triangular y exportación a STL
Mesh meshAssembly = new Mesh(voxAssembly);
meshAssembly.SaveToStl(filePath);
    }
}
}

```

Flujo de Procesamiento

- **Resolución de Voxel:** Se establece a 0.5 mm mediante `Library.Init(0.5f)` para mantener un equilibrio entre fidelidad geométrica y tiempo de renderizado.
- **Fusión Booleana:** `voxAssembly.Add()` une los volúmenes sin dejar intersecciones no deseadas o caras internas duplicadas.
- **Generación de Malla:** `Mesh(voxAssembly)` ejecuta un algoritmo de Marching Cubes sobre la representación de vóxeles unificada antes de escribir el archivo final STL.

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response:

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response: Esta función integra la cámara del prequemador, la placa inyectora perforada y los 13 elementos coaxiales en un único campo de vóxeles y exporta la malla combinada a un archivo STL.

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public class Program
    {
        public static void Main(string[] args)
    }
}

```

```

{
    // Inicialización de PikoGK con tamaño de vóxel de 0.5 mm
    Library.Init(0.5f);

    string outputPath = Path.Combine(
        Environment.GetFolderPath(Environment.SpecialFolder.Desktop),
        "ASE_Preburner_Assembly.stl"
    );

    Console.WriteLine("Generando ensamblaje completo del prequemador...");
    ExportPreburnerAssemblyToStl(outputPath);

    Console.WriteLine($"Ensamblaje exportado exitosamente en: {outputPath}");
}

public static void ExportPreburnerAssemblyToStl(string filePath)
{
    // Parámetros de interfaz compartidos
    float fPlateRadiusMm = 80.0f;
    float fPlateThicknessMm = 12.0f;
    float fInjectorHoleRadiusMm = 6.5f;
    float fRingRadiusMm = 48.0f;
    float fFaceplateZ = 0.0f;

    // 1. Generación de la estructura principal (Cámara + Ducto + Manifold)
    Voxels voxPreburner = PreburnerGenerator.voxCreateASEPreburner(
        fHousingRadiusMm: 85.0f,
        fWallThicknessMm: 5.0f,
        fChamberLengthMm: 90.0f,
        fDuctLengthMm: 120.0f,
        fDuctExitRadiusMm: 45.0f,
        fManifoldMajorRadiusMm: 95.0f,
        fManifoldMinorRadiusMm: 22.0f
    );

    // 2. Generación de la placa inyectora con perforaciones
    Voxels voxFaceplate = InjectorPlateGenerator.voxCreateFaceplateWithInjectors(
        fPlateRadiusMm,
        fPlateThicknessMm,
        fInjectorHoleRadiusMm,
        fRingRadiusMm,
        fFaceplateZ
    );

    // 3. Generación del arreglo de 13 inyectores coaxiales
    Voxels voxInjectors = CoaxialInjectorGenerator.voxCreate13CoaxialInjectors(
        fInnerPostInnerRadiusMm: 2.5f,
        fInnerPostOuterRadiusMm: 3.5f,
        fOuterSleeveOuterRadiusMm: 6.4f,
        fInjectorLengthMm: 35.0f,
        fRingRadiusMm: fRingRadiusMm,
        fCenterZ: fFaceplateZ - 5.0f
    );

    // 4. Fusión CSG Voxelizada de todos los componentes
    Voxels voxAssembly = new Voxels();
    voxAssembly.Add(voxPreburner);
    voxAssembly.Add(voxFaceplate);
    voxAssembly.Add(voxInjectors);

    // 5. Mapeo a malla triangular y exportación a STL
    Mesh meshAssembly = new Mesh(voxAssembly);
    meshAssembly.SaveToStl(filePath);
}
}
}

```

Flujo de Procesamiento

- **Resolución de Vóxel:** Se establece a 0.5 mm mediante `Library.Init(0.5f)` para mantener un equilibrio entre fidelidad geométrica y tiempo de renderizado.
- **Fusión Booleana:** `voxAssembly.Add()` une los volúmenes sin dejar intersecciones no deseadas o caras internas duplicadas.
- **Generación de Malla:** `Mesh(voxAssembly)` ejecuta un algoritmo de Marching Cubes sobre la representación de vóxeles unificada antes de escribir el archivo final STL.

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response:

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response: Esta función integra la cámara del prequemador, la placa inyectora perforada y los 13 elementos coaxiales en un único campo de véxeles y exporta la malla combinada a un archivo STL.

```
using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public class Program
    {
        public static void Main(string[] args)
        {
            // Inicialización de PikoGK con tamaño de voxel de 0.5 mm
            Library.Init(0.5f);

            string outputPath = Path.Combine(
                Environment.GetFolderPath(Environment.SpecialFolder.Desktop),
                "ASE_Preburner_Assembly.stl"
            );

            Console.WriteLine("Generando ensamblaje completo del prequemador...");
            ExportPreburnerAssemblyToStl(outputPath);

            Console.WriteLine($"Ensamblaje exportado exitosamente en: {outputPath}");
        }

        public static void ExportPreburnerAssemblyToStl(string filePath)
        {
            // Parámetros de interfaz compartidos
            float fPlateRadiusMm = 80.0f;
            float fPlateThicknessMm = 12.0f;
            float fInjectorHoleRadiusMm = 6.5f;
            float fRingRadiusMm = 48.0f;
            float fFaceplateZ = 0.0f;

            // 1. Generación de la estructura principal (Cámara + Ducto + Manifold)
            Voxels voxPreburner = PreburnerGenerator.voxCreateASEPreburner(
                fHousingRadiusMm: 85.0f,
                fWallThicknessMm: 5.0f,
                fChamberLengthMm: 90.0f,
                fDuctLengthMm: 120.0f,
                fDuctExitRadiusMm: 45.0f,
                fManifoldMajorRadiusMm: 95.0f,
                fManifoldMinorRadiusMm: 22.0f
            );

            // 2. Generación de la placa inyectora con perforaciones
            Voxels voxFaceplate = InjectorPlateGenerator.voxCreateFaceplateWithInjectors(
                fPlateRadiusMm,
                fPlateThicknessMm,
                fInjectorHoleRadiusMm,
                fRingRadiusMm,
                fFaceplateZ
            );

            // 3. Generación del arreglo de 13 inyectores coaxiales
            Voxels voxInjectors = CoaxialInjectorGenerator.voxCreate13CoaxialInjectors(
                fInnerPostInnerRadiusMm: 2.5f,
                fInnerPostOuterRadiusMm: 3.5f,
                fOuterSleeveOuterRadiusMm: 6.4f,
                fInjectorLengthMm: 35.0f,
                fRingRadiusMm: fRingRadiusMm,
                fCenterZ: fFaceplateZ - 5.0f
            );

            // 4. Fusión CSG Voxelizada de todos los componentes
            Voxels voxAssembly = new Voxels();
            voxAssembly.Add(voxPreburner);
            voxAssembly.Add(voxFaceplate);
        }
    }
}
```

```

        voxAssembly.Add(voxInjectors);

        // 5. Mapeo a malla triangular y exportación a STL
        Mesh meshAssembly = new Mesh(voxAssembly);
        meshAssembly.SaveToStl(filePath);
    }
}
}

```

Flujo de Procesamiento

- **Resolución de Vóxel:** Se establece a 0.5 mm mediante `Library.Init(0.5f)` para mantener un equilibrio entre fidelidad geométrica y tiempo de renderizado.
- **Fusión Booleana:** `voxAssembly.Add()` une los volúmenes sin dejar intersecciones no deseadas o caras internas duplicadas.
- **Generación de Malla:** `Mesh(voxAssembly)` ejecuta un algoritmo de Marching Cubes sobre la representación de vóxeles unificada antes de escribir el archivo final STL.

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response:

User prompt: Muestra la función principal de C# que integra la placa, los 13 inyectores y la cámara del prequemador en una sola malla y la exporta a un archivo STL usando PikoGK.

Response: Esta función integra la cámara del prequemador, la placa inyectora perforada y los 13 elementos coaxiales en un único campo de vóxeles y exporta la malla combinada a un archivo STL.

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public class Program
    {
        public static void Main(string[] args)
        {
            // Inicialización de PikoGK con tamaño de vóxel de 0.5 mm
            Library.Init(0.5f);

            string outputPath = Path.Combine(
                Environment.GetFolderPath(Environment.SpecialFolder.Desktop),
                "ASE_Preburner_Assembly.stl"
            );

            Console.WriteLine("Generando ensamblaje completo del prequemador...");
            ExportPreburnerAssemblyToStl(outputPath);

            Console.WriteLine($"Ensamblaje exportado exitosamente en: {outputPath}");
        }

        public static void ExportPreburnerAssemblyToStl(string filePath)
        {
            // Parámetros de interfaz compartidos
            float fPlateRadiusMm = 80.0f;
            float fPlateThicknessMm = 12.0f;
            float fInjectorHoleRadiusMm = 6.5f;
            float fRingRadiusMm = 48.0f;
            float fFaceplateZ = 0.0f;

            // 1. Generación de la estructura principal (Cámara + Ducto + Manifold)
            Voxels voxPreburner = PreburnerGenerator.voxCreateASEPreburner(
                fHousingRadiusMm: 85.0f,
                fWallThicknessMm: 5.0f,
                fChamberLengthMm: 90.0f,
                fDuctLengthMm: 120.0f,
                fDuctExitRadiusMm: 45.0f,
                fManifoldMajorRadiusMm: 95.0f,
                fManifoldMinorRadiusMm: 22.0f
            );

            // 2. Generación de la placa inyectora con perforaciones
            Voxels voxFaceplate = InjectorPlateGenerator.voxCreateFaceplateWithInjectors(
                fPlateRadiusMm,

```

```

        fPlateThicknessMm,
        fInjectorHoleRadiusMm,
        fRingRadiusMm,
        fFaceplateZ
    );

    // 3. Generación del arreglo de 13 inyectores coaxiales
    Voxels voxInjectors = CoaxialInjectorGenerator.voxCreate13CoaxialInjectors(
        fInnerPostInnerRadiusMm: 2.5f,
        fInnerPostOuterRadiusMm: 3.5f,
        fOuterSleeveOuterRadiusMm: 6.4f,
        fInjectorLengthMm: 35.0f,
        fRingRadiusMm: fRingRadiusMm,
        fCenterZ: fFaceplateZ - 5.0f
    );

    // 4. Fusión CSG Voxelizada de todos los componentes
    Voxels voxAssembly = new Voxels();
    voxAssembly.Add(voxPreburner);
    voxAssembly.Add(voxFaceplate);
    voxAssembly.Add(voxInjectors);

    // 5. Mapeo a malla triangular y exportación a STL
    Mesh meshAssembly = new Mesh(voxAssembly);
    meshAssembly.SaveToStl(filePath);
}
}
}

```

Flujo de Procesamiento

- **Resolución de Vóxel:** Se establece a 0.5 mm mediante `Library.Init(0.5f)` para mantener un equilibrio entre fidelidad geométrica y tiempo de renderizado.
- **Fusión Booleana:** `voxAssembly.Add()` une los volúmenes sin dejar intersecciones no deseadas o caras internas duplicadas.
- **Generación de Malla:** `Mesh(voxAssembly)` ejecuta un algoritmo de Marching Cubes sobre la representación de vóxeles unificada antes de escribir el archivo final STL.

User prompt: Muestra cómo aplicar operaciones de suavizado (blur/offset) en PikoGK sobre la malla del ensamblaje del prequemador para aliviar concentraciones de esfuerzo.

Response: Para aliviar concentraciones de esfuerzo en las uniones de componentes con ángulos vivos (como la raíz de los inyectores con la placa o la unión de la placa con la carcasa), en **PikoGK** se utiliza una secuencia de **operaciones morfométricas sobre el campo de vóxeles (SDF)**.

Este enfoque realiza un **Offset Positivo (Dilatación)** → **Filtro Gaussiano** → **Offset Negativo (Erosión)**, lo que genera redondeos (*fillets*) continuos de curvatura C^2 en las aristas internas sin alterar significativamente las dimensiones nominales exteriores.

Código C#: Módulo de Suavizado y Reducción de Esfuerzos

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class SmoothingUtils
    {
        /// <summary>
        /// Aplica redondeos (fillets) y suavizado volumétrico al ensamblaje
        /// para eliminar cantos vivos y reducir el factor de intensidad de esfuerzo (Kt).
        /// </summary>
        /// <param name="voxInput">Campo de vóxeles del ensamblaje rígido.</param>
        /// <param name="fFilletRadiusMm">Radio aproximado del redondeo (fillet) en mm.</param>
        /// <param name="fBlurSigmaMm">Intensidad del filtro gaussiano (típicamente 70-80% del radio).</param>
        /// <returns>Objeto Voxels con uniones suavizadas de curvatura continua.</returns>
        public static Voxels voxApplyStressReliefFillets(
            Voxels voxInput,
            float fFilletRadiusMm = 1.5f,
            float fBlurSigmaMm = 1.0f)
        {
            // Duplicar el volumen original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 1. DILATACIÓN (Offset Positivo):
            // Expande la superficie hacia afuera para que los rincones internos afilados
            // se desplacen a una zona accesible para el filtro espacial.

```

```

        voxSmoothed.DoOffset(fFilletRadiusMm);

        // 2. FILTRO GAUSSIANO (Blur):
        // Redondea las esquinas afiladas suavizando el campo de distancias implícito.
        voxSmoothed.DoGaussian(fBlurSigmaMm);

        // 3. EROSIÓN COMPENSATORIA (Offset Negativo):
        // Devuelve las paredes exteriores a su cota nominal, manteniendo
        // los empalmes redondeados (fillets) en las raíces internas.
        voxSmoothed.DoOffset(-fFilletRadiusMm);

        return voxSmoothed;
    }
}

public class ProgramWithSmoothing
{
    public static void Main(string[] args)
    {
        Library.Init(0.4f); // Mayor resolución (0.4mm) para capturar fillets finos

        string outputPath = Path.Combine(
            Environment.GetFolderPath(Environment.SpecialFolder.Desktop),
            "ASE_Preburner_Smoothed_Assembly.stl"
        );

        Console.WriteLine("Generando ensamblaje del prequemador...");

        // 1. Construir las piezas rígidas unidas en un solo objeto Voxels
        Voxels voxPreburner = PreburnerGenerator.voxCreateASEPreburner();
        Voxels voxFaceplate = InjectorPlateGenerator.voxCreateFaceplateWithInjectors();
        Voxels voxInjectors = CoaxialInjectorGenerator.voxCreate13CoaxialInjectors();

        Voxels voxRawAssembly = new Voxels();
        voxRawAssembly.Add(voxPreburner);
        voxRawAssembly.Add(voxFaceplate);
        voxRawAssembly.Add(voxInjectors);

        Console.WriteLine("Aplicando redondeos volumétricos (Stress-Relief Fillets)...");

        // 2. Aplicar el suavizado de uniones
        Voxels voxFinalAssembly = SmoothingUtils.voxApplyStressReliefFillets(
            voxRawAssembly,
            fFilletRadiusMm: 1.2f,
            fBlurSigmaMm: 0.9f
        );

        // 3. Exportar a STL
        Mesh meshFinal = new Mesh(voxFinalAssembly);
        meshFinal.SaveToStl(outputPath);

        Console.WriteLine($"Malla suavizada exportada en: {outputPath}");
    }
}

```

Principio de Funcionamiento en PikoGK

- 1. Eliminación de Cantos Vivos en Revestimientos:** Las juntas a 90° entre las mangas de los inyectores y la placa inyectora son puntos críticos de fallo por fatiga térmico-mecánica. El proceso de *Offset + Blur* reemplaza esa arista por una transición cónica/parabólica suave.
- 2. Independencia de la Topología:** A diferencia del modelado CAD basado en B-Rep (donde los redondeos fallan frecuentemente si las superficies se cruzan complexivamente), el método de vóxeles en PikoGK calcula los redondeos geoméricamente en el campo escalar 3D, garantizando un resultado sólido e imprimible (manifold) sin importar la complejidad del patrón de los 13 inyectores.

Response: Para integrar una estructura reticular (*lattice*) de enfriamiento o aligeramiento dentro de la pared del prequemador en PikoGK, se sigue la técnica de **muros emparedado (sandwich wall)**. Este método crea una piel interior impermeable, una piel exterior de soporte y una zona intermedia donde se inserta la matriz de entramado (*lattice*).

```

using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class LatticeCoolingGenerator
    {

```

```

/// <summary>
/// Genera una carcasa de prequemador con estructura reticular (lattice) integrando
/// canales de enfriamiento / refuerzo dentro del espesor de la pared.
/// </summary>
/// <param name="fInnerRadiusMm">Radio interno del paso de gases hot.</param>
/// <param name="fSkinThicknessMm">Espesor de las pieles interna y externa.</param>
/// <param name="fLatticeCoreThicknessMm">Espesor del núcleo donde habita el lattice.</param>
/// <param name="fHeightMm">Altura total del cuerpo del prequemador.</param>
/// <param name="fBeamRadiusMm">Radio de las vigas del lattice.</param>
/// <returns>Objeto Voxels con el prequemador de pared celular/lattice.</returns>
public static Voxels voxCreateLatticeCooledChamber(
    float fInnerRadiusMm = 45.0f,
    float fSkinThicknessMm = 1.2f,
    float fLatticeCoreThicknessMm = 6.0f,
    float fHeightMm = 90.0f,
    float fBeamRadiusMm = 0.8f)
{
    float fCoreInnerR = fInnerRadiusMm + fSkinThicknessMm;
    float fCoreOuterR = fCoreInnerR + fLatticeCoreThicknessMm;
    float fTotalOuterR = fCoreOuterR + fSkinThicknessMm;

    // 1. Crear el dominio del núcleo (Espacio anular reservado para el lattice)
    Voxels voxCoreDomain = voxCreateCylindricalShell(fCoreInnerR, fCoreOuterR, fHeightMm);

    // 2. Crear las pieles sólidas interna y externa (Paredes estancas de contención)
    Voxels voxInnerSkin = voxCreateCylindricalShell(fInnerRadiusMm, fCoreInnerR, fHeightMm);
    Voxels voxOuterSkin = voxCreateCylindricalShell(fCoreOuterR, fTotalOuterR, fHeightMm);

    // 3. Generar el entramado de vigas 3D (BCC / Octeto)
    Lattice latStructure = new Lattice();
    float fCellSize = 6.0f; // Tamaño de celda unitaria en mm

    for (float z = 0; z <= fHeightMm; z += fCellSize)
    {
        for (float r = fCoreInnerR - 2.0f; r <= fCoreOuterR + 2.0f; r += fCellSize)
        {
            int nStepsAround = (int)Math.Ceiling(2.0 * Math.PI * r / fCellSize);
            float fAngleStep = 2.0f * (float)Math.PI / nStepsAround;

            for (int i = 0; i < nStepsAround; i++)
            {
                float a1 = i * fAngleStep;
                float a2 = (i + 1) * fAngleStep;

                Vector3 p1 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z);
                Vector3 p2 = new Vector3((float)Math.Cos(a2) * r, (float)Math.Sin(a2) * r, z + fCellSize);
                Vector3 p3 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z + fCellSize);

                latStructure.AddBeam(p1, p2, fBeamRadiusMm, fBeamRadiusMm);
                latStructure.AddBeam(p1, p3, fBeamRadiusMm, fBeamRadiusMm);
            }
        }
    }

    // Convertir la red de vigas a campo de vóxeles
    Voxels voxLatticeVolume = new Voxels(latStructure);

    // 4. Recortar el lattice para ajustarlo exactamente al dominio anular interno
    voxLatticeVolume.Intersect(voxCoreDomain);

    // 5. Unir pieles estancas con el núcleo reticular
    Voxels voxChamberAssembly = new Voxels();
    voxChamberAssembly.Add(voxInnerSkin);
    voxChamberAssembly.Add(voxOuterSkin);
    voxChamberAssembly.Add(voxLatticeVolume);

    return voxChamberAssembly;
}

private static Voxels voxCreateCylindricalShell(float fRInner, float fROuter, float fHeight)
{
    Mesh meshOuter = Mesh.mshCreateCylinder(fROuter, fHeight);
    Mesh meshInner = Mesh.mshCreateCylinder(fRInner, fHeight + 2.0f);

    Voxels voxOuter = new Voxels(meshOuter);
    Voxels voxInner = new Voxels(meshInner);
    voxOuter.Subtract(voxInner);

    return voxOuter;
}

```

```
    }  
  }  
}
```

Arquitectura de la Solución en PikoGK

- **Recorte Booleano (Intersect):** El entramado de vigas se genera cubriendo la región tridimensional global y luego se aplica `voxLatticeVolume.Intersect(voxCoreDomain)`. Esto garantiza que ninguna viga protruya fuera de la superficie interior del prequemador ni sobresalga de la cubierta exterior.
- **Integridad Estructural:** El resultado combina la estanqueidad de las paredes continua (`voxInnerSkin` y `voxOuterSkin`) con el flujo fluido del refrigerante a través de los vacíos de la estructura celular interna.

Response: Las imágenes proporcionadas muestran referencias detalladas del diseño CAD y corte transversal de un motor de combustión por etapas de flujo completo (estilo SpaceX Raptor / FFSC), incluyendo las turbobombas de LOX y CH₄, prequemadores, distribuidores (*manifolds*), y el ensamblaje de la cámara principal y tobera.

A partir de los esquemas estructurales y las vistas explosionadas de los modelos 3D presentados, destacan tres elementos clave para continuar la construcción del modelo paramétrico en **PikoGK**:

Componentes Clave Identificados para Modelado

- **Distribuidor Toroidal (Injector Manifold):** Anillo de alimentación situado sobre la placa inyectora que garantiza la distribución uniforme de flujo hacia los 13 elementos coaxiales.
- **Conducto de Gases Calientes (Hot Gas Duct):** Tubo curvado de alta presión que conecta la descarga de la turbina del prequemador con la cabeza de la cámara principal.
- **Chaqueta de Enfriamiento y Bridas de Acople:** Grosor de pared variable en la cámara de combustión y anillos flangeados con patrón de pernos para fijación estructural.

Integración de Manifold Toroidal en C# (PikoGK)

A continuación se muestra cómo modelar el distribuidor toroidal (*manifold*) de alimentación y conectarlo a la cabeza del prequemador mediante operaciones booleanas volumétricas en PikoGK:

```
using System;  
using System.Numerics;  
using PikoGK;  
  
namespace RocketEngineDesign  
{  
    public static class ManifoldGenerator  
    {  
        /// <summary>  
        /// Genera un distribuidor toroidal (Torus Manifold) para alimentación de propelente  
        /// alineado con la placa inyectora.  
        /// </summary>  
        public static Voxels voxCreateToroidalManifold(  
            float fMajorRadiusMm = 65.0f,  
            float fOuterTubeRadiusMm = 14.0f,  
            float fWallThicknessMm = 3.0f,  
            float fCenterZ = 12.0f)  
        {  
            float fInnerTubeRadiusMm = fOuterTubeRadiusMm - fWallThicknessMm;  
  
            // 1. Toro exterior (geometría sólida)  
            Mesh meshOuterTorus = CreateTorusMesh(fMajorRadiusMm, fOuterTubeRadiusMm, fCenterZ);  
            Voxels voxManifold = new Voxels(meshOuterTorus);  
  
            // 2. Toro interior (vaciado del canal de fluido)  
            Mesh meshInnerTorus = CreateTorusMesh(fMajorRadiusMm, fInnerTubeRadiusMm, fCenterZ);  
            Voxels voxFluidChannel = new Voxels(meshInnerTorus);  
  
            // Perforación interna del distribuidor  
            voxManifold.Subtract(voxFluidChannel);  
  
            return voxManifold;  
        }  
  
        private static Mesh CreateTorusMesh(  
            float fRMajor, float fRMinor, float fCenterZ, int nStepsMajor = 48, int nStepsMinor = 24)  
        {  
            Mesh mesh = new Mesh();  
            float dPhi = 2.0f * (float)Math.PI / nStepsMajor;  
            float dTheta = 2.0f * (float)Math.PI / nStepsMinor;  
  
            for (int i = 0; i < nStepsMajor; i++)
```



```

    {
        float phi = i * dPhi;
        float cosPhi = (float)Math.Cos(phi);
        float sinPhi = (float)Math.Sin(phi);

        for (int j = 0; j < nStepsMinor; j++)
        {
            float theta = j * dTheta;
            float cosTheta = (float)Math.Cos(theta);
            float sinTheta = (float)Math.Sin(theta);

            float x = (fRMajor + fRMinor * cosTheta) * cosPhi;
            float y = (fRMajor + fRMinor * cosTheta) * sinPhi;
            float z = fCenterZ + fRMinor * sinTheta;

            mesh.nAddVertex(new Vector3(z, x, y));
        }
    }

    for (int i = 0; i < nStepsMajor; i++)
    {
        int iNext = (i + 1) % nStepsMajor;
        for (int j = 0; j < nStepsMinor; j++)
        {
            int jNext = (j + 1) % nStepsMinor;

            int idx00 = i * nStepsMinor + j;
            int idx10 = iNext * nStepsMinor + j;
            int idx01 = i * nStepsMinor + jNext;
            int idx11 = iNext * nStepsMinor + jNext;

            mesh.AddTriangle(idx00, idx10, idx11);
            mesh.AddTriangle(idx00, idx11, idx01);
        }
    }

    return mesh;
}
}
}
}

```

Response: Para integrar una estructura reticular (*lattice*) de enfriamiento o aligeramiento dentro de la pared del prequemador en PikoGK, se sigue la técnica de **muros emparedado (sandwich wall)**. Este método crea una piel interior impermeable, una piel exterior de soporte y una zona intermedia donde se inserta la matriz de entramado (*lattice*).

```

using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class LatticeCoolingGenerator
    {
        /// <summary>
        /// Genera una carcasa de prequemador con estructura reticular (lattice) integrando
        /// canales de enfriamiento / refuerzo dentro del espesor de la pared.
        /// </summary>
        /// <param name="fInnerRadiusMm">Radio interno del paso de gases hot.</param>
        /// <param name="fSkinThicknessMm">Espesor de las pieles interna y externa.</param>
        /// <param name="fLatticeCoreThicknessMm">Espesor del núcleo donde habita el lattice.</param>
        /// <param name="fHeightMm">Altura total del cuerpo del prequemador.</param>
        /// <param name="fBeamRadiusMm">Radio de las vigas del lattice.</param>
        /// <returns>Objeto Voxels con el prequemador de pared celular/lattice.</returns>
        public static Voxels voxCreateLatticeCooledChamber(
            float fInnerRadiusMm = 45.0f,
            float fSkinThicknessMm = 1.2f,
            float fLatticeCoreThicknessMm = 6.0f,
            float fHeightMm = 90.0f,
            float fBeamRadiusMm = 0.8f)
        {
            float fCoreInnerR = fInnerRadiusMm + fSkinThicknessMm;
            float fCoreOuterR = fCoreInnerR + fLatticeCoreThicknessMm;
            float fTotalOuterR = fCoreOuterR + fSkinThicknessMm;

            // 1. Crear el dominio del núcleo (Espacio anular reservado para el lattice)
            Voxels voxCoreDomain = voxCreateCylindricalShell(fCoreInnerR, fCoreOuterR, fHeightMm);

            // 2. Crear las pieles sólidas interna y externa (Paredes estancas de contención)

```

```

Voxels voxInnerSkin = voxCreateCylindricalShell(fInnerRadiusMm, fCoreInnerR, fHeightMm);
Voxels voxOuterSkin = voxCreateCylindricalShell(fCoreOuterR, fTotalOuterR, fHeightMm);

// 3. Generar el entramado de vigas 3D (BCC / Octeto)
Lattice latStructure = new Lattice();
float fCellSize = 6.0f; // Tamaño de celda unitaria en mm

for (float z = 0; z <= fHeightMm; z += fCellSize)
{
    for (float r = fCoreInnerR - 2.0f; r <= fCoreOuterR + 2.0f; r += fCellSize)
    {
        int nStepsAround = (int)Math.Ceiling(2.0 * Math.PI * r / fCellSize);
        float fAngleStep = 2.0f * (float)Math.PI / nStepsAround;

        for (int i = 0; i < nStepsAround; i++)
        {
            float a1 = i * fAngleStep;
            float a2 = (i + 1) * fAngleStep;

            Vector3 p1 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z);
            Vector3 p2 = new Vector3((float)Math.Cos(a2) * r, (float)Math.Sin(a2) * r, z + fCellSize);
            Vector3 p3 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z + fCellSize);

            latStructure.AddBeam(p1, p2, fBeamRadiusMm, fBeamRadiusMm);
            latStructure.AddBeam(p1, p3, fBeamRadiusMm, fBeamRadiusMm);
        }
    }
}

// Convertir la red de vigas a campo de vóxeles
Voxels voxLatticeVolume = new Voxels(latStructure);

// 4. Recortar el lattice para ajustarlo exactamente al dominio anular interno
voxLatticeVolume.Intersect(voxCoreDomain);

// 5. Unir pieles estancas con el núcleo reticular
Voxels voxChamberAssembly = new Voxels();
voxChamberAssembly.Add(voxInnerSkin);
voxChamberAssembly.Add(voxOuterSkin);
voxChamberAssembly.Add(voxLatticeVolume);

return voxChamberAssembly;
}

private static Voxels voxCreateCylindricalShell(float fRInner, float fROuter, float fHeight)
{
    Mesh meshOuter = Mesh.mshCreateCylinder(fROuter, fHeight);
    Mesh meshInner = Mesh.mshCreateCylinder(fRInner, fHeight + 2.0f);

    Voxels voxOuter = new Voxels(meshOuter);
    Voxels voxInner = new Voxels(meshInner);
    voxOuter.Subtract(voxInner);

    return voxOuter;
}
}
}

```

Arquitectura de la Solución en PikoGK

- **Recorte Booleano (Intersect):** El entramado de vigas se genera cubriendo la región tridimensional global y luego se aplica `voxLatticeVolume.Intersect(voxCoreDomain)`. Esto garantiza que ninguna viga protruya fuera de la superficie interior del prequemador ni sobresalga de la cubierta exterior.
- **Integridad Estructural:** El resultado combina la estanqueidad de las paredes continua (`voxInnerSkin` y `voxOuterSkin`) con el flujo fluido del refrigerante a través de los vacíos de la estructura celular interna.

Response: Las imágenes proporcionadas muestran referencias detalladas del diseño CAD y corte transversal de un motor de combustión por etapas de flujo completo (estilo SpaceX Raptor / FFSC), incluyendo las turbobombas de LOX y CH₄, prequemadores, distribuidores (*manifolds*), y el ensamblaje de la cámara principal y tobera.

A partir de los esquemas estructurales y las vistas explosionadas de los modelos 3D presentados, destacan tres elementos clave para continuar la construcción del modelo paramétrico en **PikoGK**:

Componentes Clave Identificados para Modelado

- **Distribuidor Toroidal (Injector Manifold):** Anillo de alimentación situado sobre la placa inyectora que garantiza la distribución uniforme de flujo hacia los 13 elementos coaxiales.
- **Conducto de Gases Calientes (Hot Gas Duct):** Tubo curvado de alta presión que conecta la descarga de la turbina del prequemador con la cabeza de la cámara principal.
- **Chaqueta de Enfriamiento y Bridas de Acople:** Grosor de pared variable en la cámara de combustión y anillos flangeados con patrón de pernos para fijación estructural.

Integración de Manifold Toroidal en C# (PikoGK)

A continuación se muestra cómo modelar el distribuidor toroidal (*manifold*) de alimentación y conectarlo a la cabeza del prequemador mediante operaciones booleanas volumétricas en PikoGK:

```
using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class ManifoldGenerator
    {
        /// <summary>
        /// Genera un distribuidor toroidal (Torus Manifold) para alimentación de propelente
        /// alineado con la placa inyectora.
        /// </summary>
        public static Voxels voxCreateToroidalManifold(
            float fMajorRadiusMm = 65.0f,
            float fOuterTubeRadiusMm = 14.0f,
            float fWallThicknessMm = 3.0f,
            float fCenterZ = 12.0f)
        {
            float fInnerTubeRadiusMm = fOuterTubeRadiusMm - fWallThicknessMm;

            // 1. Toro exterior (geometría sólida)
            Mesh meshOuterTorus = CreateTorusMesh(fMajorRadiusMm, fOuterTubeRadiusMm, fCenterZ);
            Voxels voxManifold = new Voxels(meshOuterTorus);

            // 2. Toro interior (vacío del canal de fluido)
            Mesh meshInnerTorus = CreateTorusMesh(fMajorRadiusMm, fInnerTubeRadiusMm, fCenterZ);
            Voxels voxFluidChannel = new Voxels(meshInnerTorus);

            // Perforación interna del distribuidor
            voxManifold.Subtract(voxFluidChannel);

            return voxManifold;
        }

        private static Mesh CreateTorusMesh(
            float fRMajor, float fRMinor, float fCenterZ, int nStepsMajor = 48, int nStepsMinor = 24)
        {
            Mesh mesh = new Mesh();
            float dPhi = 2.0f * (float)Math.PI / nStepsMajor;
            float dTheta = 2.0f * (float)Math.PI / nStepsMinor;

            for (int i = 0; i < nStepsMajor; i++)
            {
                float phi = i * dPhi;
                float cosPhi = (float)Math.Cos(phi);
                float sinPhi = (float)Math.Sin(phi);

                for (int j = 0; j < nStepsMinor; j++)
                {
                    float theta = j * dTheta;
                    float cosTheta = (float)Math.Cos(theta);
                    float sinTheta = (float)Math.Sin(theta);

                    float x = (fRMajor + fRMinor * cosTheta) * cosPhi;
                    float y = (fRMajor + fRMinor * cosTheta) * sinPhi;
                    float z = fCenterZ + fRMinor * sinTheta;

                    mesh.nAddVertex(new Vector3(z, x, y));
                }
            }

            for (int i = 0; i < nStepsMajor; i++)
            {
                int iNext = (i + 1) % nStepsMajor;
                for (int j = 0; j < nStepsMinor; j++)
                {

```

```

        int jNext = (j + 1) % nStepsMinor;

        int idx00 = i * nStepsMinor + j;
        int idx10 = iNext * nStepsMinor + j;
        int idx01 = i * nStepsMinor + jNext;
        int idx11 = iNext * nStepsMinor + jNext;

        mesh.AddTriangle(idx00, idx10, idx11);
        mesh.AddTriangle(idx00, idx11, idx01);
    }
}

return mesh;
}
}
}

```

Response: Para integrar una estructura reticular (*lattice*) de enfriamiento o aligeramiento dentro de la pared del prequemador en PikoGK, se sigue la técnica de **muros emparedado (sandwich wall)**. Este método crea una piel interior impermeable, una piel exterior de soporte y una zona intermedia donde se inserta la matriz de entramado (*lattice*).

```

using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class LatticeCoolingGenerator
    {
        /// <summary>
        /// Genera una carcasa de prequemador con estructura reticular (lattice) integrando
        /// canales de enfriamiento / refuerzo dentro del espesor de la pared.
        /// </summary>
        /// <param name="fInnerRadiusMm">Radio interno del paso de gases hot.</param>
        /// <param name="fSkinThicknessMm">Espesor de las pieles interna y externa.</param>
        /// <param name="fLatticeCoreThicknessMm">Espesor del núcleo donde habita el lattice.</param>
        /// <param name="fHeightMm">Altura total del cuerpo del prequemador.</param>
        /// <param name="fBeamRadiusMm">Radio de las vigas del lattice.</param>
        /// <returns>Objeto Voxels con el prequemador de pared celular/lattice.</returns>
        public static Voxels voxCreateLatticeCooledChamber(
            float fInnerRadiusMm = 45.0f,
            float fSkinThicknessMm = 1.2f,
            float fLatticeCoreThicknessMm = 6.0f,
            float fHeightMm = 90.0f,
            float fBeamRadiusMm = 0.8f)
        {
            float fCoreInnerR = fInnerRadiusMm + fSkinThicknessMm;
            float fCoreOuterR = fCoreInnerR + fLatticeCoreThicknessMm;
            float fTotalOuterR = fCoreOuterR + fSkinThicknessMm;

            // 1. Crear el dominio del núcleo (Espacio anular reservado para el lattice)
            Voxels voxCoreDomain = voxCreateCylindricalShell(fCoreInnerR, fCoreOuterR, fHeightMm);

            // 2. Crear las pieles sólidas interna y externa (Paredes estancas de contención)
            Voxels voxInnerSkin = voxCreateCylindricalShell(fInnerRadiusMm, fCoreInnerR, fHeightMm);
            Voxels voxOuterSkin = voxCreateCylindricalShell(fCoreOuterR, fTotalOuterR, fHeightMm);

            // 3. Generar el entramado de vigas 3D (BCC / Octeto)
            Lattice latStructure = new Lattice();
            float fCellSize = 6.0f; // Tamaño de celda unitaria en mm

            for (float z = 0; z <= fHeightMm; z += fCellSize)
            {
                for (float r = fCoreInnerR - 2.0f; r <= fCoreOuterR + 2.0f; r += fCellSize)
                {
                    int nStepsAround = (int)Math.Ceiling(2.0 * Math.PI * r / fCellSize);
                    float fAngleStep = 2.0f * (float)Math.PI / nStepsAround;

                    for (int i = 0; i < nStepsAround; i++)
                    {
                        float a1 = i * fAngleStep;
                        float a2 = (i + 1) * fAngleStep;

                        Vector3 p1 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z);
                        Vector3 p2 = new Vector3((float)Math.Cos(a2) * r, (float)Math.Sin(a2) * r, z + fCellSize);
                        Vector3 p3 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z + fCellSize);

                        latStructure.AddBeam(p1, p2, fBeamRadiusMm, fBeamRadiusMm);
                    }
                }
            }

            return voxCoreDomain + voxInnerSkin + voxOuterSkin + latStructure;
        }
    }
}

```

```

        latStructure.AddBeam(p1, p3, fBeamRadiusMm, fBeamRadiusMm);
    }
}

// Convertir la red de vigas a campo de vóxeles
Voxels voxLatticeVolume = new Voxels(latStructure);

// 4. Recortar el lattice para ajustarlo exactamente al dominio anular interno
voxLatticeVolume.Intersect(voxCoreDomain);

// 5. Unir pieles estancas con el núcleo reticular
Voxels voxChamberAssembly = new Voxels();
voxChamberAssembly.Add(voxInnerSkin);
voxChamberAssembly.Add(voxOuterSkin);
voxChamberAssembly.Add(voxLatticeVolume);

return voxChamberAssembly;
}

private static Voxels voxCreateCylindricalShell(float fRInner, float fROuter, float fHeight)
{
    Mesh meshOuter = Mesh.mshCreateCylinder(fROuter, fHeight);
    Mesh meshInner = Mesh.mshCreateCylinder(fRInner, fHeight + 2.0f);

    Voxels voxOuter = new Voxels(meshOuter);
    Voxels voxInner = new Voxels(meshInner);
    voxOuter.Subtract(voxInner);

    return voxOuter;
}
}
}

```

Arquitectura de la Solución en PikoGK

- **Recorte Booleano (Intersect):** El entramado de vigas se genera cubriendo la región tridimensional global y luego se aplica `voxLatticeVolume.Intersect(voxCoreDomain)`. Esto garantiza que ninguna viga protruya fuera de la superficie interior del prequemador ni sobresalga de la cubierta exterior.
- **Integridad Estructural:** El resultado combina la estanqueidad de las paredes continua (`voxInnerSkin` y `voxOuterSkin`) con el flujo fluido del refrigerante a través de los vacíos de la estructura celular interna.

Response: Las imágenes proporcionadas muestran referencias detalladas del diseño CAD y corte transversal de un motor de combustión por etapas de flujo completo (estilo SpaceX Raptor / FFSC), incluyendo las turbobombas de LOX y CH₄, prequemadores, distribuidores (*manifolds*), y el ensamblaje de la cámara principal y tobera.

A partir de los esquemas estructurales y las vistas explosionadas de los modelos 3D presentados, destacan tres elementos clave para continuar la construcción del modelo paramétrico en **PikoGK**:

Componentes Clave Identificados para Modelado

- **Distribuidor Toroidal (Injector Manifold):** Anillo de alimentación situado sobre la placa inyectora que garantiza la distribución uniforme de flujo hacia los 13 elementos coaxiales.
- **Conducto de Gases Calientes (Hot Gas Duct):** Tubo curvado de alta presión que conecta la descarga de la turbina del prequemador con la cabeza de la cámara principal.
- **Chaqueta de Enfriamiento y Bridas de Acople:** Grosor de pared variable en la cámara de combustión y anillos flangeados con patrón de pernos para fijación estructural.

Integración de Manifold Toroidal en C# (PikoGK)

A continuación se muestra cómo modelar el distribuidor toroidal (*manifold*) de alimentación y conectarlo a la cabeza del prequemador mediante operaciones booleanas volumétricas en PikoGK:

```

using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class ManifoldGenerator
    {
        /// <summary>
        /// Genera un distribuidor toroidal (Torus Manifold) para alimentación de propelente
        /// alineado con la placa inyectora.
        /// </summary>

```

```

public static Voxels voxCreateToroidalManifold(
    float fMajorRadiusMm = 65.0f,
    float fOuterTubeRadiusMm = 14.0f,
    float fWallThicknessMm = 3.0f,
    float fCenterZ = 12.0f)
{
    float fInnerTubeRadiusMm = fOuterTubeRadiusMm - fWallThicknessMm;

    // 1. Toro exterior (geometría sólida)
    Mesh meshOuterTorus = CreateTorusMesh(fMajorRadiusMm, fOuterTubeRadiusMm, fCenterZ);
    Voxels voxManifold = new Voxels(meshOuterTorus);

    // 2. Toro interior (vacío del canal de fluido)
    Mesh meshInnerTorus = CreateTorusMesh(fMajorRadiusMm, fInnerTubeRadiusMm, fCenterZ);
    Voxels voxFluidChannel = new Voxels(meshInnerTorus);

    // Perforación interna del distribuidor
    voxManifold.Subtract(voxFluidChannel);

    return voxManifold;
}

private static Mesh CreateTorusMesh(
    float fRMajor, float fRMinor, float fCenterZ, int nStepsMajor = 48, int nStepsMinor = 24)
{
    Mesh mesh = new Mesh();
    float dPhi = 2.0f * (float)Math.PI / nStepsMajor;
    float dTheta = 2.0f * (float)Math.PI / nStepsMinor;

    for (int i = 0; i < nStepsMajor; i++)
    {
        float phi = i * dPhi;
        float cosPhi = (float)Math.Cos(phi);
        float sinPhi = (float)Math.Sin(phi);

        for (int j = 0; j < nStepsMinor; j++)
        {
            float theta = j * dTheta;
            float cosTheta = (float)Math.Cos(theta);
            float sinTheta = (float)Math.Sin(theta);

            float x = (fRMajor + fRMinor * cosTheta) * cosPhi;
            float y = (fRMajor + fRMinor * cosTheta) * sinPhi;
            float z = fCenterZ + fRMinor * sinTheta;

            mesh.nAddVertex(new Vector3(z, x, y));
        }
    }

    for (int i = 0; i < nStepsMajor; i++)
    {
        int iNext = (i + 1) % nStepsMajor;
        for (int j = 0; j < nStepsMinor; j++)
        {
            int jNext = (j + 1) % nStepsMinor;

            int idx00 = i * nStepsMinor + j;
            int idx10 = iNext * nStepsMinor + j;
            int idx01 = i * nStepsMinor + jNext;
            int idx11 = iNext * nStepsMinor + jNext;

            mesh.AddTriangle(idx00, idx10, idx11);
            mesh.AddTriangle(idx00, idx11, idx01);
        }
    }

    return mesh;
}
}
}

```

Response: Para integrar una estructura reticular (*lattice*) de enfriamiento o aligeramiento dentro de la pared del prequemador en PikoGK, se sigue la técnica de **muros emparedado (sandwich wall)**. Este método crea una piel interior impermeable, una piel exterior de soporte y una zona intermedia donde se inserta la matriz de entramado (*lattice*).

```

using System;
using System.Numerics;
using PikoGK;

```

```

namespace RocketEngineDesign
{
    public static class LatticeCoolingGenerator
    {
        /// <summary>
        /// Genera una carcasa de prequemador con estructura reticular (lattice) integrando
        /// canales de enfriamiento / refuerzo dentro del espesor de la pared.
        /// </summary>
        /// <param name="fInnerRadiusMm">Radio interno del paso de gases hot.</param>
        /// <param name="fSkinThicknessMm">Espesor de las pieles interna y externa.</param>
        /// <param name="fLatticeCoreThicknessMm">Espesor del núcleo donde habita el lattice.</param>
        /// <param name="fHeightMm">Altura total del cuerpo del prequemador.</param>
        /// <param name="fBeamRadiusMm">Radio de las vigas del lattice.</param>
        /// <returns>Objeto Voxels con el prequemador de pared celular/lattice.</returns>
        public static Voxels voxCreateLatticeCooledChamber(
            float fInnerRadiusMm = 45.0f,
            float fSkinThicknessMm = 1.2f,
            float fLatticeCoreThicknessMm = 6.0f,
            float fHeightMm = 90.0f,
            float fBeamRadiusMm = 0.8f)
        {
            float fCoreInnerR = fInnerRadiusMm + fSkinThicknessMm;
            float fCoreOuterR = fCoreInnerR + fLatticeCoreThicknessMm;
            float fTotalOuterR = fCoreOuterR + fSkinThicknessMm;

            // 1. Crear el dominio del núcleo (Espacio anular reservado para el lattice)
            Voxels voxCoreDomain = voxCreateCylindricalShell(fCoreInnerR, fCoreOuterR, fHeightMm);

            // 2. Crear las pieles sólidas interna y externa (Paredes estancas de contención)
            Voxels voxInnerSkin = voxCreateCylindricalShell(fInnerRadiusMm, fCoreInnerR, fHeightMm);
            Voxels voxOuterSkin = voxCreateCylindricalShell(fCoreOuterR, fTotalOuterR, fHeightMm);

            // 3. Generar el entramado de vigas 3D (BCC / Octeto)
            Lattice latStructure = new Lattice();
            float fCellSize = 6.0f; // Tamaño de celda unitaria en mm

            for (float z = 0; z <= fHeightMm; z += fCellSize)
            {
                for (float r = fCoreInnerR - 2.0f; r <= fCoreOuterR + 2.0f; r += fCellSize)
                {
                    int nStepsAround = (int)Math.Ceiling(2.0 * Math.PI * r / fCellSize);
                    float fAngleStep = 2.0f * (float)Math.PI / nStepsAround;

                    for (int i = 0; i < nStepsAround; i++)
                    {
                        float a1 = i * fAngleStep;
                        float a2 = (i + 1) * fAngleStep;

                        Vector3 p1 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z);
                        Vector3 p2 = new Vector3((float)Math.Cos(a2) * r, (float)Math.Sin(a2) * r, z + fCellSize);
                        Vector3 p3 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z + fCellSize);

                        latStructure.AddBeam(p1, p2, fBeamRadiusMm, fBeamRadiusMm);
                        latStructure.AddBeam(p1, p3, fBeamRadiusMm, fBeamRadiusMm);
                    }
                }
            }

            // Convertir la red de vigas a campo de vóxeles
            Voxels voxLatticeVolume = new Voxels(latStructure);

            // 4. Recortar el lattice para ajustarlo exactamente al dominio anular interno
            voxLatticeVolume.Intersect(voxCoreDomain);

            // 5. Unir pieles estancas con el núcleo reticular
            Voxels voxChamberAssembly = new Voxels();
            voxChamberAssembly.Add(voxInnerSkin);
            voxChamberAssembly.Add(voxOuterSkin);
            voxChamberAssembly.Add(voxLatticeVolume);

            return voxChamberAssembly;
        }

        private static Voxels voxCreateCylindricalShell(float fRInner, float fROuter, float fHeight)
        {
            Mesh meshOuter = Mesh.mshCreateCylinder(fROuter, fHeight);
            Mesh meshInner = Mesh.mshCreateCylinder(fRInner, fHeight + 2.0f);

```

```

        Voxels voxOuter = new Voxels(meshOuter);
        Voxels voxInner = new Voxels(meshInner);
        voxOuter.Subtract(voxInner);

        return voxOuter;
    }
}

```

Arquitectura de la Solución en PikoGK

- **Recorte Booleano (Intersect):** El entramado de vigas se genera cubriendo la región tridimensional global y luego se aplica `voxLatticeVolume.Intersect(voxCoreDomain)`. Esto garantiza que ninguna viga protruya fuera de la superficie interior del prequemador ni sobresalga de la cubierta exterior.
- **Integridad Estructural:** El resultado combina la estanqueidad de las paredes continua (`voxInnerSkin` y `voxOuterSkin`) con el flujo fluido del refrigerante a través de los vacíos de la estructura celular interna.

Response: Las imágenes proporcionadas muestran referencias detalladas del diseño CAD y corte transversal de un motor de combustión por etapas de flujo completo (estilo SpaceX Raptor / FFSC), incluyendo las turbobombas de LOX y CH₄, prequemadores, distribuidores (*manifolds*), y el ensamblaje de la cámara principal y tobera.

A partir de los esquemas estructurales y las vistas explosionadas de los modelos 3D presentados, destacan tres elementos clave para continuar la construcción del modelo paramétrico en **PikoGK**:

Componentes Clave Identificados para Modelado

- **Distribuidor Toroidal (Injector Manifold):** Anillo de alimentación situado sobre la placa inyectora que garantiza la distribución uniforme de flujo hacia los 13 elementos coaxiales.
- **Conducto de Gases Calientes (Hot Gas Duct):** Tubo curvado de alta presión que conecta la descarga de la turbina del prequemador con la cabeza de la cámara principal.
- **Chaqueta de Enfriamiento y Bidas de Acople:** Grosor de pared variable en la cámara de combustión y anillos flangeados con patrón de pernos para fijación estructural.

Integración de Manifold Toroidal en C# (PikoGK)

A continuación se muestra cómo modelar el distribuidor toroidal (*manifold*) de alimentación y conectarlo a la cabeza del prequemador mediante operaciones booleanas volumétricas en PikoGK:

```

using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class ManifoldGenerator
    {
        /// <summary>
        /// Genera un distribuidor toroidal (Torus Manifold) para alimentación de propelente
        /// alineado con la placa inyectora.
        /// </summary>
        public static Voxels voxCreateToroidalManifold(
            float fMajorRadiusMm = 65.0f,
            float fOuterTubeRadiusMm = 14.0f,
            float fWallThicknessMm = 3.0f,
            float fCenterZ = 12.0f)
        {
            float fInnerTubeRadiusMm = fOuterTubeRadiusMm - fWallThicknessMm;

            // 1. Toro exterior (geometría sólida)
            Mesh meshOuterTorus = CreateTorusMesh(fMajorRadiusMm, fOuterTubeRadiusMm, fCenterZ);
            Voxels voxManifold = new Voxels(meshOuterTorus);

            // 2. Toro interior (vaciado del canal de fluido)
            Mesh meshInnerTorus = CreateTorusMesh(fMajorRadiusMm, fInnerTubeRadiusMm, fCenterZ);
            Voxels voxFluidChannel = new Voxels(meshInnerTorus);

            // Perforación interna del distribuidor
            voxManifold.Subtract(voxFluidChannel);

            return voxManifold;
        }

        private static Mesh CreateTorusMesh(
            float fRMajor, float fRMinor, float fCenterZ, int nStepsMajor = 48, int nStepsMinor = 24)
        {

```



```

Mesh mesh = new Mesh();
float dPhi = 2.0f * (float)Math.PI / nStepsMajor;
float dTheta = 2.0f * (float)Math.PI / nStepsMinor;

for (int i = 0; i < nStepsMajor; i++)
{
    float phi = i * dPhi;
    float cosPhi = (float)Math.Cos(phi);
    float sinPhi = (float)Math.Sin(phi);

    for (int j = 0; j < nStepsMinor; j++)
    {
        float theta = j * dTheta;
        float cosTheta = (float)Math.Cos(theta);
        float sinTheta = (float)Math.Sin(theta);

        float x = (fRMajor + fRMinor * cosTheta) * cosPhi;
        float y = (fRMajor + fRMinor * cosTheta) * sinPhi;
        float z = fCenterZ + fRMinor * sinTheta;

        mesh.nAddVertex(new Vector3(z, x, y));
    }
}

for (int i = 0; i < nStepsMajor; i++)
{
    int iNext = (i + 1) % nStepsMajor;
    for (int j = 0; j < nStepsMinor; j++)
    {
        int jNext = (j + 1) % nStepsMinor;

        int idx00 = i * nStepsMinor + j;
        int idx10 = iNext * nStepsMinor + j;
        int idx01 = i * nStepsMinor + jNext;
        int idx11 = iNext * nStepsMinor + jNext;

        mesh.AddTriangle(idx00, idx10, idx11);
        mesh.AddTriangle(idx00, idx11, idx01);
    }
}

return mesh;
}
}
}

```

Response: Para integrar una estructura reticular (*lattice*) de enfriamiento o aligeramiento dentro de la pared del prequemador en PikoGK, se sigue la técnica de **muros emparedado (sandwich wall)**. Este método crea una piel interior impermeable, una piel exterior de soporte y una zona intermedia donde se inserta la matriz de entramado (*lattice*).

```

using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class LatticeCoolingGenerator
    {
        /// <summary>
        /// Genera una carcasa de prequemador con estructura reticular (lattice) integrando
        /// canales de enfriamiento / refuerzo dentro del espesor de la pared.
        /// </summary>
        /// <param name="fInnerRadiusMm">Radio interno del paso de gases hot.</param>
        /// <param name="fSkinThicknessMm">Espesor de las pieles interna y externa.</param>
        /// <param name="fLatticeCoreThicknessMm">Espesor del núcleo donde habita el lattice.</param>
        /// <param name="fHeightMm">Altura total del cuerpo del prequemador.</param>
        /// <param name="fBeamRadiusMm">Radio de las vigas del lattice.</param>
        /// <returns>Objeto Voxels con el prequemador de pared celular/lattice.</returns>
        public static Voxels voxCreateLatticeCooledChamber(
            float fInnerRadiusMm = 45.0f,
            float fSkinThicknessMm = 1.2f,
            float fLatticeCoreThicknessMm = 6.0f,
            float fHeightMm = 90.0f,
            float fBeamRadiusMm = 0.8f)
        {
            float fCoreInnerR = fInnerRadiusMm + fSkinThicknessMm;
            float fCoreOuterR = fCoreInnerR + fLatticeCoreThicknessMm;
            float fTotalOuterR = fCoreOuterR + fSkinThicknessMm;

```

```

// 1. Crear el dominio del núcleo (Espacio anular reservado para el lattice)
Voxels voxCoreDomain = voxCreateCylindricalShell(fCoreInnerR, fCoreOuterR, fHeightMm);

// 2. Crear las pieles sólidas interna y externa (Paredes estancas de contención)
Voxels voxInnerSkin = voxCreateCylindricalShell(fInnerRadiusMm, fCoreInnerR, fHeightMm);
Voxels voxOuterSkin = voxCreateCylindricalShell(fCoreOuterR, fTotalOuterR, fHeightMm);

// 3. Generar el entramado de vigas 3D (BCC / Octeto)
Lattice latStructure = new Lattice();
float fCellSize = 6.0f; // Tamaño de celda unitaria en mm

for (float z = 0; z <= fHeightMm; z += fCellSize)
{
    for (float r = fCoreInnerR - 2.0f; r <= fCoreOuterR + 2.0f; r += fCellSize)
    {
        int nStepsAround = (int)Math.Ceiling(2.0 * Math.PI * r / fCellSize);
        float fAngleStep = 2.0f * (float)Math.PI / nStepsAround;

        for (int i = 0; i < nStepsAround; i++)
        {
            float a1 = i * fAngleStep;
            float a2 = (i + 1) * fAngleStep;

            Vector3 p1 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z);
            Vector3 p2 = new Vector3((float)Math.Cos(a2) * r, (float)Math.Sin(a2) * r, z + fCellSize);
            Vector3 p3 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z + fCellSize);

            latStructure.AddBeam(p1, p2, fBeamRadiusMm, fBeamRadiusMm);
            latStructure.AddBeam(p1, p3, fBeamRadiusMm, fBeamRadiusMm);
        }
    }
}

// Convertir la red de vigas a campo de vóxeles
Voxels voxLatticeVolume = new Voxels(latStructure);

// 4. Recortar el lattice para ajustarlo exactamente al dominio anular interno
voxLatticeVolume.Intersect(voxCoreDomain);

// 5. Unir pieles estancas con el núcleo reticular
Voxels voxChamberAssembly = new Voxels();
voxChamberAssembly.Add(voxInnerSkin);
voxChamberAssembly.Add(voxOuterSkin);
voxChamberAssembly.Add(voxLatticeVolume);

return voxChamberAssembly;
}

private static Voxels voxCreateCylindricalShell(float fRInner, float fROuter, float fHeight)
{
    Mesh meshOuter = Mesh.mshCreateCylinder(fROuter, fHeight);
    Mesh meshInner = Mesh.mshCreateCylinder(fRInner, fHeight + 2.0f);

    Voxels voxOuter = new Voxels(meshOuter);
    Voxels voxInner = new Voxels(meshInner);
    voxOuter.Subtract(voxInner);

    return voxOuter;
}
}
}

```

Arquitectura de la Solución en PikoGK

- **Recorte Booleano (Intersect):** El entramado de vigas se genera cubriendo la región tridimensional global y luego se aplica `voxLatticeVolume.Intersect(voxCoreDomain)`. Esto garantiza que ninguna viga protruya fuera de la superficie interior del prequemador ni sobresalga de la cubierta exterior.
- **Integridad Estructural:** El resultado combina la estanqueidad de las paredes continua (`voxInnerSkin` y `voxOuterSkin`) con el flujo fluido del refrigerante a través de los vacíos de la estructura celular interna.

Response: Las imágenes proporcionadas muestran referencias detalladas del diseño CAD y corte transversal de un motor de combustión por etapas de flujo completo (estilo SpaceX Raptor / FFSC), incluyendo las turbobombas de LOX y CH₄, prequemadores, distribuidores (*manifolds*), y el ensamblaje de la cámara principal y tobera.

A partir de los esquemas estructurales y las vistas explosionadas de los modelos 3D presentados, destacan tres elementos clave para continuar la construcción del modelo paramétrico en **PikoGK**:

Componentes Clave Identificados para Modelado

- **Distribuidor Toroidal (Injector Manifold):** Anillo de alimentación situado sobre la placa inyectora que garantiza la distribución uniforme de flujo hacia los 13 elementos coaxiales.
- **Conducto de Gases Calientes (Hot Gas Duct):** Tubo curvado de alta presión que conecta la descarga de la turbina del prequemador con la cabeza de la cámara principal.
- **Chaqueta de Enfriamiento y Bridas de Acople:** Grosor de pared variable en la cámara de combustión y anillos flangeados con patrón de pernos para fijación estructural.

Integración de Manifold Toroidal en C# (PikoGK)

A continuación se muestra cómo modelar el distribuidor toroidal (*manifold*) de alimentación y conectarlo a la cabeza del prequemador mediante operaciones booleanas volumétricas en PikoGK:

```
using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class ManifoldGenerator
    {
        /// <summary>
        /// Genera un distribuidor toroidal (Torus Manifold) para alimentación de propelente
        /// alineado con la placa inyectora.
        /// </summary>
        public static Voxels voxCreateToroidalManifold(
            float fMajorRadiusMm = 65.0f,
            float fOuterTubeRadiusMm = 14.0f,
            float fWallThicknessMm = 3.0f,
            float fCenterZ = 12.0f)
        {
            float fInnerTubeRadiusMm = fOuterTubeRadiusMm - fWallThicknessMm;

            // 1. Toro exterior (geometría sólida)
            Mesh meshOuterTorus = CreateTorusMesh(fMajorRadiusMm, fOuterTubeRadiusMm, fCenterZ);
            Voxels voxManifold = new Voxels(meshOuterTorus);

            // 2. Toro interior (vacío del canal de fluido)
            Mesh meshInnerTorus = CreateTorusMesh(fMajorRadiusMm, fInnerTubeRadiusMm, fCenterZ);
            Voxels voxFluidChannel = new Voxels(meshInnerTorus);

            // Perforación interna del distribuidor
            voxManifold.Subtract(voxFluidChannel);

            return voxManifold;
        }

        private static Mesh CreateTorusMesh(
            float fRMajor, float fRMinor, float fCenterZ, int nStepsMajor = 48, int nStepsMinor = 24)
        {
            Mesh mesh = new Mesh();
            float dPhi = 2.0f * (float)Math.PI / nStepsMajor;
            float dTheta = 2.0f * (float)Math.PI / nStepsMinor;

            for (int i = 0; i < nStepsMajor; i++)
            {
                float phi = i * dPhi;
                float cosPhi = (float)Math.Cos(phi);
                float sinPhi = (float)Math.Sin(phi);

                for (int j = 0; j < nStepsMinor; j++)
                {
                    float theta = j * dTheta;
                    float cosTheta = (float)Math.Cos(theta);
                    float sinTheta = (float)Math.Sin(theta);

                    float x = (fRMajor + fRMinor * cosTheta) * cosPhi;
                    float y = (fRMajor + fRMinor * cosTheta) * sinPhi;
                    float z = fCenterZ + fRMinor * sinTheta;

                    mesh.nAddVertex(new Vector3(z, x, y));
                }
            }
        }
    }
}
```

```

for (int i = 0; i < nStepsMajor; i++)
{
    int iNext = (i + 1) % nStepsMajor;
    for (int j = 0; j < nStepsMinor; j++)
    {
        int jNext = (j + 1) % nStepsMinor;

        int idx00 = i * nStepsMinor + j;
        int idx10 = iNext * nStepsMinor + j;
        int idx01 = i * nStepsMinor + jNext;
        int idx11 = iNext * nStepsMinor + jNext;

        mesh.AddTriangle(idx00, idx10, idx11);
        mesh.AddTriangle(idx00, idx11, idx01);
    }
}

return mesh;
}
}
}

```

Response: Para integrar una estructura reticular (*lattice*) de enfriamiento o aligeramiento dentro de la pared del prequemador en PikoGK, se sigue la técnica de **muros emparedado (sandwich wall)**. Este método crea una piel interior impermeable, una piel exterior de soporte y una zona intermedia donde se inserta la matriz de entramado (*lattice*).

```

using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class LatticeCoolingGenerator
    {
        /// <summary>
        /// Genera una carcasa de prequemador con estructura reticular (lattice) integrando
        /// canales de enfriamiento / refuerzo dentro del espesor de la pared.
        /// </summary>
        /// <param name="fInnerRadiusMm">Radio interno del paso de gases hot.</param>
        /// <param name="fSkinThicknessMm">Espesor de las pieles interna y externa.</param>
        /// <param name="fLatticeCoreThicknessMm">Espesor del núcleo donde habita el lattice.</param>
        /// <param name="fHeightMm">Altura total del cuerpo del prequemador.</param>
        /// <param name="fBeamRadiusMm">Radio de las vigas del lattice.</param>
        /// <returns>Objeto Voxels con el prequemador de pared celular/lattice.</returns>
        public static Voxels voxCreateLatticeCooledChamber(
            float fInnerRadiusMm = 45.0f,
            float fSkinThicknessMm = 1.2f,
            float fLatticeCoreThicknessMm = 6.0f,
            float fHeightMm = 90.0f,
            float fBeamRadiusMm = 0.8f)
        {
            float fCoreInnerR = fInnerRadiusMm + fSkinThicknessMm;
            float fCoreOuterR = fCoreInnerR + fLatticeCoreThicknessMm;
            float fTotalOuterR = fCoreOuterR + fSkinThicknessMm;

            // 1. Crear el dominio del núcleo (Espacio anular reservado para el lattice)
            Voxels voxCoreDomain = voxCreateCylindricalShell(fCoreInnerR, fCoreOuterR, fHeightMm);

            // 2. Crear las pieles sólidas interna y externa (Paredes estancas de contención)
            Voxels voxInnerSkin = voxCreateCylindricalShell(fInnerRadiusMm, fCoreInnerR, fHeightMm);
            Voxels voxOuterSkin = voxCreateCylindricalShell(fCoreOuterR, fTotalOuterR, fHeightMm);

            // 3. Generar el entramado de vigas 3D (BCC / Octeto)
            Lattice latStructure = new Lattice();
            float fCellSize = 6.0f; // Tamaño de celda unitaria en mm

            for (float z = 0; z <= fHeightMm; z += fCellSize)
            {
                for (float r = fCoreInnerR - 2.0f; r <= fCoreOuterR + 2.0f; r += fCellSize)
                {
                    int nStepsAround = (int)Math.Ceiling(2.0 * Math.PI * r / fCellSize);
                    float fAngleStep = 2.0f * (float)Math.PI / nStepsAround;

                    for (int i = 0; i < nStepsAround; i++)
                    {
                        float a1 = i * fAngleStep;
                        float a2 = (i + 1) * fAngleStep;

```

```

        Vector3 p1 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z);
        Vector3 p2 = new Vector3((float)Math.Cos(a2) * r, (float)Math.Sin(a2) * r, z + fCellSize);
        Vector3 p3 = new Vector3((float)Math.Cos(a1) * r, (float)Math.Sin(a1) * r, z + fCellSize);

        latStructure.AddBeam(p1, p2, fBeamRadiusMm, fBeamRadiusMm);
        latStructure.AddBeam(p1, p3, fBeamRadiusMm, fBeamRadiusMm);
    }
}

// Convertir la red de vigas a campo de vóxeles
Voxels voxLatticeVolume = new Voxels(latStructure);

// 4. Recortar el lattice para ajustarlo exactamente al dominio anular interno
voxLatticeVolume.Intersect(voxCoreDomain);

// 5. Unir pieles estancas con el núcleo reticular
Voxels voxChamberAssembly = new Voxels();
voxChamberAssembly.Add(voxInnerSkin);
voxChamberAssembly.Add(voxOuterSkin);
voxChamberAssembly.Add(voxLatticeVolume);

return voxChamberAssembly;
}

private static Voxels voxCreateCylindricalShell(float fRInner, float fROuter, float fHeight)
{
    Mesh meshOuter = Mesh.mshCreateCylinder(fROuter, fHeight);
    Mesh meshInner = Mesh.mshCreateCylinder(fRInner, fHeight + 2.0f);

    Voxels voxOuter = new Voxels(meshOuter);
    Voxels voxInner = new Voxels(meshInner);
    voxOuter.Subtract(voxInner);

    return voxOuter;
}
}
}

```

Arquitectura de la Solución en PikoGK

- **Recorte Booleano (Intersect):** El entramado de vigas se genera cubriendo la región tridimensional global y luego se aplica `voxLatticeVolume.Intersect(voxCoreDomain)`. Esto garantiza que ninguna viga protruya fuera de la superficie interior del prequemador ni sobresalga de la cubierta exterior.
- **Integridad Estructural:** El resultado combina la estanqueidad de las paredes continua (`voxInnerSkin` y `voxOuterSkin`) con el flujo fluido del refrigerante a través de los vacíos de la estructura celular interna.

Response: Las imágenes proporcionadas muestran referencias detalladas del diseño CAD y corte transversal de un motor de combustión por etapas de flujo completo (estilo SpaceX Raptor / FFSC), incluyendo las turbobombas de LOX y CH₄, prequemadores, distribuidores (*manifolds*), y el ensamblaje de la cámara principal y tobera.

A partir de los esquemas estructurales y las vistas explosionadas de los modelos 3D presentados, destacan tres elementos clave para continuar la construcción del modelo paramétrico en **PikoGK**:

Componentes Clave Identificados para Modelado

- **Distribuidor Toroidal (Injector Manifold):** Anillo de alimentación situado sobre la placa inyectora que garantiza la distribución uniforme de flujo hacia los 13 elementos coaxiales.
- **Conducto de Gases Calientes (Hot Gas Duct):** Tubo curvado de alta presión que conecta la descarga de la turbina del prequemador con la cabeza de la cámara principal.
- **Chaqueta de Enfriamiento y Bridas de Acople:** Grosor de pared variable en la cámara de combustión y anillos flangeados con patrón de pernos para fijación estructural.

Integración de Manifold Toroidal en C# (PikoGK)

A continuación se muestra cómo modelar el distribuidor toroidal (*manifold*) de alimentación y conectarlo a la cabeza del prequemador mediante operaciones booleanas volumétricas en PikoGK:

```

using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class ManifoldGenerator
    {

```

```

{
    /// <summary>
    /// Genera un distribuidor toroidal (Torus Manifold) para alimentación de propelente
    /// alineado con la placa inyectora.
    /// </summary>
    public static Voxels voxCreateToroidalManifold(
        float fMajorRadiusMm = 65.0f,
        float fOuterTubeRadiusMm = 14.0f,
        float fWallThicknessMm = 3.0f,
        float fCenterZ = 12.0f)
    {
        float fInnerTubeRadiusMm = fOuterTubeRadiusMm - fWallThicknessMm;

        // 1. Toro exterior (geometría sólida)
        Mesh meshOuterTorus = CreateTorusMesh(fMajorRadiusMm, fOuterTubeRadiusMm, fCenterZ);
        Voxels voxManifold = new Voxels(meshOuterTorus);

        // 2. Toro interior (vacío del canal de fluido)
        Mesh meshInnerTorus = CreateTorusMesh(fMajorRadiusMm, fInnerTubeRadiusMm, fCenterZ);
        Voxels voxFluidChannel = new Voxels(meshInnerTorus);

        // Perforación interna del distribuidor
        voxManifold.Subtract(voxFluidChannel);

        return voxManifold;
    }

    private static Mesh CreateTorusMesh(
        float fRMajor, float fRMinor, float fCenterZ, int nStepsMajor = 48, int nStepsMinor = 24)
    {
        Mesh mesh = new Mesh();
        float dPhi = 2.0f * (float)Math.PI / nStepsMajor;
        float dTheta = 2.0f * (float)Math.PI / nStepsMinor;

        for (int i = 0; i < nStepsMajor; i++)
        {
            float phi = i * dPhi;
            float cosPhi = (float)Math.Cos(phi);
            float sinPhi = (float)Math.Sin(phi);

            for (int j = 0; j < nStepsMinor; j++)
            {
                float theta = j * dTheta;
                float cosTheta = (float)Math.Cos(theta);
                float sinTheta = (float)Math.Sin(theta);

                float x = (fRMajor + fRMinor * cosTheta) * cosPhi;
                float y = (fRMajor + fRMinor * cosTheta) * sinPhi;
                float z = fCenterZ + fRMinor * sinTheta;

                mesh.nAddVertex(new Vector3(z, x, y));
            }
        }

        for (int i = 0; i < nStepsMajor; i++)
        {
            int iNext = (i + 1) % nStepsMajor;
            for (int j = 0; j < nStepsMinor; j++)
            {
                int jNext = (j + 1) % nStepsMinor;

                int idx00 = i * nStepsMinor + j;
                int idx10 = iNext * nStepsMinor + j;
                int idx01 = i * nStepsMinor + jNext;
                int idx11 = iNext * nStepsMinor + jNext;

                mesh.AddTriangle(idx00, idx10, idx11);
                mesh.AddTriangle(idx00, idx11, idx01);
            }
        }

        return mesh;
    }
}

```

User prompt: Muestra el código en C# para PikoGK para modelar el conducto curvado de gases calientes (Hot Gas Duct) que une la salida de la turbina con la cámara.

Response: Este código genera el conducto curvado de gases calientes (*Hot Gas Duct*) en C# para **PikoGK**. Utiliza barrido paramétrico sobre una trayectoria en arco (*elbow curve*) para crear el cuerpo del tubo, la cavidad interior del fluido y las bridas de acople estructural en ambos extremos.

Código C#: Generador del Conducto de Gases Calientes

```
using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class HotGasDuctGenerator
    {
        /// <summary>
        /// Genera el conducto curvado de gases calientes (Hot Gas Duct) con cavidad interior
        /// y bridas de conexión en los extremos.
        /// </summary>
        /// <param name="fBendRadiusMm">Radio de curvatura del eje central del tubo.</param>
        /// <param name="fOuterRadiusMm">Radio exterior del tubo.</param>
        /// <param name="fWallThicknessMm">Espesor de pared del conducto.</param>
        /// <param name="fBendAngleDeg">Ángulo total de la curva en grados (ej. 90°).</param>
        /// <param name="fFlangeRadiusMm">Radio exterior de las bridas de montaje.</param>
        /// <param name="fFlangeThicknessMm">Espesor de las bridas de montaje.</param>
        public static Voxels voxCreateHotGasDuct(
            float fBendRadiusMm = 75.0f,
            float fOuterRadiusMm = 16.0f,
            float fWallThicknessMm = 3.0f,
            float fBendAngleDeg = 90.0f,
            float fFlangeRadiusMm = 24.0f,
            float fFlangeThicknessMm = 8.0f)
        {
            float fInnerRadiusMm = fOuterRadiusMm - fWallThicknessMm;
            float fBendAngleRad = fBendAngleDeg * (float)Math.PI / 180.0f;

            // 1. Malla del cuerpo exterior del tubo curvado
            Mesh meshOuterDuct = CreateSweptPipeMesh(fBendRadiusMm, fOuterRadiusMm, fBendAngleRad);
            Voxels voxDuct = new Voxels(meshOuterDuct);

            // 2. Malla del canal de fluido interior (ligeramente extendida para perforación limpia)
            Mesh meshInnerPassage = CreateSweptPipeMesh(fBendRadiusMm, fInnerRadiusMm, fBendAngleRad, fExtensionMm: 10.0f);
            Voxels voxFluidPassage = new Voxels(meshInnerPassage);

            // 3. Bridas de acople estructural en la entrada y salida
            Voxels voxInletFlange = voxCreateFlange(
                new Vector3(0, 0, 0),
                new Vector3(1, 0, 0),
                fFlangeRadiusMm,
                fFlangeThicknessMm
            );

            Vector3 vOutletPos = GetCurvePoint(fBendRadiusMm, fBendAngleRad);
            Vector3 vOutletDir = GetCurveTangent(fBendAngleRad);

            Voxels voxOutletFlange = voxCreateFlange(
                vOutletPos,
                vOutletDir,
                fFlangeRadiusMm,
                fFlangeThicknessMm
            );

            // 4. Unión de cuerpo y bridas
            voxDuct.Add(voxInletFlange);
            voxDuct.Add(voxOutletFlange);

            // 5. Vaciado del conducto interno (Boolean Subtract)
            voxDuct.Subtract(voxFluidPassage);

            return voxDuct;
        }

        private static Mesh CreateSweptPipeMesh(
            float fBendRadius, float fCrossRadius, float fMaxAngleRad, float fExtensionMm = 0.0f,
            int nStepsBend = 36, int nStepsCircle = 24)
        {
            Mesh mesh = new Mesh();
        }
    }
}
```

```

float dAngle = fMaxAngleRad / nStepsBend;
float dPhi = 2.0f * (float)Math.PI / nStepsCircle;

for (int i = 0; i <= nStepsBend; i++)
{
    float fAngle = i * dAngle;
    Vector3 vCenter = GetCurvePoint(fBendRadius, fAngle);
    Vector3 vTangent = GetCurveTangent(fAngle);

    // Si hay extensión, alargamos los extremos para asegurar perforación completa
    if (i == 0 && fExtensionMm > 0.0f)
        vCenter -= vTangent * fExtensionMm;
    else if (i == nStepsBend && fExtensionMm > 0.0f)
        vCenter += vTangent * fExtensionMm;

    // Sistema de coordenadas local para el plano transversal
    Vector3 vNormal = new Vector3(-(float)Math.Sin(fAngle), 0, (float)Math.Cos(fAngle));
    Vector3 vBinormal = Vector3.UnitY;

    for (int j = 0; j < nStepsCircle; j++)
    {
        float phi = j * dPhi;
        float cosP = (float)Math.Cos(phi);
        float sinP = (float)Math.Sin(phi);

        Vector3 vPos = vCenter + fCrossRadius * (cosP * vNormal + sinP * vBinormal);
        mesh.nAddVertex(vPos);
    }
}

// Triangulación de la pared del cilindro curvado
for (int i = 0; i < nStepsBend; i++)
{
    for (int j = 0; j < nStepsCircle; j++)
    {
        int jNext = (j + 1) % nStepsCircle;

        int idx00 = i * nStepsCircle + j;
        int idx10 = (i + 1) * nStepsCircle + j;
        int idx01 = i * nStepsCircle + jNext;
        int idx11 = (i + 1) * nStepsCircle + jNext;

        mesh.AddTriangle(idx00, idx10, idx11);
        mesh.AddTriangle(idx00, idx11, idx01);
    }
}

return mesh;
}

private static Voxels voxCreateFlange(Vector3 vCenter, Vector3 vNormal, float fRadius, float fThickness)
{
    Mesh meshFlange = new Mesh();
    int nSteps = 32;
    float dTheta = 2.0f * (float)Math.PI / nSteps;

    // Construcción del plano local ortogonal a la normal
    Vector3 vUp = Math.Abs(vNormal.Y) > 0.9f ? Vector3.UnitZ : Vector3.UnitY;
    Vector3 vTangent = Vector3.Normalize(Vector3.Cross(vNormal, vUp));
    Vector3 vBinormal = Vector3.Normalize(Vector3.Cross(vTangent, vNormal));

    Vector3 vStart = vCenter - vNormal * (fThickness * 0.5f);
    Vector3 vEnd = vCenter + vNormal * (fThickness * 0.5f);

    int[] botIndices = new int[nSteps];
    int[] topIndices = new int[nSteps];

    int vCenterBot = meshFlange.nAddVertex(vStart);
    int vCenterTop = meshFlange.nAddVertex(vEnd);

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * dTheta;
        Vector3 vOffset = fRadius * ((float)Math.Cos(theta) * vTangent + (float)Math.Sin(theta) * vBinormal)

        botIndices[s] = meshFlange.nAddVertex(vStart + vOffset);
        topIndices[s] = meshFlange.nAddVertex(vEnd + vOffset);
    }
}

```



```

        for (int s = 0; s < nSteps; s++)
        {
            int sNext = (s + 1) % nSteps;
            meshFlange.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
            meshFlange.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
            meshFlange.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
            meshFlange.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
        }

        return new Voxels(meshFlange);
    }

    private static Vector3 GetCurvePoint(float fR, float fAngleRad)
    {
        return new Vector3(
            fR * (float)Math.Sin(fAngleRad),
            0.0f,
            fR * (1.0f - (float)Math.Cos(fAngleRad))
        );
    }

    private static Vector3 GetCurveTangent(float fAngleRad)
    {
        return new Vector3(
            (float)Math.Cos(fAngleRad),
            0.0f,
            (float)Math.Sin(fAngleRad)
        );
    }
}
}
}

```

Aspectos Geométricos y Funcionales

- **Barrido Cinemático:** La función `CreateSweptPipeMesh` evalúa la trayectoria del conducto mediante vectores de Frenet-Serret (T, N, B) para orientar las secciones circulares transversales a lo largo del arco.
- **Ajuste de Tolerancia para Operaciones Booleanas:** El canal interior de fluido incluye un parámetro `fExtensionMm` que alarga los extremos de la malla de perforación, previniendo caras co-planares o paredes residuales no deseadas en las bridas de entrada y salida.
- **Unión Sólida Estructural:** Las bridas en los extremos se adaptan dinámicamente a la tangente de la curva en el origen ($\theta = 0$) y en el extremo final ($\theta = 90^\circ$), listas para recibir pernos de fijación o ser fusionadas con la salida de la turbina.

Response: Este módulo en C# para **PikoGK** genera la geometría del inyector coaxial con álabes helicoidales (*swirl vanes*) en el canal anular de combustible (GCH_4) alrededor del tubo conducto central de oxidante (GOX), reproduciendo el esquema de atomización y mezcla de la imagen.

```

using System;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class SwirlerInjectorGenerator
    {
        /// <summary>
        /// Genera un elemento inyector coaxial con álabes de remolino (Swirl Vanes)
        /// para atomización y mezcla de GCH4 y GOX.
        /// </summary>
        public static Voxels voxCreateSwirlInjectorElement(
            float fCoreTubeInnerRadiusMm = 2.0f,
            float fCoreTubeOuterRadiusMm = 3.0f,
            float fSwirlerOuterRadiusMm = 6.0f,
            float fElementLengthMm = 30.0f,
            int nVaneCount = 6,
            float fHelixPitchMm = 25.0f)
        {
            // 1. Tubo central de oxidante (GOX Core Flow Tube)
            Mesh meshCoreOuter = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
            Mesh meshCoreInner = CreateCylinderMesh(fCoreTubeInnerRadiusMm, fElementLengthMm + 4.0f, -2.0f);

            Voxels voxCoreTube = new Voxels(meshCoreOuter);
            Voxels voxCoreVoid = new Voxels(meshCoreInner);
            voxCoreTube.Subtract(voxCoreVoid);

            // 2. Manga exterior del paso anular (GCH4 Outer Sleeve)
            Mesh meshSleeveOuter = CreateCylinderMesh(fSwirlerOuterRadiusMm, fElementLengthMm);

```

```

Mesh meshSleeveInner = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);

Voxels voxSwirlChamber = new Voxels(meshSleeveOuter);
Voxels voxSwirlVoid = new Voxels(meshSleeveInner);
voxSwirlChamber.Subtract(voxSwirlVoid);

// 3. Generación de los álabes helicoidales (Swirl Vanes)
Voxels voxVanes = voxCreateHelicalVanes(
    fCoreTubeOuterRadiusMm,
    fSwirlerOuterRadiusMm,
    fElementLengthMm,
    nVaneCount,
    fHelixPitchMm
);

// 4. Integrar álabes dentro del canal anular de combustible
voxSwirlChamber.Intersect(voxVanes);

// 5. Unir conducto central y ensamble de remolino exterior
Voxels voxInjector = new Voxels();
voxInjector.Add(voxCoreTube);
voxInjector.Add(voxSwirlChamber);

return voxInjector;
}

private static Voxels voxCreateHelicalVanes(
    float fRInner, float fROuter, float fLength, int nVanes, float fPitchMm)
{
    Voxels voxVanesGroup = new Voxels();
    float fVaneThicknessMm = 0.8f;
    int nStepsZ = 30;
    float fDeltaZ = fLength / nStepsZ;

    for (int v = 0; v < nVanes; v++)
    {
        float fBaseAngle = v * (2.0f * (float)Math.PI / nVanes);
        Mesh meshVane = new Mesh();

        for (int i = 0; i <= nStepsZ; i++)
        {
            float fZ = i * fDeltaZ;
            float fAngle = fBaseAngle + (fZ / fPitchMm) * 2.0f * (float)Math.PI;

            Vector3 vRadDir = new Vector3((float)Math.Cos(fAngle), (float)Math.Sin(fAngle), 0);
            Vector3 pInner = new Vector3(fZ, fRInner * vRadDir.X, fRInner * vRadDir.Y);
            Vector3 pOuter = new Vector3(fZ, fROuter * vRadDir.X, fROuter * vRadDir.Y);

            meshVane.nAddVertex(pInner);
            meshVane.nAddVertex(pOuter);
        }

        for (int i = 0; i < nStepsZ; i++)
        {
            int i0 = i * 2;
            int i1 = i0 + 1;
            int i2 = (i + 1) * 2;
            int i3 = i2 + 1;

            meshVane.AddTriangle(i0, i1, i3);
            meshVane.AddTriangle(i0, i3, i2);
        }

        Voxels voxSingleVane = new Voxels(meshVane);
        voxSingleVane.DoOffset(fVaneThicknessMm * 0.5f);
        voxVanesGroup.Add(voxSingleVane);
    }

    return voxVanesGroup;
}

private static Mesh CreateCylinderMesh(float fRadius, float fHeight, float fStartZ = 0.0f, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int vCenterBot = mesh.nAddVertex(new Vector3(fStartZ, 0, 0));
    int vCenterTop = mesh.nAddVertex(new Vector3(fStartZ + fHeight, 0, 0));

```

```

int[] botIndices = new int[nSteps];
int[] topIndices = new int[nSteps];

for (int s = 0; s < nSteps; s++)
{
    float theta = s * fDTheta;
    float cosT = (float)Math.Cos(theta);
    float sinT = (float)Math.Sin(theta);

    botIndices[s] = mesh.nAddVertex(new Vector3(fStartZ, fRadius * cosT, fRadius * sinT));
    topIndices[s] = mesh.nAddVertex(new Vector3(fStartZ + fHeight, fRadius * cosT, fRadius * sinT));
}

for (int s = 0; s < nSteps; s++)
{
    int sNext = (s + 1) % nSteps;
    mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
    mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
    mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
    mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
}

return mesh;
}
}
}

```

Características Técnicas del Diseño

- **Geometría Helicoidal Imprimible:** La función `voxCreateHelicalVanes` calcula cintas parabólicas en coordenadas cilíndricas que sufren una dilatación volumétrica de vóxeles (`DoOffset`), formando los álabes de grosor constante de 0.8 mm.
- **Inyección de Momentum Angular:** La rotación forzada del combustible aumenta la atomización centrífuga al entrar en contacto con el núcleo axial de oxidante en la zona de mezcla.

User prompt: Muestra cómo construir en PikoGK las galerías de distribución internas de GOX y GCH4 integradas en la pared del inyector.

Response: Este código en C# para **PikoGK** genera el cuerpo principal del inyector integrando las **galerías internas de distribución de combustible (GCH₄) y oxidante (GOX)**.

Para permitir la fabricación mediante impresión 3D en metal (LPBF) sin necesidad de soportes internos en las cavidades, las galerías se modelan utilizando una **sección transversal autosoportada (tipo gota o diamante)** revolucionada alrededor del eje central del inyector.

Código C#: Galerías de Distribución Integradas (GCH₄ / GOX)

```

using System;
using System.Numerics;
using System.Collections.Generic;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorManifoldsGenerator
    {
        /// <summary>
        /// Genera la cabeza del inyector con las galerías internas de distribución
        /// de GOX y GCH4 vaciadas directamente en el cuerpo sólido.
        /// </summary>
        public static Voxels voxCreateInjectorHousingWithGalleries(
            float fHousingOuterRadiusMm = 80.0f,
            float fHousingHeightMm = 90.0f,
            float fGch4GalleryRadiusMm = 55.0f, // Radio medio del toroide GCH4 (exterior)
            float fGoxGalleryRadiusMm = 32.0f, // Radio medio del toroide GOX (interior)
            float fChannelWidthMm = 14.0f,
            float fChannelHeightMm = 22.0f)
        {
            // 1. Crear el cuerpo sólido exterior de la cabeza del inyector
            Mesh meshHousing = CreateConeFrustumMesh(
                fRadiusBottom: fHousingOuterRadiusMm,
                fRadiusTop: fHousingOuterRadiusMm * 0.75f,
                fHeight: fHousingHeightMm
            );
            Voxels voxHousing = new Voxels(meshHousing);

            // 2. Generar cavidad de la galería de GCH4 (Canal superior / exterior - Naranja)
            Voxels voxGch4Gallery = voxCreateRevolvedTeardropGallery(
                fMajorRadiusMm: fGch4GalleryRadiusMm,

```

```

        fZOffsetMm: 55.0f,
        fWidthMm: fChannelWidthMm,
        fHeightMm: fChannelHeightMm
    );

    // Entrada de alimentación lateral para GCH4
    Voxels voxGch4Inlet = voxCreateRadialInletDuct(
        vStart: new Vector3(fGch4GalleryRadiusMm, 0, 55.0f),
        vDirection: new Vector3(1, 0, 0),
        fLengthMm: 35.0f,
        fDuctRadiusMm: fChannelWidthMm * 0.45f
    );
    voxGch4Gallery.Add(voxGch4Inlet);

    // 3. Generar cavidad de la galería de GOX (Canal inferior / interior - Azul)
    Voxels voxGoxGallery = voxCreateRevolvedTeardropGallery(
        fMajorRadiusMm: fGoxGalleryRadiusMm,
        fZOffsetMm: 35.0f,
        fWidthMm: fChannelWidthMm * 0.85f,
        fHeightMm: fChannelHeightMm * 0.85f
    );

    // Entrada de alimentación lateral para GOX
    Voxels voxGoxInlet = voxCreateRadialInletDuct(
        vStart: new Vector3(fGoxGalleryRadiusMm, 0, 35.0f),
        vDirection: Vector3.Normalize(new Vector3(1, 1, 0)),
        fLengthMm: 50.0f,
        fDuctRadiusMm: fChannelWidthMm * 0.40f
    );
    voxGoxGallery.Add(voxGoxInlet);

    // 4. Perforación central para el tubo de flujo principal / GOX central
    Mesh meshCenterCore = CreateCylinderMesh(fRadius: 12.0f, fHeight: fHousingHeightMm + 10.0f, fStartZ: -5.
    Voxels voxCenterCore = new Voxels(meshCenterCore);

    // 5. Operaciones Booleanas de Vaciado de Galerías
    voxHousing.Subtract(voxGch4Gallery);
    voxHousing.Subtract(voxGoxGallery);
    voxHousing.Subtract(voxCenterCore);

    return voxHousing;
}

/// <summary>
/// Revolución de un perfil autosoportado en forma de gota/diamante
/// para prevenir colapsos en impresión 3D de metal sin soportes.
/// </summary>
private static Voxels voxCreateRevolvedTeardropGallery(
    float fMajorRadiusMm, float fZOffsetMm, float fWidthMm, float fHeightMm, int nCircleSteps = 64)
{
    Mesh meshGallery = new Mesh();
    float dTheta = 2.0f * (float)Math.PI / nCircleSteps;

    // Perfil 2D autosoportado (4 vértices: superior en punta 45°, inferior, lados)
    Vector2[] profile2D = new Vector2[]
    {
        new Vector2(0, fHeightMm * 0.5f), // Vértice superior (Cresta a 45°)
        new Vector2(fWidthMm * 0.5f, 0), // Extremo exterior
        new Vector2(0, -fHeightMm * 0.5f), // Vértice inferior
        new Vector2(-fWidthMm * 0.5f, 0) // Extremo interior
    };

    int nProfilePoints = profile2D.Length;

    for (int s = 0; s <= nCircleSteps; s++)
    {
        float theta = s * dTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        for (int p = 0; p < nProfilePoints; p++)
        {
            float r = fMajorRadiusMm + profile2D[p].X;
            float z = fZOffsetMm + profile2D[p].Y;

            Vector3 pos = new Vector3(r * cosT, r * sinT, z);
            meshGallery.nAddVertex(pos);
        }
    }
}

```

```

// Triangulación de la superficie de revolución
for (int s = 0; s < nCircleSteps; s++)
{
    for (int p = 0; p < nProfilePoints; p++)
    {
        int pNext = (p + 1) % nProfilePoints;

        int idx00 = s * nProfilePoints + p;
        int idx10 = (s + 1) * nProfilePoints + p;
        int idx01 = s * nProfilePoints + pNext;
        int idx11 = (s + 1) * nProfilePoints + pNext;

        meshGallery.AddTriangle(idx00, idx10, idx11);
        meshGallery.AddTriangle(idx00, idx11, idx01);
    }
}

return new Voxels(meshGallery);
}

private static Voxels voxCreateRadialInletDuct(
    Vector3 vStart, Vector3 vDirection, float fLengthMm, float fDuctRadiusMm, int nSteps = 24)
{
    Mesh meshDuct = new Mesh();
    Vector3 vEnd = vStart + Vector3.Normalize(vDirection) * fLengthMm;

    // Creación de tubo de alimentación utilizando vectores ortogonales
    Vector3 vUp = Math.Abs(vDirection.Z) > 0.9f ? Vector3.UnitY : Vector3.UnitZ;
    Vector3 vTangent = Vector3.Normalize(Vector3.Cross(vDirection, vUp));
    Vector3 vBinormal = Vector3.Normalize(Vector3.Cross(vTangent, vDirection));

    float dTheta = 2.0f * (float)Math.PI / nSteps;
    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * dTheta;
        Vector3 vOffset = fDuctRadiusMm * ((float)Math.Cos(theta) * vTangent + (float)Math.Sin(theta) * vBinormal);

        int v0 = meshDuct.nAddVertex(vStart + vOffset);
        int v1 = meshDuct.nAddVertex(vEnd + vOffset);

        int sNext = (s + 1) % nSteps;
        // Construcción simplificada de pared de tubo
    }

    // Convertir a Voxels y expandir levemente para asegurar unión limpia
    Voxels voxDuct = new Voxels(meshDuct);
    voxDuct.DoOffset(fDuctRadiusMm * 0.5f);
    return voxDuct;
}

private static Mesh CreateConeFrustumMesh(float fRadiusBottom, float fRadiusTop, float fHeight, int nSteps =
{
    Mesh mesh = new Mesh();
    float dTheta = 2.0f * (float)Math.PI / nSteps;

    int vCenterBot = mesh.nAddVertex(new Vector3(0, 0, 0));
    int vCenterTop = mesh.nAddVertex(new Vector3(0, 0, fHeight));

    int[] botIndices = new int[nSteps];
    int[] topIndices = new int[nSteps];

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * dTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        botIndices[s] = mesh.nAddVertex(new Vector3(fRadiusBottom * cosT, fRadiusBottom * sinT, 0));
        topIndices[s] = mesh.nAddVertex(new Vector3(fRadiusTop * cosT, fRadiusTop * sinT, fHeight));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
        mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
        mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
        mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
    }
}

```

```

    }

    return mesh;
}

private static Mesh CreateCylinderMesh(float fRadius, float fHeight, float fStartZ = 0.0f, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int vCenterBot = mesh.nAddVertex(new Vector3(0, 0, fStartZ));
    int vCenterTop = mesh.nAddVertex(new Vector3(0, 0, fStartZ + fHeight));

    int[] botIndices = new int[nSteps];
    int[] topIndices = new int[nSteps];

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fDTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        botIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ));
        topIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ + fHeight));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
        mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
        mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
        mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
    }

    return mesh;
}
}
}
}

```

Principios de Diseño Térmico y de Fabricación Integrados

- **Geometría Autosoportada (Autonome DFM):** La función `voxCreateRevolvedTeardropGallery` construye perfiles en forma de diamante con techos inclinados a $\geq 45^\circ$. Esto evita deformaciones o colapsos del material refundido en impresoras 3D de lecho de polvo (LPBF) sin necesidad de rellenar la cavidad con estructuras de soporte.
- **Aislamiento por Pared Compartida:** Las galerías de GCH_4 (más fría/gas de metano) y GOX están separadas axial y radialmente por un espesor mínimo de pared dentro del voxel grid, distribuyendo uniformemente la presión de entrada antes de alimentar los elementos inyectoros coaxiales.
- **Integración Booleana de Canales:** Los conductos de entrada (`voxGch4Inlet`, `voxGoxInlet`) y las cavidades colindan con el bloque exterior y sufren una resta booleana limpia (`Subtract`), dejando pasajes cerrados estancos.

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectoros coaxiales.

Response: Este código en C# para **PikoGK** realiza el ensamble completo de la cabeza del inyector. Integra el cuerpo exterior, las **galerías internas de distribución de GCH_4 y GOX** y una **matriz radial de 13 inyectoros coaxiales** (1 central + 6 en anillo interno + 6 en anillo externo) con sus respectivos canales de alimentación pasantes.

Código C#: Ensamble Completo del Inyector Multielemento

```

using System;
using System.Numerics;
using System.Collections.Generic;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssemblyGenerator
    {
        /// <summary>
        /// Genera el ensamble monolítico completo del inyector de cohete,
        /// incluyendo cuerpo principal, galerías internas de distribución
        /// y 13 elementos inyectoros coaxiales orientados radialmente.
        /// </summary>
        public static Voxels voxCreateFullInjectorAssembly(
            float fHousingOuterRadiusMm = 75.0f,

```

```

float fHousingHeightMm = 85.0f,
float fInnerRingRadiusMm = 28.0f,
float fOuterRingRadiusMm = 52.0f)
{
    // 1. Generar el cuerpo base del inyector con sus galerías de distribución
    Voxels voxHousing = InjectorManifoldsGenerator.voxCreateInjectorHousingWithGalleries(
        fHousingOuterRadiusMm: fHousingOuterRadiusMm,
        fHousingHeightMm: fHousingHeightMm,
        fGch4GalleryRadiusMm: fOuterRingRadiusMm,
        fGoxGalleryRadiusMm: fInnerRingRadiusMm
    );

    // 2. Definir las posiciones de la matriz de 13 inyectores (1 Centro, 6 Anillo Interno, 6 Anillo Externo)
    List<Vector3> listInjectorPositions = GetInjectorPatternPositions(
        fInnerRingRadiusMm,
        fOuterRingRadiusMm
    );

    Voxels voxInjectorsGroup = new Voxels();
    Voxels voxInjectorDrillPassages = new Voxels();

    // 3. Generar e integrar cada elemento inyector coaxial en sus coordenadas
    foreach (Vector3 vPos in listInjectorPositions)
    {
        // Elemento inyector coaxial con swirlers desplazado a la posición (vPos.X, vPos.Y)
        Voxels voxSingleInjector = voxCreatePositionedInjectorElement(
            vOffset: vPos,
            fCoreTubeInnerRadiusMm: 2.0f,
            fCoreTubeOuterRadiusMm: 3.2f,
            fSwirlerOuterRadiusMm: 6.5f,
            fElementLengthMm: fHousingHeightMm + 10.0f
        );
        voxInjectorsGroup.Add(voxSingleInjector);

        // Perforaciones de alimentacion vertical desde galerias a la camara
        Mesh meshPassage = CreateCylinderMeshAtPosition(
            vCenter: vPos,
            fRadius: 6.6f,
            fHeight: fHousingHeightMm + 14.0f,
            fZStart: -7.0f
        );
        voxInjectorDrillPassages.Add(new Voxels(meshPassage));
    }

    // 4. Operación Booleana: Taladrar los alojamientos de los inyectores en el cuerpo principal
    voxHousing.Subtract(voxInjectorDrillPassages);

    // 5. Operación Booleana: Fusionar la matriz de inyectores coaxiales en el bloque
    voxHousing.Add(voxInjectorsGroup);

    return voxHousing;
}

/// <summary>
/// Calculador del patrón espacial de 13 inyectores (1 central, 6 en radio interno, 6 en radio externo).
/// </summary>
private static List<Vector3> GetInjectorPatternPositions(float fR1, float fR2)
{
    List<Vector3> positions = new List<Vector3>();

    // Inyector central (0, 0)
    positions.Add(new Vector3(0.0f, 0.0f, 0.0f));

    // Anillo interior: 6 inyectores separados por 60 grados
    for (int i = 0; i < 6; i++)
    {
        float fAngle = i * (float)(Math.PI / 3.0);
        positions.Add(new Vector3(
            fR1 * (float)Math.Cos(fAngle),
            fR1 * (float)Math.Sin(fAngle),
            0.0f
        ));
    }

    // Anillo exterior: 6 inyectores desfasados 30 grados respecto al interior
    for (int i = 0; i < 6; i++)
    {
        float fAngle = (i * (float)(Math.PI / 3.0)) + (float)(Math.PI / 6.0);
        positions.Add(new Vector3(

```

```

        fR2 * (float)Math.Cos(fAngle),
        fR2 * (float)Math.Sin(fAngle),
        0.0f
    ));
}

return positions;
}

/// <summary>
/// Genera la geometría de un inyector coaxial individual desplazado espacialmente.
/// </summary>
private static Voxels voxCreatePositionedInjectorElement(
    Vector3 vOffset,
    float fCoreTubeInnerRadiusMm,
    float fCoreTubeOuterRadiusMm,
    float fSwirlerOuterRadiusMm,
    float fElementLengthMm)
{
    // Malla del tubo central desplazada
    Mesh meshCoreOuter = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -5.
    Mesh meshCoreInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeInnerRadiusMm, fElementLengthMm + 10

    Voxels voxCore = new Voxels(meshCoreOuter);
    voxCore.Subtract(new Voxels(meshCoreInner));

    // Malla de la manga anular externa desplazada
    Mesh meshSleeveOuter = CreateCylinderMeshAtPosition(vOffset, fSwirlerOuterRadiusMm, fElementLengthMm, -5
    Mesh meshSleeveInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -

    Voxels voxSwirlChannel = new Voxels(meshSleeveOuter);
    voxSwirlChannel.Subtract(new Voxels(meshSleeveInner));

    // Combinación final del elemento
    Voxels voxElement = new Voxels();
    voxElement.Add(voxCore);
    voxElement.Add(voxSwirlChannel);

    return voxElement;
}

private static Mesh CreateCylinderMeshAtPosition(
    Vector3 vCenter, float fRadius, float fHeight, float fZStart = 0.0f, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int vCenterBot = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart));
    int vCenterTop = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart + fHeight));

    int[] botIndices = new int[nSteps];
    int[] topIndices = new int[nSteps];

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fDTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        float x = vCenter.X + fRadius * cosT;
        float y = vCenter.Y + fRadius * sinT;

        botIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart));
        topIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart + fHeight));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
        mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
        mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
        mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
    }

    return mesh;
}
}
}

```


Arquitectura Constructiva y Flujo Metodológico

1. **Patrón Multielemento Distribuido:** Mediante la función `GetInjectorPatternPositions`, el algoritmo establece 13 coordenadas cartesianas alineadas concéntricamente con los radios medios de las galerías toroide de distribución de oxidante (R_1) y combustible (R_2).
2. **Secuencia de Operaciones Booleanas de Voxelizado:**
 - **Paso A:** Generación del cuerpo cónico monolítico con cavidades de galerías en perfil de gota/diamante autosoportado para impresión 3D en aleación de níquel o Inconel.
 - **Paso B:** Extracción mediante `Subtract` de los 13 puertos verticales de alojamiento para interconectar los toroides de distribución con la zona de mezclado.
 - **Paso C:** Fusión mediante `Add` de los 13 inyectores coaxiales en una sola representación de espacio implícito de vóxeles libre de brechas geométricas (*watertight*).

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectores coaxiales.

Response: Este código en C# para **PikoGK** ensambla de forma completa la **cabeza del inyector**. Unifica el cuerpo exterior, las galerías internas de distribución (GOX y GCH₄) y una matriz de **13 inyectores coaxiales con álabes helicoidales** (1 central, 6 en el anillo interior y 6 en el anillo exterior), junto con los microconductos de alimentación que conectan cada galería con su respectivo inyector.

Código C#: Ensamble Completo del Inyector Multielemento

```
using System;
using System.Collections.Generic;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssembly
    {
        /// <summary>
        /// Genera el ensamble monolítico completo del inyector:
        /// - Cuerpo estructural con cara de inyección
        /// - Galerías toroidales de distribución (GOX azul, GCH4 naranja)
        /// - Matriz de 13 elementos inyectores coaxiales con swirlers helicoidales
        /// - Canales internos de interconexión entre galerías e inyectores
        /// </summary>
        public static Voxels voxAssembleCompleteInjectorHead(
            float fHousingRadiusMm = 75.0f,
            float fHousingHeightMm = 90.0f,
            float fInnerRingRadiusMm = 24.0f,
            float fOuterRingRadiusMm = 48.0f)
        {
            // -----
            // 1. CUERPO BASE Y GALERÍAS DE DISTRIBUCIÓN
            // -----
            // Generar el bloque sólido exterior del housing
            Voxels voxAssembly = voxCreateBaseHousing(fHousingRadiusMm, fHousingHeightMm);

            // Generar la galería exterior de GCH4 (Metano gaseoso)
            float fGch4GalleryRadius = 50.0f;
            float fGch4GalleryZ = 55.0f;
            Voxels voxGch4Gallery = voxCreateRevolvedGallery(
                fMajorRadiusMm: fGch4GalleryRadius,
                fZOffsetMm: fGch4GalleryZ,
                fWidthMm: 16.0f,
                fHeightMm: 22.0f
            );

            // Generar la galería interior de GOX (Oxígeno gaseoso)
            float fGoxGalleryRadius = 28.0f;
            float fGoxGalleryZ = 35.0f;
            Voxels voxGoxGallery = voxCreateRevolvedGallery(
                fMajorRadiusMm: fGoxGalleryRadius,
                fZOffsetMm: fGoxGalleryZ,
                fWidthMm: 14.0f,
                fHeightMm: 18.0f
            );

            // Vaciado de las galerías dentro del cuerpo principal (Boolean Subtract)
            voxAssembly.Subtract(voxGch4Gallery);
            voxAssembly.Subtract(voxGoxGallery);

            // -----
        }
    }
}
```

```

// 2. GENERACIÓN DE LAS POSICIONES DE LA MATRIZ DE 13 INYECTORES
// -----
List<Vector2> injectorPositions = Get13ElementPattern(fInnerRingRadiusMm, fOuterRingRadiusMm);

Voxels voxAllSwirlers = new Voxels();
Voxels voxAllFeedChannels = new Voxels();

float fInjectorLength = 45.0f;
float fInjectorStartZ = 10.0f; // Altura de la cara del inyector

// -----
// 3. CONSTRUCCIÓN E INTEGRACIÓN DE CADA INYECTOR COAXIAL
// -----
foreach (Vector2 pos in injectorPositions)
{
    Vector3 vPos3D = new Vector3(pos.X, pos.Y, fInjectorStartZ);

    // A) Elemento inyector coaxial (Tubo GOX + Swirler GCH4)
    Voxels voxSingleInjector = voxCreateSwirlInjectorElement(
        fCoreTubeInnerRadiusMm: 2.0f,
        fCoreTubeOuterRadiusMm: 3.2f,
        fSwirlerOuterRadiusMm: 6.5f,
        fElementLengthMm: fInjectorLength,
        nVaneCount: 6,
        fHelixPitchMm: 20.0f
    );

    // Posicionar inyector en la matriz
    voxSingleInjector.Translate(vPos3D);
    voxAllSwirlers.Add(voxSingleInjector);

    // B) Conductos de alimentación desde Galería GCH4 hacia la cámara del Swirler
    float fDistToCenter = pos.Length();
    if (fDistToCenter > 0.1f)
    {
        Vector2 vDirToCenter = Vector2.Normalize(-pos);

        // Tap directo desde la galería de GCH4
        Vector3 pGch4Gallery = new Vector3(
            pos.X - vDirToCenter.X * Math.Abs(fGch4GalleryRadius - fDistToCenter),
            pos.Y - vDirToCenter.Y * Math.Abs(fGch4GalleryRadius - fDistToCenter),
            fGch4GalleryZ
        );

        Vector3 pInjectorGch4Port = new Vector3(pos.X, pos.Y, fInjectorStartZ + fInjectorLength - 8.0f);
        Voxels voxGch4Feeder = voxCreateCylindricalDuct(pGch4Gallery, pInjectorGch4Port, fRadiusMm: 2.2f);
        voxAllFeedChannels.Add(voxGch4Feeder);

        // Tap directo desde la galería de GOX hacia el tubo central
        Vector3 pGoxGallery = new Vector3(
            pos.X - vDirToCenter.X * Math.Abs(fGoxGalleryRadius - fDistToCenter),
            pos.Y - vDirToCenter.Y * Math.Abs(fGoxGalleryRadius - fDistToCenter),
            fGoxGalleryZ
        );

        Vector3 pInjectorGoxPort = new Vector3(pos.X, pos.Y, fInjectorStartZ + fInjectorLength - 22.0f);
        Voxels voxGoxFeeder = voxCreateCylindricalDuct(pGoxGallery, pInjectorGoxPort, fRadiusMm: 1.8f);
        voxAllFeedChannels.Add(voxGoxFeeder);
    }
}

// -----
// 4. ENSAMBLE FINAL MEDIANTE OPERACIONES BOOLEANAS
// -----
// Vaciar los conductos de alimentación de las galerías
voxAssembly.Subtract(voxAllFeedChannels);

// Fusionar los inyectores coaxiales al cuerpo
voxAssembly.Add(voxAllSwirlers);

return voxAssembly;
}

// =====
// MÉTODOS AUXILIARES DE PATRÓN GEOMÉTRICO Y COMPONENTES
// =====

/// <summary>
/// Calcula las coordenadas 2D de una distribución de 13 inyectores (1+6+6).

```

```

/// </summary>
private static List<Vector2> Get13ElementPattern(float fRadiusRing1, float fRadiusRing2)
{
    List<Vector2> list = new List<Vector2>();

    // 1. Inyector central
    list.Add(Vector2.Zero);

    // 2. Anillo Interior (6 inyectores separados 60°)
    for (int i = 0; i < 6; i++)
    {
        float angle = i * (float)Math.PI / 3.0f;
        list.Add(new Vector2(
            fRadiusRing1 * (float)Math.Cos(angle),
            fRadiusRing1 * (float)Math.Sin(angle)
        ));
    }

    // 3. Anillo Exterior (6 inyectores alternados a 60°)
    float fOffsetAngle = (float)Math.PI / 6.0f; // 30° desfasado
    for (int i = 0; i < 6; i++)
    {
        float angle = i * (float)Math.PI / 3.0f + fOffsetAngle;
        list.Add(new Vector2(
            fRadiusRing2 * (float)Math.Cos(angle),
            fRadiusRing2 * (float)Math.Sin(angle)
        ));
    }

    return list;
}

private static Voxels voxCreateSwirlInjectorElement(
    float fCoreTubeInnerRadiusMm,
    float fCoreTubeOuterRadiusMm,
    float fSwirlerOuterRadiusMm,
    float fElementLengthMm,
    int nVaneCount,
    float fHelixPitchMm)
{
    // Tubo central GOX
    Mesh meshCoreOuter = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
    Mesh meshCoreInner = CreateCylinderMesh(fCoreTubeInnerRadiusMm, fElementLengthMm + 4.0f, -2.0f);
    Voxels voxCoreTube = new Voxels(meshCoreOuter);
    voxCoreTube.Subtract(new Voxels(meshCoreInner));

    // Cámara anular GCH4
    Mesh meshSleeveOuter = CreateCylinderMesh(fSwirlerOuterRadiusMm, fElementLengthMm);
    Mesh meshSleeveInner = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
    Voxels voxSwirlChamber = new Voxels(meshSleeveOuter);
    voxSwirlChamber.Subtract(new Voxels(meshSleeveInner));

    // Álabes helicoidales
    Voxels voxVanes = voxCreateHelicalVanes(
        fCoreTubeOuterRadiusMm, fSwirlerOuterRadiusMm, fElementLengthMm, nVaneCount, fHelixPitchMm);

    voxSwirlChamber.Intersect(voxVanes);

    Voxels voxElement = new Voxels();
    voxElement.Add(voxCoreTube);
    voxElement.Add(voxSwirlChamber);

    return voxElement;
}

private static Voxels voxCreateHelicalVanes(
    float fRInner, float fROuter, float fLength, int nVanes, float fPitchMm)
{
    Voxels voxVanesGroup = new Voxels();
    float fVaneThicknessMm = 0.8f;
    int nStepsZ = 20;
    float fDeltaZ = fLength / nStepsZ;

    for (int v = 0; v < nVanes; v++)
    {
        float fBaseAngle = v * (2.0f * (float)Math.PI / nVanes);
        Mesh meshVane = new Mesh();

        for (int i = 0; i <= nStepsZ; i++)

```

```

    {
        float fZ = i * fDeltaZ;
        float fAngle = fBaseAngle + (fZ / fPitchMm) * 2.0f * (float)Math.PI;

        Vector3 vRadDir = new Vector3((float)Math.Cos(fAngle), (float)Math.Sin(fAngle), 0);
        meshVane.nAddVertex(new Vector3(fRInner * vRadDir.X, fRInner * vRadDir.Y, fZ));
        meshVane.nAddVertex(new Vector3(fROuter * vRadDir.X, fROuter * vRadDir.Y, fZ));
    }

    for (int i = 0; i < nStepsZ; i++)
    {
        int i0 = i * 2, i1 = i0 + 1, i2 = (i + 1) * 2, i3 = i2 + 1;
        meshVane.AddTriangle(i0, i1, i3);
        meshVane.AddTriangle(i0, i3, i2);
    }

    Voxels voxSingleVane = new Voxels(meshVane);
    voxSingleVane.DoOffset(fVaneThicknessMm * 0.5f);
    voxVanesGroup.Add(voxSingleVane);
}

return voxVanesGroup;
}

private static Voxels voxCreateRevolvedGallery(
    float fMajorRadiusMm, float fZOffsetMm, float fWidthMm, float fHeightMm, int nCircleSteps = 48)
{
    Mesh meshGallery = new Mesh();
    float dTheta = 2.0f * (float)Math.PI / nCircleSteps;

    Vector2[] profile2D = new Vector2[]
    {
        new Vector2(0, fHeightMm * 0.5f),
        new Vector2(fWidthMm * 0.5f, 0),
        new Vector2(0, -fHeightMm * 0.5f),
        new Vector2(-fWidthMm * 0.5f, 0)
    };

    int nProfilePoints = profile2D.Length;

    for (int s = 0; s <= nCircleSteps; s++)
    {
        float theta = s * dTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        for (int p = 0; p < nProfilePoints; p++)
        {
            float r = fMajorRadiusMm + profile2D[p].X;
            float z = fZOffsetMm + profile2D[p].Y;

            meshGallery.nAddVertex(new Vector3(r * cosT, r * sinT, z));
        }
    }

    for (int s = 0; s < nCircleSteps; s++)
    {
        for (int p = 0; p < nProfilePoints; p++)
        {
            int pNext = (p + 1) % nProfilePoints;
            int idx00 = s * nProfilePoints + p;
            int idx10 = (s + 1) * nProfilePoints + p;
            int idx01 = s * nProfilePoints + pNext;
            int idx11 = (s + 1) * nProfilePoints + pNext;

            meshGallery.AddTriangle(idx00, idx10, idx11);
            meshGallery.AddTriangle(idx00, idx11, idx01);
        }
    }

    return new Voxels(meshGallery);
}

private static Voxels voxCreateBaseHousing(float fRadius, float fHeight)
{
    Mesh mesh = CreateCylinderMesh(fRadius, fHeight);
    return new Voxels(mesh);
}

```

```

private static Voxels voxCreateCylindricalDuct(Vector3 pStart, Vector3 pEnd, float fRadiusMm)
{
    Mesh mesh = new Mesh();
    Vector3 vDir = Vector3.Normalize(pEnd - pStart);
    float fLength = (pEnd - pStart).Length();

    Vector3 vUp = Math.Abs(vDir.Z) > 0.9f ? Vector3.UnitY : Vector3.UnitZ;
    Vector3 vTangent = Vector3.Normalize(Vector3.Cross(vDir, vUp));
    Vector3 vBinormal = Vector3.Normalize(Vector3.Cross(vTangent, vDir));

    int nSteps = 16;
    float dTheta = 2.0f * (float)Math.PI / nSteps;

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * dTheta;
        Vector3 vOffset = fRadiusMm * ((float)Math.Cos(theta) * vTangent + (float)Math.Sin(theta) * vBinormal);

        int v0 = mesh.nAddVertex(pStart + vOffset);
        int v1 = mesh.nAddVertex(pEnd + vOffset);
    }

    Voxels voxDuct = new Voxels(mesh);
    voxDuct.DoOffset(fRadiusMm * 0.5f);
    return voxDuct;
}

private static Mesh CreateCylinderMesh(float fRadius, float fHeight, float fStartZ = 0.0f, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int vCenterBot = mesh.nAddVertex(new Vector3(0, 0, fStartZ));
    int vCenterTop = mesh.nAddVertex(new Vector3(0, 0, fStartZ + fHeight));

    int[] botIndices = new int[nSteps];
    int[] topIndices = new int[nSteps];

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fDTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        botIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ));
        topIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ + fHeight));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
        mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
        mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
        mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
    }

    return mesh;
}
}
}

```

Detalles Destacados de la Arquitectura del Modelo

- Topología Integrada de Fluido:** La función `Get13ElementPattern` calcula la ubicación radial exacta de cada inyector. Para cada elemento, la subrutina calcula vectores tridimensionales desde las galerías toroidales (`voxCreateCylindricalDuct`) que perforan el cuerpo principal para alimentar de forma balanceada el oxidante y el combustible a cada inyector.
- Eficiencia Booleana Voxelizada:** Al trabajar directamente sobre vóxeles con **PikoGK**, la unión de 13 swirlers con sus rampas helicoidales y microconductos internos evita los errores geométrica y computacionalmente costosos de intersección de mallas tradicionales (B-Rep/STL).
- Impresión 3D Monolítica (LPBF):** Todo el conjunto se exporta en una única pieza sólida sin necesidad de soldaduras entre inyectores y placa, maximizando la rigidez estructural ante altas presiones y pulsaciones acústicas de combustión.

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectores coaxiales.

Response: Este código en C# para **PikoGK** realiza el ensamble completo de la cabeza del inyector. Integra el cuerpo exterior, las **galerías internas de distribución de GCH₄ y GOX** y una **matriz radial de 13 inyectores coaxiales** (1 central + 6 en anillo interno + 6 en anillo externo) con sus respectivos canales de alimentación pasantes.

Código C#: Ensamble Completo del Inyector Multielemento

```
using System;
using System.Numerics;
using System.Collections.Generic;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssemblyGenerator
    {
        /// <summary>
        /// Genera el ensamble monolítico completo del inyector de cohete,
        /// incluyendo cuerpo principal, galerías internas de distribución
        /// y 13 elementos inyectores coaxiales orientados radialmente.
        /// </summary>
        public static Voxels voxCreateFullInjectorAssembly(
            float fHousingOuterRadiusMm = 75.0f,
            float fHousingHeightMm = 85.0f,
            float fInnerRingRadiusMm = 28.0f,
            float fOuterRingRadiusMm = 52.0f)
        {
            // 1. Generar el cuerpo base del inyector con sus galerías de distribución
            Voxels voxHousing = InjectorManifoldsGenerator.voxCreateInjectorHousingWithGalleries(
                fHousingOuterRadiusMm: fHousingOuterRadiusMm,
                fHousingHeightMm: fHousingHeightMm,
                fGch4GalleryRadiusMm: fOuterRingRadiusMm,
                fGoxGalleryRadiusMm: fInnerRingRadiusMm
            );

            // 2. Definir las posiciones de la matriz de 13 inyectores (1 Centro, 6 Anillo Interno, 6 Anillo Externo)
            List<Vector3> listInjectorPositions = GetInjectorPatternPositions(
                fInnerRingRadiusMm,
                fOuterRingRadiusMm
            );

            Voxels voxInjectorsGroup = new Voxels();
            Voxels voxInjectorDrillPassages = new Voxels();

            // 3. Generar e integrar cada elemento inyector coaxial en sus coordenadas
            foreach (Vector3 vPos in listInjectorPositions)
            {
                // Elemento inyector coaxial con swirlers desplazado a la posición (vPos.X, vPos.Y)
                Voxels voxSingleInjector = voxCreatePositionedInjectorElement(
                    vOffset: vPos,
                    fCoreTubeInnerRadiusMm: 2.0f,
                    fCoreTubeOuterRadiusMm: 3.2f,
                    fSwirlerOuterRadiusMm: 6.5f,
                    fElementLengthMm: fHousingHeightMm + 10.0f
                );
                voxInjectorsGroup.Add(voxSingleInjector);

                // Perforaciones de alimentacion vertical desde galerias a la camara
                Mesh meshPassage = CreateCylinderMeshAtPosition(
                    vCenter: vPos,
                    fRadius: 6.6f,
                    fHeight: fHousingHeightMm + 14.0f,
                    fZStart: -7.0f
                );
                voxInjectorDrillPassages.Add(new Voxels(meshPassage));
            }

            // 4. Operación Booleana: Taladrar los alojamientos de los inyectores en el cuerpo principal
            voxHousing.Subtract(voxInjectorDrillPassages);

            // 5. Operación Booleana: Fusionar la matriz de inyectores coaxiales en el bloque
            voxHousing.Add(voxInjectorsGroup);

            return voxHousing;
        }

        /// <summary>
        /// Calculador del patrón espacial de 13 inyectores (1 central, 6 en radio interno, 6 en radio externo).
        /// </summary>
        private static List<Vector3> GetInjectorPatternPositions(float fR1, float fR2)
    }
}
```

```

{
    List<Vector3> positions = new List<Vector3>();

    // Inyector central (0, 0)
    positions.Add(new Vector3(0.0f, 0.0f, 0.0f));

    // Anillo interior: 6 inyectores separados por 60 grados
    for (int i = 0; i < 6; i++)
    {
        float fAngle = i * (float)(Math.PI / 3.0);
        positions.Add(new Vector3(
            fR1 * (float)Math.Cos(fAngle),
            fR1 * (float)Math.Sin(fAngle),
            0.0f
        ));
    }

    // Anillo exterior: 6 inyectores desfasados 30 grados respecto al interior
    for (int i = 0; i < 6; i++)
    {
        float fAngle = (i * (float)(Math.PI / 3.0)) + (float)(Math.PI / 6.0);
        positions.Add(new Vector3(
            fR2 * (float)Math.Cos(fAngle),
            fR2 * (float)Math.Sin(fAngle),
            0.0f
        ));
    }

    return positions;
}

/// <summary>
/// Genera la geometría de un inyector coaxial individual desplazado espacialmente.
/// </summary>
private static Voxels voxCreatePositionedInjectorElement(
    Vector3 vOffset,
    float fCoreTubeInnerRadiusMm,
    float fCoreTubeOuterRadiusMm,
    float fSwirlerOuterRadiusMm,
    float fElementLengthMm)
{
    // Malla del tubo central desplazada
    Mesh meshCoreOuter = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -5.
    Mesh meshCoreInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeInnerRadiusMm, fElementLengthMm + 10

    Voxels voxCore = new Voxels(meshCoreOuter);
    voxCore.Subtract(new Voxels(meshCoreInner));

    // Malla de la manga anular externa desplazada
    Mesh meshSleeveOuter = CreateCylinderMeshAtPosition(vOffset, fSwirlerOuterRadiusMm, fElementLengthMm, -5
    Mesh meshSleeveInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -

    Voxels voxSwirlChannel = new Voxels(meshSleeveOuter);
    voxSwirlChannel.Subtract(new Voxels(meshSleeveInner));

    // Combinación final del elemento
    Voxels voxElement = new Voxels();
    voxElement.Add(voxCore);
    voxElement.Add(voxSwirlChannel);

    return voxElement;
}

private static Mesh CreateCylinderMeshAtPosition(
    Vector3 vCenter, float fRadius, float fHeight, float fZStart = 0.0f, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int vCenterBot = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart));
    int vCenterTop = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart + fHeight));

    int[] botIndices = new int[nSteps];
    int[] topIndices = new int[nSteps];

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fDTheta;
        float cosT = (float)Math.Cos(theta);

```

```

        float sinT = (float)Math.Sin(theta);

        float x = vCenter.X + fRadius * cosT;
        float y = vCenter.Y + fRadius * sinT;

        botIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart));
        topIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart + fHeight));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
        mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
        mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
        mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
    }

    return mesh;
}
}
}

```

Arquitectura Constructiva y Flujo Metodológico

- Patrón Multielemento Distribuido:** Mediante la función `GetInjectorPatternPositions`, el algoritmo establece 13 coordenadas cartesianas alineadas concéntricamente con los radios medios de las galerías toroide de distribución de oxidante (R_1) y combustible (R_2).
- Secuencia de Operaciones Booleanas de Voxelizado:**
 - Paso A:** Generación del cuerpo cónico monolítico con cavidades de galerías en perfil de gota/diamante autosoportado para impresión 3D en aleación de níquel o Inconel.
 - Paso B:** Extracción mediante `Subtract` de los 13 puertos verticales de alojamiento para interconectar los toroides de distribución con la zona de mezclado.
 - Paso C:** Fusión mediante `Add` de los 13 inyectores coaxiales en una sola representación de espacio implícito de vóxeles libre de brechas geométricas (*watertight*).

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectores coaxiales.

Response: Este código en C# para **PikoGK** ensambla de forma completa la **cabeza del inyector**. Unifica el cuerpo exterior, las galerías internas de distribución (GOX y GCH₄) y una matriz de **13 inyectores coaxiales con álabes helicoidales** (1 central, 6 en el anillo interior y 6 en el anillo exterior), junto con los microconductos de alimentación que conectan cada galería con su respectivo inyector.

Código C#: Ensamble Completo del Inyector Multielemento

```

using System;
using System.Collections.Generic;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssembly
    {
        /// <summary>
        /// Genera el ensamble monolítico completo del inyector:
        /// - Cuerpo estructural con cara de inyección
        /// - Galerías toroidales de distribución (GOX azul, GCH4 naranja)
        /// - Matriz de 13 elementos inyectores coaxiales con swirlers helicoidales
        /// - Canales internos de interconexión entre galerías e inyectores
        /// </summary>
        public static Voxels voxAssembleCompleteInjectorHead(
            float fHousingRadiusMm = 75.0f,
            float fHousingHeightMm = 90.0f,
            float fInnerRingRadiusMm = 24.0f,
            float fOuterRingRadiusMm = 48.0f)
        {
            // -----
            // 1. CUERPO BASE Y GALERÍAS DE DISTRIBUCIÓN
            // -----
            // Generar el bloque sólido exterior del housing
            Voxels voxAssembly = voxCreateBaseHousing(fHousingRadiusMm, fHousingHeightMm);

            // Generar la galería exterior de GCH4 (Metano gaseoso)
            float fGch4GalleryRadius = 50.0f;

```



```

float fGch4GalleryZ = 55.0f;
Voxels voxGch4Gallery = voxCreateRevolvedGallery(
    fMajorRadiusMm: fGch4GalleryRadius,
    fZOffsetMm: fGch4GalleryZ,
    fWidthMm: 16.0f,
    fHeightMm: 22.0f
);

// Generar la galería interior de GOX (Oxígeno gaseoso)
float fGoxGalleryRadius = 28.0f;
float fGoxGalleryZ = 35.0f;
Voxels voxGoxGallery = voxCreateRevolvedGallery(
    fMajorRadiusMm: fGoxGalleryRadius,
    fZOffsetMm: fGoxGalleryZ,
    fWidthMm: 14.0f,
    fHeightMm: 18.0f
);

// Vaciado de las galerías dentro del cuerpo principal (Boolean Subtract)
voxAssembly.Subtract(voxGch4Gallery);
voxAssembly.Subtract(voxGoxGallery);

// -----
// 2. GENERACIÓN DE LAS POSICIONES DE LA MATRIZ DE 13 INYECTORES
// -----
List<Vector2> injectorPositions = Get13ElementPattern(fInnerRingRadiusMm, fOuterRingRadiusMm);

Voxels voxAllSwirlers = new Voxels();
Voxels voxAllFeedChannels = new Voxels();

float fInjectorLength = 45.0f;
float fInjectorStartZ = 10.0f; // Altura de la cara del inyector

// -----
// 3. CONSTRUCCIÓN E INTEGRACIÓN DE CADA INYECTOR COAXIAL
// -----
foreach (Vector2 pos in injectorPositions)
{
    Vector3 vPos3D = new Vector3(pos.X, pos.Y, fInjectorStartZ);

    // A) Elemento inyector coaxial (Tubo GOX + Swirler GCH4)
    Voxels voxSingleInjector = voxCreateSwirlInjectorElement(
        fCoreTubeInnerRadiusMm: 2.0f,
        fCoreTubeOuterRadiusMm: 3.2f,
        fSwirlerOuterRadiusMm: 6.5f,
        fElementLengthMm: fInjectorLength,
        nVaneCount: 6,
        fHelixPitchMm: 20.0f
    );

    // Posicionar inyector en la matriz
    voxSingleInjector.Translate(vPos3D);
    voxAllSwirlers.Add(voxSingleInjector);

    // B) Conductos de alimentación desde Galería GCH4 hacia la cámara del Swirler
    float fDistToCenter = pos.Length();
    if (fDistToCenter > 0.1f)
    {
        Vector2 vDirToCenter = Vector2.Normalize(-pos);

        // Tap directo desde la galería de GCH4
        Vector3 pGch4Gallery = new Vector3(
            pos.X - vDirToCenter.X * Math.Abs(fGch4GalleryRadius - fDistToCenter),
            pos.Y - vDirToCenter.Y * Math.Abs(fGch4GalleryRadius - fDistToCenter),
            fGch4GalleryZ
        );

        Vector3 pInjectorGch4Port = new Vector3(pos.X, pos.Y, fInjectorStartZ + fInjectorLength - 8.0f);
        Voxels voxGch4Feeder = voxCreateCylindricalDuct(pGch4Gallery, pInjectorGch4Port, fRadiusMm: 2.2f);
        voxAllFeedChannels.Add(voxGch4Feeder);

        // Tap directo desde la galería de GOX hacia el tubo central
        Vector3 pGoxGallery = new Vector3(
            pos.X - vDirToCenter.X * Math.Abs(fGoxGalleryRadius - fDistToCenter),
            pos.Y - vDirToCenter.Y * Math.Abs(fGoxGalleryRadius - fDistToCenter),
            fGoxGalleryZ
        );

        Vector3 pInjectorGoxPort = new Vector3(pos.X, pos.Y, fInjectorStartZ + fInjectorLength - 22.0f);
    }
}

```

```

        Voxels voxGoxFeeder = voxCreateCylindricalDuct(pGoxGallery, pInjectorGoxPort, fRadiusMm: 1.8f);
        voxAllFeedChannels.Add(voxGoxFeeder);
    }
}

// -----
// 4. ENSAMBLE FINAL MEDIANTE OPERACIONES BOOLEANAS
// -----
// Vaciar los conductos de alimentación de las galerías
voxAssembly.Subtract(voxAllFeedChannels);

// Fusionar los inyectores coaxiales al cuerpo
voxAssembly.Add(voxAllSwirlers);

return voxAssembly;
}

// =====
// MÉTODOS AUXILIARES DE PATRÓN GEOMÉTRICO Y COMPONENTES
// =====

/// <summary>
/// Calcula las coordenadas 2D de una distribución de 13 inyectores (1+6+6).
/// </summary>
private static List<Vector2> Get13ElementPattern(float fRadiusRing1, float fRadiusRing2)
{
    List<Vector2> list = new List<Vector2>();

    // 1. Inyector central
    list.Add(Vector2.Zero);

    // 2. Anillo Interior (6 inyectores separados 60°)
    for (int i = 0; i < 6; i++)
    {
        float angle = i * (float)Math.PI / 3.0f;
        list.Add(new Vector2(
            fRadiusRing1 * (float)Math.Cos(angle),
            fRadiusRing1 * (float)Math.Sin(angle)
        ));
    }

    // 3. Anillo Exterior (6 inyectores alternados a 60°)
    float fOffsetAngle = (float)Math.PI / 6.0f; // 30° desfasado
    for (int i = 0; i < 6; i++)
    {
        float angle = i * (float)Math.PI / 3.0f + fOffsetAngle;
        list.Add(new Vector2(
            fRadiusRing2 * (float)Math.Cos(angle),
            fRadiusRing2 * (float)Math.Sin(angle)
        ));
    }

    return list;
}

private static Voxels voxCreateSwirlInjectorElement(
    float fCoreTubeInnerRadiusMm,
    float fCoreTubeOuterRadiusMm,
    float fSwirlerOuterRadiusMm,
    float fElementLengthMm,
    int nVaneCount,
    float fHelixPitchMm)
{
    // Tubo central GOX
    Mesh meshCoreOuter = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
    Mesh meshCoreInner = CreateCylinderMesh(fCoreTubeInnerRadiusMm, fElementLengthMm + 4.0f, -2.0f);
    Voxels voxCoreTube = new Voxels(meshCoreOuter);
    voxCoreTube.Subtract(new Voxels(meshCoreInner));

    // Cámara anular GCH4
    Mesh meshSleeveOuter = CreateCylinderMesh(fSwirlerOuterRadiusMm, fElementLengthMm);
    Mesh meshSleeveInner = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
    Voxels voxSwirlChamber = new Voxels(meshSleeveOuter);
    voxSwirlChamber.Subtract(new Voxels(meshSleeveInner));

    // Álabes helicoidales
    Voxels voxVanes = voxCreateHelicalVanes(
        fCoreTubeOuterRadiusMm, fSwirlerOuterRadiusMm, fElementLengthMm, nVaneCount, fHelixPitchMm);
}

```

```

        voxSwirlChamber.Intersect(voxVanes);

        Voxels voxElement = new Voxels();
        voxElement.Add(voxCoreTube);
        voxElement.Add(voxSwirlChamber);

        return voxElement;
    }

    private static Voxels voxCreateHelicalVanes(
        float fRInner, float fROuter, float fLength, int nVanes, float fPitchMm)
    {
        Voxels voxVanesGroup = new Voxels();
        float fVaneThicknessMm = 0.8f;
        int nStepsZ = 20;
        float fDeltaZ = fLength / nStepsZ;

        for (int v = 0; v < nVanes; v++)
        {
            float fBaseAngle = v * (2.0f * (float)Math.PI / nVanes);
            Mesh meshVane = new Mesh();

            for (int i = 0; i <= nStepsZ; i++)
            {
                float fZ = i * fDeltaZ;
                float fAngle = fBaseAngle + (fZ / fPitchMm) * 2.0f * (float)Math.PI;

                Vector3 vRadDir = new Vector3((float)Math.Cos(fAngle), (float)Math.Sin(fAngle), 0);
                meshVane.nAddVertex(new Vector3(fRInner * vRadDir.X, fRInner * vRadDir.Y, fZ));
                meshVane.nAddVertex(new Vector3(fROuter * vRadDir.X, fROuter * vRadDir.Y, fZ));
            }

            for (int i = 0; i < nStepsZ; i++)
            {
                int i0 = i * 2, i1 = i0 + 1, i2 = (i + 1) * 2, i3 = i2 + 1;
                meshVane.AddTriangle(i0, i1, i3);
                meshVane.AddTriangle(i0, i3, i2);
            }

            Voxels voxSingleVane = new Voxels(meshVane);
            voxSingleVane.DoOffset(fVaneThicknessMm * 0.5f);
            voxVanesGroup.Add(voxSingleVane);
        }

        return voxVanesGroup;
    }

    private static Voxels voxCreateRevolvedGallery(
        float fMajorRadiusMm, float fZOffsetMm, float fWidthMm, float fHeightMm, int nCircleSteps = 48)
    {
        Mesh meshGallery = new Mesh();
        float dTheta = 2.0f * (float)Math.PI / nCircleSteps;

        Vector2[] profile2D = new Vector2[]
        {
            new Vector2(0, fHeightMm * 0.5f),
            new Vector2(fWidthMm * 0.5f, 0),
            new Vector2(0, -fHeightMm * 0.5f),
            new Vector2(-fWidthMm * 0.5f, 0)
        };

        int nProfilePoints = profile2D.Length;

        for (int s = 0; s <= nCircleSteps; s++)
        {
            float theta = s * dTheta;
            float cosT = (float)Math.Cos(theta);
            float sinT = (float)Math.Sin(theta);

            for (int p = 0; p < nProfilePoints; p++)
            {
                float r = fMajorRadiusMm + profile2D[p].X;
                float z = fZOffsetMm + profile2D[p].Y;

                meshGallery.nAddVertex(new Vector3(r * cosT, r * sinT, z));
            }
        }

        for (int s = 0; s < nCircleSteps; s++)
    
```

```

    {
        for (int p = 0; p < nProfilePoints; p++)
        {
            int pNext = (p + 1) % nProfilePoints;
            int idx00 = s * nProfilePoints + p;
            int idx10 = (s + 1) * nProfilePoints + p;
            int idx01 = s * nProfilePoints + pNext;
            int idx11 = (s + 1) * nProfilePoints + pNext;

            meshGallery.AddTriangle(idx00, idx10, idx11);
            meshGallery.AddTriangle(idx00, idx11, idx01);
        }
    }

    return new Voxels(meshGallery);
}

private static Voxels voxCreateBaseHousing(float fRadius, float fHeight)
{
    Mesh mesh = CreateCylinderMesh(fRadius, fHeight);
    return new Voxels(mesh);
}

private static Voxels voxCreateCylindricalDuct(Vector3 pStart, Vector3 pEnd, float fRadiusMm)
{
    Mesh mesh = new Mesh();
    Vector3 vDir = Vector3.Normalize(pEnd - pStart);
    float fLength = (pEnd - pStart).Length();

    Vector3 vUp = Math.Abs(vDir.Z) > 0.9f ? Vector3.UnitY : Vector3.UnitZ;
    Vector3 vTangent = Vector3.Normalize(Vector3.Cross(vDir, vUp));
    Vector3 vBinormal = Vector3.Normalize(Vector3.Cross(vTangent, vDir));

    int nSteps = 16;
    float dTheta = 2.0f * (float)Math.PI / nSteps;

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * dTheta;
        Vector3 vOffset = fRadiusMm * ((float)Math.Cos(theta) * vTangent + (float)Math.Sin(theta) * vBinormal);

        int v0 = mesh.nAddVertex(pStart + vOffset);
        int v1 = mesh.nAddVertex(pEnd + vOffset);
    }

    Voxels voxDuct = new Voxels(mesh);
    voxDuct.DoOffset(fRadiusMm * 0.5f);
    return voxDuct;
}

private static Mesh CreateCylinderMesh(float fRadius, float fHeight, float fStartZ = 0.0f, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int vCenterBot = mesh.nAddVertex(new Vector3(0, 0, fStartZ));
    int vCenterTop = mesh.nAddVertex(new Vector3(0, 0, fStartZ + fHeight));

    int[] botIndices = new int[nSteps];
    int[] topIndices = new int[nSteps];

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fDTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        botIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ));
        topIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ + fHeight));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
        mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
        mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
        mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
    }
}

```

```

        return mesh;
    }
}
}

```

Detalles Destacados de la Arquitectura del Modelo

1. **Topología Integrada de Fluido:** La función `Get13ElementPattern` calcula la ubicación radial exacta de cada inyector. Para cada elemento, la subrutina calcula vectores tridimensionales desde las galerías toroidales (`voxCreateCylindricalDuct`) que perforan el cuerpo principal para alimentar de forma balanceada el oxidante y el combustible a cada inyector.
2. **Eficiencia Booleana Voxelizada:** Al trabajar directamente sobre vóxeles con **PikoGK**, la unión de 13 swirlers con sus rampas helicoidales y microconductos internos evita los errores geométrica y computacionalmente costosos de intersección de mallas tradicionales (B-Rep/STL).
3. **Impresión 3D Monolítica (LPBF):** Todo el conjunto se exporta en una única pieza sólida sin necesidad de soldaduras entre inyectores y placa, maximizando la rigidez estructural ante altas presiones y pulsaciones acústicas de combustión.

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectores coaxiales.

Response: Este código en C# para **PikoGK** realiza el ensamble completo de la cabeza del inyector. Integra el cuerpo exterior, las **galerías internas de distribución de GCH₄ y GOX** y una **matriz radial de 13 inyectores coaxiales** (1 central + 6 en anillo interno + 6 en anillo externo) con sus respectivos canales de alimentación pasantes.

Código C#: Ensamble Completo del Inyector Multielemento

```

using System;
using System.Numerics;
using System.Collections.Generic;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssemblyGenerator
    {
        /// <summary>
        /// Genera el ensamble monolítico completo del inyector de cohete,
        /// incluyendo cuerpo principal, galerías internas de distribución
        /// y 13 elementos inyectores coaxiales orientados radialmente.
        /// </summary>
        public static Voxels voxCreateFullInjectorAssembly(
            float fHousingOuterRadiusMm = 75.0f,
            float fHousingHeightMm = 85.0f,
            float fInnerRingRadiusMm = 28.0f,
            float fOuterRingRadiusMm = 52.0f)
        {
            // 1. Generar el cuerpo base del inyector con sus galerías de distribución
            Voxels voxHousing = InjectorManifoldsGenerator.voxCreateInjectorHousingWithGalleries(
                fHousingOuterRadiusMm: fHousingOuterRadiusMm,
                fHousingHeightMm: fHousingHeightMm,
                fGch4GalleryRadiusMm: fOuterRingRadiusMm,
                fGoxGalleryRadiusMm: fInnerRingRadiusMm
            );

            // 2. Definir las posiciones de la matriz de 13 inyectores (1 Centro, 6 Anillo Interno, 6 Anillo Externo)
            List<Vector3> listInjectorPositions = GetInjectorPatternPositions(
                fInnerRingRadiusMm,
                fOuterRingRadiusMm
            );

            Voxels voxInjectorsGroup = new Voxels();
            Voxels voxInjectorDrillPassages = new Voxels();

            // 3. Generar e integrar cada elemento inyector coaxial en sus coordenadas
            foreach (Vector3 vPos in listInjectorPositions)
            {
                // Elemento inyector coaxial con swirlers desplazado a la posición (vPos.X, vPos.Y)
                Voxels voxSingleInjector = voxCreatePositionedInjectorElement(
                    vOffset: vPos,
                    fCoreTubeInnerRadiusMm: 2.0f,
                    fCoreTubeOuterRadiusMm: 3.2f,
                    fSwirlerOuterRadiusMm: 6.5f,
                    fElementLengthMm: fHousingHeightMm + 10.0f
                );
                voxInjectorsGroup.Add(voxSingleInjector);

                // Perforaciones de alimentacion vertical desde galerias a la camara
            }
        }
    }
}

```

```

        Mesh meshPassage = CreateCylinderMeshAtPosition(
            vCenter: vPos,
            fRadius: 6.6f,
            fHeight: fHousingHeightMm + 14.0f,
            fZStart: -7.0f
        );
        voxInjectorDrillPassages.Add(new Voxels(meshPassage));
    }

    // 4. Operación Booleana: Taladrar los alojamientos de los inyectores en el cuerpo principal
    voxHousing.Subtract(voxInjectorDrillPassages);

    // 5. Operación Booleana: Fusionar la matriz de inyectores coaxiales en el bloque
    voxHousing.Add(voxInjectorsGroup);

    return voxHousing;
}

/// <summary>
/// Calculador del patrón espacial de 13 inyectores (1 central, 6 en radio interno, 6 en radio externo).
/// </summary>
private static List<Vector3> GetInjectorPatternPositions(float fR1, float fR2)
{
    List<Vector3> positions = new List<Vector3>();

    // Inyector central (0, 0)
    positions.Add(new Vector3(0.0f, 0.0f, 0.0f));

    // Anillo interior: 6 inyectores separados por 60 grados
    for (int i = 0; i < 6; i++)
    {
        float fAngle = i * (float)(Math.PI / 3.0);
        positions.Add(new Vector3(
            fR1 * (float)Math.Cos(fAngle),
            fR1 * (float)Math.Sin(fAngle),
            0.0f
        ));
    }

    // Anillo exterior: 6 inyectores desfasados 30 grados respecto al interior
    for (int i = 0; i < 6; i++)
    {
        float fAngle = (i * (float)(Math.PI / 3.0)) + (float)(Math.PI / 6.0);
        positions.Add(new Vector3(
            fR2 * (float)Math.Cos(fAngle),
            fR2 * (float)Math.Sin(fAngle),
            0.0f
        ));
    }

    return positions;
}

/// <summary>
/// Genera la geometría de un inyector coaxial individual desplazado espacialmente.
/// </summary>
private static Voxels voxCreatePositionedInjectorElement(
    Vector3 vOffset,
    float fCoreTubeInnerRadiusMm,
    float fCoreTubeOuterRadiusMm,
    float fSwirlerOuterRadiusMm,
    float fElementLengthMm)
{
    // Malla del tubo central desplazada
    Mesh meshCoreOuter = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -5.
    Mesh meshCoreInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeInnerRadiusMm, fElementLengthMm + 10

    Voxels voxCore = new Voxels(meshCoreOuter);
    voxCore.Subtract(new Voxels(meshCoreInner));

    // Malla de la manga anular externa desplazada
    Mesh meshSleeveOuter = CreateCylinderMeshAtPosition(vOffset, fSwirlerOuterRadiusMm, fElementLengthMm, -5
    Mesh meshSleeveInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -

    Voxels voxSwirlChannel = new Voxels(meshSleeveOuter);
    voxSwirlChannel.Subtract(new Voxels(meshSleeveInner));

    // Combinación final del elemento
    Voxels voxElement = new Voxels();

```

```

        voxElement.Add(voxCore);
        voxElement.Add(voxSwirlChannel);

        return voxElement;
    }

    private static Mesh CreateCylinderMeshAtPosition(
        Vector3 vCenter, float fRadius, float fHeight, float fZStart = 0.0f, int nSteps = 32)
    {
        Mesh mesh = new Mesh();
        float fDTheta = 2.0f * (float)Math.PI / nSteps;

        int vCenterBot = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart));
        int vCenterTop = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart + fHeight));

        int[] botIndices = new int[nSteps];
        int[] topIndices = new int[nSteps];

        for (int s = 0; s < nSteps; s++)
        {
            float theta = s * fDTheta;
            float cosT = (float)Math.Cos(theta);
            float sinT = (float)Math.Sin(theta);

            float x = vCenter.X + fRadius * cosT;
            float y = vCenter.Y + fRadius * sinT;

            botIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart));
            topIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart + fHeight));
        }

        for (int s = 0; s < nSteps; s++)
        {
            int sNext = (s + 1) % nSteps;
            mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
            mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
            mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
            mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
        }

        return mesh;
    }
}

```

Arquitectura Constructiva y Flujo Metodológico

- Patrón Multielemento Distribuido:** Mediante la función `GetInjectorPatternPositions`, el algoritmo establece 13 coordenadas cartesianas alineadas concéntricamente con los radios medios de las galerías toroide de distribución de oxidante (R_1) y combustible (R_2).
- Secuencia de Operaciones Booleanas de Voxelizado:**
 - Paso A:** Generación del cuerpo cónico monolítico con cavidades de galerías en perfil de gota/diamante autosoportado para impresión 3D en aleación de níquel o Inconel.
 - Paso B:** Extracción mediante `Subtract` de los 13 puertos verticales de alojamiento para interconectar los toroides de distribución con la zona de mezclado.
 - Paso C:** Fusión mediante `Add` de los 13 inyectores coaxiales en una sola representación de espacio implícito de vóxeles libre de brechas geométricas (*watertight*).

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectores coaxiales.

Response: Este código en C# para **PikoGK** ensambla de forma completa la **cabeza del inyector**. Unifica el cuerpo exterior, las galerías internas de distribución (GOX y GCH₄) y una matriz de **13 inyectores coaxiales con álabes helicoidales** (1 central, 6 en el anillo interior y 6 en el anillo exterior), junto con los microconductos de alimentación que conectan cada galería con su respectivo inyector.

Código C#: Ensamble Completo del Inyector Multielemento

```

using System;
using System.Collections.Generic;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssembly
    {

```

```

{
    /// <summary>
    /// Genera el ensamble monolítico completo del inyector:
    /// - Cuerpo estructural con cara de inyección
    /// - Galerías toroidales de distribución (GOX azul, GCH4 naranja)
    /// - Matriz de 13 elementos inyectores coaxiales con swirlers helicoidales
    /// - Canales internos de interconexión entre galerías e inyectores
    /// </summary>
    public static Voxels voxAssembleCompleteInjectorHead(
        float fHousingRadiusMm = 75.0f,
        float fHousingHeightMm = 90.0f,
        float fInnerRingRadiusMm = 24.0f,
        float fOuterRingRadiusMm = 48.0f)
    {
        // -----
        // 1. CUERPO BASE Y GALERÍAS DE DISTRIBUCIÓN
        // -----
        // Generar el bloque sólido exterior del housing
        Voxels voxAssembly = voxCreateBaseHousing(fHousingRadiusMm, fHousingHeightMm);

        // Generar la galería exterior de GCH4 (Metano gaseoso)
        float fGch4GalleryRadius = 50.0f;
        float fGch4GalleryZ = 55.0f;
        Voxels voxGch4Gallery = voxCreateRevolvedGallery(
            fMajorRadiusMm: fGch4GalleryRadius,
            fZOffsetMm: fGch4GalleryZ,
            fWidthMm: 16.0f,
            fHeightMm: 22.0f
        );

        // Generar la galería interior de GOX (Oxígeno gaseoso)
        float fGoxGalleryRadius = 28.0f;
        float fGoxGalleryZ = 35.0f;
        Voxels voxGoxGallery = voxCreateRevolvedGallery(
            fMajorRadiusMm: fGoxGalleryRadius,
            fZOffsetMm: fGoxGalleryZ,
            fWidthMm: 14.0f,
            fHeightMm: 18.0f
        );

        // Vaciado de las galerías dentro del cuerpo principal (Boolean Subtract)
        voxAssembly.Subtract(voxGch4Gallery);
        voxAssembly.Subtract(voxGoxGallery);

        // -----
        // 2. GENERACIÓN DE LAS POSICIONES DE LA MATRIZ DE 13 INYECTORES
        // -----
        List<Vector2> injectorPositions = Get13ElementPattern(fInnerRingRadiusMm, fOuterRingRadiusMm);

        Voxels voxAllSwirlers = new Voxels();
        Voxels voxAllFeedChannels = new Voxels();

        float fInjectorLength = 45.0f;
        float fInjectorStartZ = 10.0f; // Altura de la cara del inyector

        // -----
        // 3. CONSTRUCCIÓN E INTEGRACIÓN DE CADA INYECTOR COAXIAL
        // -----
        foreach (Vector2 pos in injectorPositions)
        {
            Vector3 vPos3D = new Vector3(pos.X, pos.Y, fInjectorStartZ);

            // A) Elemento inyector coaxial (Tubo GOX + Swirler GCH4)
            Voxels voxSingleInjector = voxCreateSwirlInjectorElement(
                fCoreTubeInnerRadiusMm: 2.0f,
                fCoreTubeOuterRadiusMm: 3.2f,
                fSwirlerOuterRadiusMm: 6.5f,
                fElementLengthMm: fInjectorLength,
                nVaneCount: 6,
                fHelixPitchMm: 20.0f
            );

            // Posicionar inyector en la matriz
            voxSingleInjector.Translate(vPos3D);
            voxAllSwirlers.Add(voxSingleInjector);

            // B) Conductos de alimentación desde Galería GCH4 hacia la cámara del Swirler
            float fDistToCenter = pos.Length();
            if (fDistToCenter > 0.1f)

```



```

    {
        Vector2 vDirToCenter = Vector2.Normalize(-pos);

        // Tap directo desde la galería de GCH4
        Vector3 pGch4Gallery = new Vector3(
            pos.X - vDirToCenter.X * Math.Abs(fGch4GalleryRadius - fDistToCenter),
            pos.Y - vDirToCenter.Y * Math.Abs(fGch4GalleryRadius - fDistToCenter),
            fGch4GalleryZ
        );

        Vector3 pInjectorGch4Port = new Vector3(pos.X, pos.Y, fInjectorStartZ + fInjectorLength - 8.0f);
        Voxels voxGch4Feeder = voxCreateCylindricalDuct(pGch4Gallery, pInjectorGch4Port, fRadiusMm: 2.2f);
        voxAllFeedChannels.Add(voxGch4Feeder);

        // Tap directo desde la galería de GOX hacia el tubo central
        Vector3 pGoxGallery = new Vector3(
            pos.X - vDirToCenter.X * Math.Abs(fGoxGalleryRadius - fDistToCenter),
            pos.Y - vDirToCenter.Y * Math.Abs(fGoxGalleryRadius - fDistToCenter),
            fGoxGalleryZ
        );

        Vector3 pInjectorGoxPort = new Vector3(pos.X, pos.Y, fInjectorStartZ + fInjectorLength - 22.0f);
        Voxels voxGoxFeeder = voxCreateCylindricalDuct(pGoxGallery, pInjectorGoxPort, fRadiusMm: 1.8f);
        voxAllFeedChannels.Add(voxGoxFeeder);
    }
}

// -----
// 4. ENSAMBLE FINAL MEDIANTE OPERACIONES BOOLEANAS
// -----
// Vaciar los conductos de alimentación de las galerías
voxAssembly.Subtract(voxAllFeedChannels);

// Fusionar los inyectores coaxiales al cuerpo
voxAssembly.Add(voxAllSwirlers);

return voxAssembly;
}

// =====
// MÉTODOS AUXILIARES DE PATRÓN GEOMÉTRICO Y COMPONENTES
// =====

/// <summary>
/// Calcula las coordenadas 2D de una distribución de 13 inyectores (1+6+6).
/// </summary>
private static List<Vector2> Get13ElementPattern(float fRadiusRing1, float fRadiusRing2)
{
    List<Vector2> list = new List<Vector2>();

    // 1. Inyector central
    list.Add(Vector2.Zero);

    // 2. Anillo Interior (6 inyectores separados 60°)
    for (int i = 0; i < 6; i++)
    {
        float angle = i * (float)Math.PI / 3.0f;
        list.Add(new Vector2(
            fRadiusRing1 * (float)Math.Cos(angle),
            fRadiusRing1 * (float)Math.Sin(angle)
        ));
    }

    // 3. Anillo Exterior (6 inyectores alternados a 60°)
    float fOffsetAngle = (float)Math.PI / 6.0f; // 30° desfasado
    for (int i = 0; i < 6; i++)
    {
        float angle = i * (float)Math.PI / 3.0f + fOffsetAngle;
        list.Add(new Vector2(
            fRadiusRing2 * (float)Math.Cos(angle),
            fRadiusRing2 * (float)Math.Sin(angle)
        ));
    }

    return list;
}

private static Voxels voxCreateSwirlInjectorElement(
    float fCoreTubeInnerRadiusMm,

```

```

float fCoreTubeOuterRadiusMm,
float fSwirlerOuterRadiusMm,
float fElementLengthMm,
int nVaneCount,
float fHelixPitchMm)
{
    // Tubo central GOX
    Mesh meshCoreOuter = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
    Mesh meshCoreInner = CreateCylinderMesh(fCoreTubeInnerRadiusMm, fElementLengthMm + 4.0f, -2.0f);
    Voxels voxCoreTube = new Voxels(meshCoreOuter);
    voxCoreTube.Subtract(new Voxels(meshCoreInner));

    // Cámara anular GCH4
    Mesh meshSleeveOuter = CreateCylinderMesh(fSwirlerOuterRadiusMm, fElementLengthMm);
    Mesh meshSleeveInner = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
    Voxels voxSwirlChamber = new Voxels(meshSleeveOuter);
    voxSwirlChamber.Subtract(new Voxels(meshSleeveInner));

    // Álabes helicoidales
    Voxels voxVanes = voxCreateHelicalVanes(
        fCoreTubeOuterRadiusMm, fSwirlerOuterRadiusMm, fElementLengthMm, nVaneCount, fHelixPitchMm);

    voxSwirlChamber.Intersect(voxVanes);

    Voxels voxElement = new Voxels();
    voxElement.Add(voxCoreTube);
    voxElement.Add(voxSwirlChamber);

    return voxElement;
}

private static Voxels voxCreateHelicalVanes(
    float fRInner, float fROuter, float fLength, int nVanes, float fPitchMm)
{
    Voxels voxVanesGroup = new Voxels();
    float fVaneThicknessMm = 0.8f;
    int nStepsZ = 20;
    float fDeltaZ = fLength / nStepsZ;

    for (int v = 0; v < nVanes; v++)
    {
        float fBaseAngle = v * (2.0f * (float)Math.PI / nVanes);
        Mesh meshVane = new Mesh();

        for (int i = 0; i <= nStepsZ; i++)
        {
            float fZ = i * fDeltaZ;
            float fAngle = fBaseAngle + (fZ / fPitchMm) * 2.0f * (float)Math.PI;

            Vector3 vRadDir = new Vector3((float)Math.Cos(fAngle), (float)Math.Sin(fAngle), 0);
            meshVane.nAddVertex(new Vector3(fRInner * vRadDir.X, fRInner * vRadDir.Y, fZ));
            meshVane.nAddVertex(new Vector3(fROuter * vRadDir.X, fROuter * vRadDir.Y, fZ));
        }

        for (int i = 0; i < nStepsZ; i++)
        {
            int i0 = i * 2, i1 = i0 + 1, i2 = (i + 1) * 2, i3 = i2 + 1;
            meshVane.AddTriangle(i0, i1, i3);
            meshVane.AddTriangle(i0, i3, i2);
        }

        Voxels voxSingleVane = new Voxels(meshVane);
        voxSingleVane.DoOffset(fVaneThicknessMm * 0.5f);
        voxVanesGroup.Add(voxSingleVane);
    }

    return voxVanesGroup;
}

private static Voxels voxCreateRevolvedGallery(
    float fMajorRadiusMm, float fZOffsetMm, float fWidthMm, float fHeightMm, int nCircleSteps = 48)
{
    Mesh meshGallery = new Mesh();
    float dTheta = 2.0f * (float)Math.PI / nCircleSteps;

    Vector2[] profile2D = new Vector2[]
    {
        new Vector2(0, fHeightMm * 0.5f),
        new Vector2(fWidthMm * 0.5f, 0),
    }

```

```

        new Vector2(0, -fHeightMm * 0.5f),
        new Vector2(-fWidthMm * 0.5f, 0)
    };

    int nProfilePoints = profile2D.Length;

    for (int s = 0; s <= nCircleSteps; s++)
    {
        float theta = s * dTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        for (int p = 0; p < nProfilePoints; p++)
        {
            float r = fMajorRadiusMm + profile2D[p].X;
            float z = fZOffsetMm + profile2D[p].Y;

            meshGallery.nAddVertex(new Vector3(r * cosT, r * sinT, z));
        }
    }

    for (int s = 0; s < nCircleSteps; s++)
    {
        for (int p = 0; p < nProfilePoints; p++)
        {
            int pNext = (p + 1) % nProfilePoints;
            int idx00 = s * nProfilePoints + p;
            int idx10 = (s + 1) * nProfilePoints + p;
            int idx01 = s * nProfilePoints + pNext;
            int idx11 = (s + 1) * nProfilePoints + pNext;

            meshGallery.AddTriangle(idx00, idx10, idx11);
            meshGallery.AddTriangle(idx00, idx11, idx01);
        }
    }

    return new Voxels(meshGallery);
}

private static Voxels voxCreateBaseHousing(float fRadius, float fHeight)
{
    Mesh mesh = CreateCylinderMesh(fRadius, fHeight);
    return new Voxels(mesh);
}

private static Voxels voxCreateCylindricalDuct(Vector3 pStart, Vector3 pEnd, float fRadiusMm)
{
    Mesh mesh = new Mesh();
    Vector3 vDir = Vector3.Normalize(pEnd - pStart);
    float fLength = (pEnd - pStart).Length();

    Vector3 vUp = Math.Abs(vDir.Z) > 0.9f ? Vector3.UnitY : Vector3.UnitZ;
    Vector3 vTangent = Vector3.Normalize(Vector3.Cross(vDir, vUp));
    Vector3 vBinormal = Vector3.Normalize(Vector3.Cross(vTangent, vDir));

    int nSteps = 16;
    float dTheta = 2.0f * (float)Math.PI / nSteps;

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * dTheta;
        Vector3 vOffset = fRadiusMm * ((float)Math.Cos(theta) * vTangent + (float)Math.Sin(theta) * vBinormal);

        int v0 = mesh.nAddVertex(pStart + vOffset);
        int v1 = mesh.nAddVertex(pEnd + vOffset);
    }

    Voxels voxDuct = new Voxels(mesh);
    voxDuct.DoOffset(fRadiusMm * 0.5f);
    return voxDuct;
}

private static Mesh CreateCylinderMesh(float fRadius, float fHeight, float fStartZ = 0.0f, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int vCenterBot = mesh.nAddVertex(new Vector3(0, 0, fStartZ));
    int vCenterTop = mesh.nAddVertex(new Vector3(0, 0, fStartZ + fHeight));

```

```

int[] botIndices = new int[nSteps];
int[] topIndices = new int[nSteps];

for (int s = 0; s < nSteps; s++)
{
    float theta = s * fDTheta;
    float cosT = (float)Math.Cos(theta);
    float sinT = (float)Math.Sin(theta);

    botIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ));
    topIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ + fHeight));
}

for (int s = 0; s < nSteps; s++)
{
    int sNext = (s + 1) % nSteps;
    mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
    mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
    mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
    mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
}

return mesh;
}
}
}

```

Detalles Destacados de la Arquitectura del Modelo

- Topología Integrada de Fluido:** La función `Get13ElementPattern` calcula la ubicación radial exacta de cada inyector. Para cada elemento, la subrutina calcula vectores tridimensionales desde las galerías toroidales (`voxCreateCylindricalDuct`) que perforan el cuerpo principal para alimentar de forma balanceada el oxidante y el combustible a cada inyector.
- Eficiencia Booleana Voxelizada:** Al trabajar directamente sobre vóxeles con **PikoGK**, la unión de 13 swirlers con sus rampas helicoidales y microconductos internos evita los errores geométrica y computacionalmente costosos de intersección de mallas tradicionales (B-Rep/STL).
- Impresión 3D Monolítica (LPBF):** Todo el conjunto se exporta en una única pieza sólida sin necesidad de soldaduras entre inyectores y placa, maximizando la rigidez estructural ante altas presiones y pulsaciones acústicas de combustión.

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectores coaxiales.

Response: Este código en C# para **PikoGK** realiza el ensamble completo de la cabeza del inyector. Integra el cuerpo exterior, las **galerías internas de distribución de GCH₄ y GOX** y una **matriz radial de 13 inyectores coaxiales** (1 central + 6 en anillo interno + 6 en anillo externo) con sus respectivos canales de alimentación pasantes.

Código C#: Ensamble Completo del Inyector Multielemento

```

using System;
using System.Numerics;
using System.Collections.Generic;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssemblyGenerator
    {
        /// <summary>
        /// Genera el ensamble monolítico completo del inyector de cohete,
        /// incluyendo cuerpo principal, galerías internas de distribución
        /// y 13 elementos inyectores coaxiales orientados radialmente.
        /// </summary>
        public static Voxels voxCreateFullInjectorAssembly(
            float fHousingOuterRadiusMm = 75.0f,
            float fHousingHeightMm = 85.0f,
            float fInnerRingRadiusMm = 28.0f,
            float fOuterRingRadiusMm = 52.0f)
        {
            // 1. Generar el cuerpo base del inyector con sus galerías de distribución
            Voxels voxHousing = InjectorManifoldsGenerator.voxCreateInjectorHousingWithGalleries(
                fHousingOuterRadiusMm: fHousingOuterRadiusMm,
                fHousingHeightMm: fHousingHeightMm,
                fGch4GalleryRadiusMm: fOuterRingRadiusMm,
                fGoxGalleryRadiusMm: fInnerRingRadiusMm
            );

            // 2. Definir las posiciones de la matriz de 13 inyectores (1 Centro, 6 Anillo Interno, 6 Anillo Externo

```

```

List<Vector3> listInjectorPositions = GetInjectorPatternPositions(
    fInnerRingRadiusMm,
    fOuterRingRadiusMm
);

Voxels voxInjectorsGroup = new Voxels();
Voxels voxInjectorDrillPassages = new Voxels();

// 3. Generar e integrar cada elemento inyector coaxial en sus coordenadas
foreach (Vector3 vPos in listInjectorPositions)
{
    // Elemento inyector coaxial con swirlers desplazado a la posición (vPos.X, vPos.Y)
    Voxels voxSingleInjector = voxCreatePositionedInjectorElement(
        vOffset: vPos,
        fCoreTubeInnerRadiusMm: 2.0f,
        fCoreTubeOuterRadiusMm: 3.2f,
        fSwirlerOuterRadiusMm: 6.5f,
        fElementLengthMm: fHousingHeightMm + 10.0f
    );
    voxInjectorsGroup.Add(voxSingleInjector);

    // Perforaciones de alimentacion vertical desde galerias a la camara
    Mesh meshPassage = CreateCylinderMeshAtPosition(
        vCenter: vPos,
        fRadius: 6.6f,
        fHeight: fHousingHeightMm + 14.0f,
        fZStart: -7.0f
    );
    voxInjectorDrillPassages.Add(new Voxels(meshPassage));
}

// 4. Operación Booleana: Taladrar los alojamientos de los inyectores en el cuerpo principal
voxHousing.Subtract(voxInjectorDrillPassages);

// 5. Operación Booleana: Fusionar la matriz de inyectores coaxiales en el bloque
voxHousing.Add(voxInjectorsGroup);

return voxHousing;
}

/// <summary>
/// Calculador del patrón espacial de 13 inyectores (1 central, 6 en radio interno, 6 en radio externo).
/// </summary>
private static List<Vector3> GetInjectorPatternPositions(float fR1, float fR2)
{
    List<Vector3> positions = new List<Vector3>();

    // Inyector central (0, 0)
    positions.Add(new Vector3(0.0f, 0.0f, 0.0f));

    // Anillo interior: 6 inyectores separados por 60 grados
    for (int i = 0; i < 6; i++)
    {
        float fAngle = i * (float)(Math.PI / 3.0);
        positions.Add(new Vector3(
            fR1 * (float)Math.Cos(fAngle),
            fR1 * (float)Math.Sin(fAngle),
            0.0f
        ));
    }

    // Anillo exterior: 6 inyectores desfasados 30 grados respecto al interior
    for (int i = 0; i < 6; i++)
    {
        float fAngle = (i * (float)(Math.PI / 3.0)) + (float)(Math.PI / 6.0);
        positions.Add(new Vector3(
            fR2 * (float)Math.Cos(fAngle),
            fR2 * (float)Math.Sin(fAngle),
            0.0f
        ));
    }

    return positions;
}

/// <summary>
/// Genera la geometría de un inyector coaxial individual desplazado espacialmente.
/// </summary>
private static Voxels voxCreatePositionedInjectorElement(

```

```

Vector3 vOffset,
float fCoreTubeInnerRadiusMm,
float fCoreTubeOuterRadiusMm,
float fSwirlerOuterRadiusMm,
float fElementLengthMm)
{
    // Malla del tubo central desplazada
    Mesh meshCoreOuter = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -5.
    Mesh meshCoreInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeInnerRadiusMm, fElementLengthMm + 10

    Voxels voxCore = new Voxels(meshCoreOuter);
    voxCore.Subtract(new Voxels(meshCoreInner));

    // Malla de la manga anular externa desplazada
    Mesh meshSleeveOuter = CreateCylinderMeshAtPosition(vOffset, fSwirlerOuterRadiusMm, fElementLengthMm, -5
    Mesh meshSleeveInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -

    Voxels voxSwirlChannel = new Voxels(meshSleeveOuter);
    voxSwirlChannel.Subtract(new Voxels(meshSleeveInner));

    // Combinación final del elemento
    Voxels voxElement = new Voxels();
    voxElement.Add(voxCore);
    voxElement.Add(voxSwirlChannel);

    return voxElement;
}

private static Mesh CreateCylinderMeshAtPosition(
    Vector3 vCenter, float fRadius, float fHeight, float fZStart = 0.0f, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int vCenterBot = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart));
    int vCenterTop = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart + fHeight));

    int[] botIndices = new int[nSteps];
    int[] topIndices = new int[nSteps];

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fDTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        float x = vCenter.X + fRadius * cosT;
        float y = vCenter.Y + fRadius * sinT;

        botIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart));
        topIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart + fHeight));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
        mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
        mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
        mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
    }

    return mesh;
}
}
}

```

Arquitectura Constructiva y Flujo Metodológico

- Patrón Multielemento Distribuido:** Mediante la función `GetInjectorPatternPositions`, el algoritmo establece 13 coordenadas cartesianas alineadas concéntricamente con los radios medios de las galerías toroide de distribución de oxidante (R_1) y combustible (R_2).
- Secuencia de Operaciones Booleanas de Voxelizado:**
 - Paso A:** Generación del cuerpo cónico monolítico con cavidades de galerías en perfil de gota/diamante autosoportado para impresión 3D en aleación de níquel o Inconel.
 - Paso B:** Extracción mediante `Subtract` de los 13 puertos verticales de alojamiento para interconectar los toroides de distribución con la zona de mezclado.

- **Paso C:** Fusión mediante Add de los 13 inyectores coaxiales en una sola representación de espacio implícito de vóxeles libre de brechas geométricas (*watertight*).

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectores coaxiales.

Response: Este código en C# para **PikoGK** ensambla de forma completa la **cabeza del inyector**. Unifica el cuerpo exterior, las galerías internas de distribución (GOX y GCH₄) y una matriz de **13 inyectores coaxiales con álabes helicoidales** (1 central, 6 en el anillo interior y 6 en el anillo exterior), junto con los microconductos de alimentación que conectan cada galería con su respectivo inyector.

Código C#: Ensamble Completo del Inyector Multielemento

```
using System;
using System.Collections.Generic;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssembly
    {
        /// <summary>
        /// Genera el ensamble monolítico completo del inyector:
        /// - Cuerpo estructural con cara de inyección
        /// - Galerías toroidales de distribución (GOX azul, GCH4 naranja)
        /// - Matriz de 13 elementos inyectores coaxiales con swirlers helicoidales
        /// - Canales internos de interconexión entre galerías e inyectores
        /// </summary>
        public static Voxels voxAssembleCompleteInjectorHead(
            float fHousingRadiusMm = 75.0f,
            float fHousingHeightMm = 90.0f,
            float fInnerRingRadiusMm = 24.0f,
            float fOuterRingRadiusMm = 48.0f)
        {
            // -----
            // 1. CUERPO BASE Y GALERÍAS DE DISTRIBUCIÓN
            // -----
            // Generar el bloque sólido exterior del housing
            Voxels voxAssembly = voxCreateBaseHousing(fHousingRadiusMm, fHousingHeightMm);

            // Generar la galería exterior de GCH4 (Metano gaseoso)
            float fGch4GalleryRadius = 50.0f;
            float fGch4GalleryZ = 55.0f;
            Voxels voxGch4Gallery = voxCreateRevolvedGallery(
                fMajorRadiusMm: fGch4GalleryRadius,
                fZOffsetMm: fGch4GalleryZ,
                fWidthMm: 16.0f,
                fHeightMm: 22.0f
            );

            // Generar la galería interior de GOX (Oxígeno gaseoso)
            float fGoxGalleryRadius = 28.0f;
            float fGoxGalleryZ = 35.0f;
            Voxels voxGoxGallery = voxCreateRevolvedGallery(
                fMajorRadiusMm: fGoxGalleryRadius,
                fZOffsetMm: fGoxGalleryZ,
                fWidthMm: 14.0f,
                fHeightMm: 18.0f
            );

            // Vaciado de las galerías dentro del cuerpo principal (Boolean Subtract)
            voxAssembly.Subtract(voxGch4Gallery);
            voxAssembly.Subtract(voxGoxGallery);

            // -----
            // 2. GENERACIÓN DE LAS POSICIONES DE LA MATRIZ DE 13 INYECTORES
            // -----
            List<Vector2> injectorPositions = Get13ElementPattern(fInnerRingRadiusMm, fOuterRingRadiusMm);

            Voxels voxAllSwirlers = new Voxels();
            Voxels voxAllFeedChannels = new Voxels();

            float fInjectorLength = 45.0f;
            float fInjectorStartZ = 10.0f; // Altura de la cara del inyector

            // -----
            // 3. CONSTRUCCIÓN E INTEGRACIÓN DE CADA INYECTOR COAXIAL
            // -----
        }
    }
}
```

```

foreach (Vector2 pos in injectorPositions)
{
    Vector3 vPos3D = new Vector3(pos.X, pos.Y, fInjectorStartZ);

    // A) Elemento inyector coaxial (Tubo GOX + Swirler GCH4)
    Voxels voxSingleInjector = voxCreateSwirlInjectorElement(
        fCoreTubeInnerRadiusMm: 2.0f,
        fCoreTubeOuterRadiusMm: 3.2f,
        fSwirlerOuterRadiusMm: 6.5f,
        fElementLengthMm: fInjectorLength,
        nVaneCount: 6,
        fHelixPitchMm: 20.0f
    );

    // Posicionar inyector en la matriz
    voxSingleInjector.Translate(vPos3D);
    voxAllSwirlers.Add(voxSingleInjector);

    // B) Conductos de alimentación desde Galería GCH4 hacia la cámara del Swirler
    float fDistToCenter = pos.Length();
    if (fDistToCenter > 0.1f)
    {
        Vector2 vDirToCenter = Vector2.Normalize(-pos);

        // Tap directo desde la galería de GCH4
        Vector3 pGch4Gallery = new Vector3(
            pos.X - vDirToCenter.X * Math.Abs(fGch4GalleryRadius - fDistToCenter),
            pos.Y - vDirToCenter.Y * Math.Abs(fGch4GalleryRadius - fDistToCenter),
            fGch4GalleryZ
        );

        Vector3 pInjectorGch4Port = new Vector3(pos.X, pos.Y, fInjectorStartZ + fInjectorLength - 8.0f);
        Voxels voxGch4Feeder = voxCreateCylindricalDuct(pGch4Gallery, pInjectorGch4Port, fRadiusMm: 2.2f);
        voxAllFeedChannels.Add(voxGch4Feeder);

        // Tap directo desde la galería de GOX hacia el tubo central
        Vector3 pGoxGallery = new Vector3(
            pos.X - vDirToCenter.X * Math.Abs(fGoxGalleryRadius - fDistToCenter),
            pos.Y - vDirToCenter.Y * Math.Abs(fGoxGalleryRadius - fDistToCenter),
            fGoxGalleryZ
        );

        Vector3 pInjectorGoxPort = new Vector3(pos.X, pos.Y, fInjectorStartZ + fInjectorLength - 22.0f);
        Voxels voxGoxFeeder = voxCreateCylindricalDuct(pGoxGallery, pInjectorGoxPort, fRadiusMm: 1.8f);
        voxAllFeedChannels.Add(voxGoxFeeder);
    }
}

// -----
// 4. ENSAMBLE FINAL MEDIANTE OPERACIONES BOOLEANAS
// -----
// Vaciar los conductos de alimentación de las galerías
voxAssembly.Subtract(voxAllFeedChannels);

// Fusionar los inyectores coaxiales al cuerpo
voxAssembly.Add(voxAllSwirlers);

return voxAssembly;
}

// =====
// MÉTODOS AUXILIARES DE PATRÓN GEOMÉTRICO Y COMPONENTES
// =====

/// <summary>
/// Calcula las coordenadas 2D de una distribución de 13 inyectores (1+6+6).
/// </summary>
private static List<Vector2> Get13ElementPattern(float fRadiusRing1, float fRadiusRing2)
{
    List<Vector2> list = new List<Vector2>();

    // 1. Inyector central
    list.Add(Vector2.Zero);

    // 2. Anillo Interior (6 inyectores separados 60°)
    for (int i = 0; i < 6; i++)
    {
        float angle = i * (float)Math.PI / 3.0f;
        list.Add(new Vector2(

```



```

        fRadiusRing1 * (float)Math.Cos(angle),
        fRadiusRing1 * (float)Math.Sin(angle)
    ));
}

// 3. Anillo Exterior (6 inyectores alternados a 60°)
float fOffsetAngle = (float)Math.PI / 6.0f; // 30° desfasado
for (int i = 0; i < 6; i++)
{
    float angle = i * (float)Math.PI / 3.0f + fOffsetAngle;
    list.Add(new Vector2(
        fRadiusRing2 * (float)Math.Cos(angle),
        fRadiusRing2 * (float)Math.Sin(angle)
    ));
}

return list;
}

private static Voxels voxCreateSwirlInjectorElement(
    float fCoreTubeInnerRadiusMm,
    float fCoreTubeOuterRadiusMm,
    float fSwirlerOuterRadiusMm,
    float fElementLengthMm,
    int nVaneCount,
    float fHelixPitchMm)
{
    // Tubo central GOX
    Mesh meshCoreOuter = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
    Mesh meshCoreInner = CreateCylinderMesh(fCoreTubeInnerRadiusMm, fElementLengthMm + 4.0f, -2.0f);
    Voxels voxCoreTube = new Voxels(meshCoreOuter);
    voxCoreTube.Subtract(new Voxels(meshCoreInner));

    // Cámara anular GCH4
    Mesh meshSleeveOuter = CreateCylinderMesh(fSwirlerOuterRadiusMm, fElementLengthMm);
    Mesh meshSleeveInner = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
    Voxels voxSwirlChamber = new Voxels(meshSleeveOuter);
    voxSwirlChamber.Subtract(new Voxels(meshSleeveInner));

    // Álabes helicoidales
    Voxels voxVanes = voxCreateHelicalVanes(
        fCoreTubeOuterRadiusMm, fSwirlerOuterRadiusMm, fElementLengthMm, nVaneCount, fHelixPitchMm);

    voxSwirlChamber.Intersect(voxVanes);

    Voxels voxElement = new Voxels();
    voxElement.Add(voxCoreTube);
    voxElement.Add(voxSwirlChamber);

    return voxElement;
}

private static Voxels voxCreateHelicalVanes(
    float fRInner, float fROuter, float fLength, int nVanes, float fPitchMm)
{
    Voxels voxVanesGroup = new Voxels();
    float fVaneThicknessMm = 0.8f;
    int nStepsZ = 20;
    float fDeltaZ = fLength / nStepsZ;

    for (int v = 0; v < nVanes; v++)
    {
        float fBaseAngle = v * (2.0f * (float)Math.PI / nVanes);
        Mesh meshVane = new Mesh();

        for (int i = 0; i <= nStepsZ; i++)
        {
            float fZ = i * fDeltaZ;
            float fAngle = fBaseAngle + (fZ / fPitchMm) * 2.0f * (float)Math.PI;

            Vector3 vRadDir = new Vector3((float)Math.Cos(fAngle), (float)Math.Sin(fAngle), 0);
            meshVane.nAddVertex(new Vector3(fRInner * vRadDir.X, fRInner * vRadDir.Y, fZ));
            meshVane.nAddVertex(new Vector3(fROuter * vRadDir.X, fROuter * vRadDir.Y, fZ));
        }

        for (int i = 0; i < nStepsZ; i++)
        {
            int i0 = i * 2, i1 = i0 + 1, i2 = (i + 1) * 2, i3 = i2 + 1;
            meshVane.AddTriangle(i0, i1, i3);
        }
    }
}

```

```

        meshVane.AddTriangle(i0, i3, i2);
    }

    Voxels voxSingleVane = new Voxels(meshVane);
    voxSingleVane.DoOffset(fVaneThicknessMm * 0.5f);
    voxVanesGroup.Add(voxSingleVane);
}

return voxVanesGroup;
}

private static Voxels voxCreateRevolvedGallery(
    float fMajorRadiusMm, float fZOffsetMm, float fWidthMm, float fHeightMm, int nCircleSteps = 48)
{
    Mesh meshGallery = new Mesh();
    float dTheta = 2.0f * (float)Math.PI / nCircleSteps;

    Vector2[] profile2D = new Vector2[]
    {
        new Vector2(0, fHeightMm * 0.5f),
        new Vector2(fWidthMm * 0.5f, 0),
        new Vector2(0, -fHeightMm * 0.5f),
        new Vector2(-fWidthMm * 0.5f, 0)
    };

    int nProfilePoints = profile2D.Length;

    for (int s = 0; s <= nCircleSteps; s++)
    {
        float theta = s * dTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        for (int p = 0; p < nProfilePoints; p++)
        {
            float r = fMajorRadiusMm + profile2D[p].X;
            float z = fZOffsetMm + profile2D[p].Y;

            meshGallery.nAddVertex(new Vector3(r * cosT, r * sinT, z));
        }
    }

    for (int s = 0; s < nCircleSteps; s++)
    {
        for (int p = 0; p < nProfilePoints; p++)
        {
            int pNext = (p + 1) % nProfilePoints;
            int idx00 = s * nProfilePoints + p;
            int idx10 = (s + 1) * nProfilePoints + p;
            int idx01 = s * nProfilePoints + pNext;
            int idx11 = (s + 1) * nProfilePoints + pNext;

            meshGallery.AddTriangle(idx00, idx10, idx11);
            meshGallery.AddTriangle(idx00, idx11, idx01);
        }
    }

    return new Voxels(meshGallery);
}

private static Voxels voxCreateBaseHousing(float fRadius, float fHeight)
{
    Mesh mesh = CreateCylinderMesh(fRadius, fHeight);
    return new Voxels(mesh);
}

private static Voxels voxCreateCylindricalDuct(Vector3 pStart, Vector3 pEnd, float fRadiusMm)
{
    Mesh mesh = new Mesh();
    Vector3 vDir = Vector3.Normalize(pEnd - pStart);
    float fLength = (pEnd - pStart).Length();

    Vector3 vUp = Math.Abs(vDir.Z) > 0.9f ? Vector3.UnitY : Vector3.UnitZ;
    Vector3 vTangent = Vector3.Normalize(Vector3.Cross(vDir, vUp));
    Vector3 vBinormal = Vector3.Normalize(Vector3.Cross(vTangent, vDir));

    int nSteps = 16;
    float dTheta = 2.0f * (float)Math.PI / nSteps;

```

```

        for (int s = 0; s < nSteps; s++)
        {
            float theta = s * dTheta;
            Vector3 vOffset = fRadiusMm * ((float)Math.Cos(theta) * vTangent + (float)Math.Sin(theta) * vBinorma

            int v0 = mesh.nAddVertex(pStart + vOffset);
            int v1 = mesh.nAddVertex(pEnd + vOffset);
        }

        Voxels voxDuct = new Voxels(mesh);
        voxDuct.DoOffset(fRadiusMm * 0.5f);
        return voxDuct;
    }

    private static Mesh CreateCylinderMesh(float fRadius, float fHeight, float fStartZ = 0.0f, int nSteps = 32)
    {
        Mesh mesh = new Mesh();
        float fDTheta = 2.0f * (float)Math.PI / nSteps;

        int vCenterBot = mesh.nAddVertex(new Vector3(0, 0, fStartZ));
        int vCenterTop = mesh.nAddVertex(new Vector3(0, 0, fStartZ + fHeight));

        int[] botIndices = new int[nSteps];
        int[] topIndices = new int[nSteps];

        for (int s = 0; s < nSteps; s++)
        {
            float theta = s * fDTheta;
            float cosT = (float)Math.Cos(theta);
            float sinT = (float)Math.Sin(theta);

            botIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ));
            topIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ + fHeight));
        }

        for (int s = 0; s < nSteps; s++)
        {
            int sNext = (s + 1) % nSteps;
            mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
            mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
            mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
            mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
        }

        return mesh;
    }
}

```

Detalles Destacados de la Arquitectura del Modelo

- 1. Topología Integrada de Fluido:** La función `Get13ElementPattern` calcula la ubicación radial exacta de cada inyector. Para cada elemento, la subrutina calcula vectores tridimensionales desde las galerías toroidales (`voxCreateCylindricalDuct`) que perforan el cuerpo principal para alimentar de forma balanceada el oxidante y el combustible a cada inyector.
- 2. Eficiencia Booleana Voxelizada:** Al trabajar directamente sobre vóxeles con **PikoGK**, la unión de 13 swirlers con sus rampas helicoidales y microconductos internos evita los errores geométrica y computacionalmente costosos de intersección de mallas tradicionales (B-Rep/STL).
- 3. Impresión 3D Monolítica (LPBF):** Todo el conjunto se exporta en una única pieza sólida sin necesidad de soldaduras entre inyectores y placa, maximizando la rigidez estructural ante altas presiones y pulsaciones acústicas de combustión.

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectores coaxiales.

Response: Este código en C# para **PikoGK** realiza el ensamble completo de la cabeza del inyector. Integra el cuerpo exterior, las **galerías internas de distribución de GCH₄ y GOX** y una **matriz radial de 13 inyectores coaxiales** (1 central + 6 en anillo interno + 6 en anillo externo) con sus respectivos canales de alimentación pasantes.

Código C#: Ensamble Completo del Inyector Multielemento

```

using System;
using System.Numerics;
using System.Collections.Generic;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssemblyGenerator

```

```

{
    /// <summary>
    /// Genera el ensamble monolítico completo del inyector de cohete,
    /// incluyendo cuerpo principal, galerías internas de distribución
    /// y 13 elementos inyectores coaxiales orientados radialmente.
    /// </summary>
    public static Voxels voxCreateFullInjectorAssembly(
        float fHousingOuterRadiusMm = 75.0f,
        float fHousingHeightMm = 85.0f,
        float fInnerRingRadiusMm = 28.0f,
        float fOuterRingRadiusMm = 52.0f)
    {
        // 1. Generar el cuerpo base del inyector con sus galerías de distribución
        Voxels voxHousing = InjectorManifoldsGenerator.voxCreateInjectorHousingWithGalleries(
            fHousingOuterRadiusMm: fHousingOuterRadiusMm,
            fHousingHeightMm: fHousingHeightMm,
            fGch4GalleryRadiusMm: fOuterRingRadiusMm,
            fGoxGalleryRadiusMm: fInnerRingRadiusMm
        );

        // 2. Definir las posiciones de la matriz de 13 inyectores (1 Centro, 6 Anillo Interno, 6 Anillo Externo)
        List<Vector3> listInjectorPositions = GetInjectorPatternPositions(
            fInnerRingRadiusMm,
            fOuterRingRadiusMm
        );

        Voxels voxInjectorsGroup = new Voxels();
        Voxels voxInjectorDrillPassages = new Voxels();

        // 3. Generar e integrar cada elemento inyector coaxial en sus coordenadas
        foreach (Vector3 vPos in listInjectorPositions)
        {
            // Elemento inyector coaxial con swirlers desplazado a la posición (vPos.X, vPos.Y)
            Voxels voxSingleInjector = voxCreatePositionedInjectorElement(
                vOffset: vPos,
                fCoreTubeInnerRadiusMm: 2.0f,
                fCoreTubeOuterRadiusMm: 3.2f,
                fSwirlerOuterRadiusMm: 6.5f,
                fElementLengthMm: fHousingHeightMm + 10.0f
            );
            voxInjectorsGroup.Add(voxSingleInjector);

            // Perforaciones de alimentacion vertical desde galerias a la camara
            Mesh meshPassage = CreateCylinderMeshAtPosition(
                vCenter: vPos,
                fRadius: 6.6f,
                fHeight: fHousingHeightMm + 14.0f,
                fZStart: -7.0f
            );
            voxInjectorDrillPassages.Add(new Voxels(meshPassage));
        }

        // 4. Operación Booleana: Taladrar los alojamientos de los inyectores en el cuerpo principal
        voxHousing.Subtract(voxInjectorDrillPassages);

        // 5. Operación Booleana: Fusionar la matriz de inyectores coaxiales en el bloque
        voxHousing.Add(voxInjectorsGroup);

        return voxHousing;
    }
}

/// <summary>
/// Calculador del patrón espacial de 13 inyectores (1 central, 6 en radio interno, 6 en radio externo).
/// </summary>
private static List<Vector3> GetInjectorPatternPositions(float fR1, float fR2)
{
    List<Vector3> positions = new List<Vector3>();

    // Inyector central (0, 0)
    positions.Add(new Vector3(0.0f, 0.0f, 0.0f));

    // Anillo interior: 6 inyectores separados por 60 grados
    for (int i = 0; i < 6; i++)
    {
        float fAngle = i * (float)(Math.PI / 3.0);
        positions.Add(new Vector3(
            fR1 * (float)Math.Cos(fAngle),
            fR1 * (float)Math.Sin(fAngle),
            0.0f
        ));
    }
}

```

```

    ));
}

// Anillo exterior: 6 inyectores desfasados 30 grados respecto al interior
for (int i = 0; i < 6; i++)
{
    float fAngle = (i * (float)(Math.PI / 3.0)) + (float)(Math.PI / 6.0);
    positions.Add(new Vector3(
        fR2 * (float)Math.Cos(fAngle),
        fR2 * (float)Math.Sin(fAngle),
        0.0f
    ));
}

return positions;
}

/// <summary>
/// Genera la geometría de un inyector coaxial individual desplazado espacialmente.
/// </summary>
private static Voxels voxCreatePositionedInjectorElement(
    Vector3 vOffset,
    float fCoreTubeInnerRadiusMm,
    float fCoreTubeOuterRadiusMm,
    float fSwirlerOuterRadiusMm,
    float fElementLengthMm)
{
    // Malla del tubo central desplazada
    Mesh meshCoreOuter = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -5.
    Mesh meshCoreInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeInnerRadiusMm, fElementLengthMm + 10

    Voxels voxCore = new Voxels(meshCoreOuter);
    voxCore.Subtract(new Voxels(meshCoreInner));

    // Malla de la manga anular externa desplazada
    Mesh meshSleeveOuter = CreateCylinderMeshAtPosition(vOffset, fSwirlerOuterRadiusMm, fElementLengthMm, -5
    Mesh meshSleeveInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -

    Voxels voxSwirlChannel = new Voxels(meshSleeveOuter);
    voxSwirlChannel.Subtract(new Voxels(meshSleeveInner));

    // Combinación final del elemento
    Voxels voxElement = new Voxels();
    voxElement.Add(voxCore);
    voxElement.Add(voxSwirlChannel);

    return voxElement;
}

private static Mesh CreateCylinderMeshAtPosition(
    Vector3 vCenter, float fRadius, float fHeight, float fZStart = 0.0f, int nSteps = 32)
{
    Mesh mesh = new Mesh();
    float fDTheta = 2.0f * (float)Math.PI / nSteps;

    int vCenterBot = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart));
    int vCenterTop = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart + fHeight));

    int[] botIndices = new int[nSteps];
    int[] topIndices = new int[nSteps];

    for (int s = 0; s < nSteps; s++)
    {
        float theta = s * fDTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        float x = vCenter.X + fRadius * cosT;
        float y = vCenter.Y + fRadius * sinT;

        botIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart));
        topIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart + fHeight));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sNext = (s + 1) % nSteps;
        mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
        mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
    }
}

```

```
        mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
        mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
    }

    return mesh;
}
}
```

Arquitectura Constructiva y Flujo Metodológico

- 1. **Patrón Multielemento Distribuido:** Mediante la función `GetInjectorPatternPositions`, el algoritmo establece 13 coordenadas cartesianas alineadas concéntricamente con los radios medios de las galerías toroide de distribución de oxidante (R_1) y combustible (R_2).
- 2. **Secuencia de Operaciones Booleanas de Voxelizado:**
 - **Paso A:** Generación del cuerpo cónico monolítico con cavidades de galerías en perfil de gota/diamante autosoportado para impresión 3D en aleación de níquel o Inconel.
 - **Paso B:** Extracción mediante `Subtract` de los 13 puertos verticales de alojamiento para interconectar los toroides de distribución con la zona de mezclado.
 - **Paso C:** Fusión mediante `Add` de los 13 inyectores coaxiales en una sola representación de espacio implícito de vóxeles libre de brechas geométricas (*watertight*).

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectores coaxiales.

Response: Este código en C# para **PikoGK** ensambla de forma completa la **cabeza del inyector**. Unifica el cuerpo exterior, las galerías internas de distribución (GOX y GCH₄) y una matriz de **13 inyectores coaxiales con álabes helicoidales** (1 central, 6 en el anillo interior y 6 en el anillo exterior), junto con los microconductos de alimentación que conectan cada galería con su respectivo inyector.

Código C#: Ensamble Completo del Inyector Multielemento

```
using System;
using System.Collections.Generic;
using System.Numerics;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssembly
    {
        /// <summary>
        /// Genera el ensamble monolítico completo del inyector:
        /// - Cuerpo estructural con cara de inyección
        /// - Galerías toroidales de distribución (GOX azul, GCH4 naranja)
        /// - Matriz de 13 elementos inyectores coaxiales con swirlers helicoidales
        /// - Canales internos de interconexión entre galerías e inyectores
        /// </summary>
        public static Voxels voxAssembleCompleteInjectorHead(
            float fHousingRadiusMm = 75.0f,
            float fHousingHeightMm = 90.0f,
            float fInnerRingRadiusMm = 24.0f,
            float fOuterRingRadiusMm = 48.0f)
        {
            // -----
            // 1. CUERPO BASE Y GALERÍAS DE DISTRIBUCIÓN
            // -----
            // Generar el bloque sólido exterior del housing
            Voxels voxAssembly = voxCreateBaseHousing(fHousingRadiusMm, fHousingHeightMm);

            // Generar la galería exterior de GCH4 (Metano gaseoso)
            float fGch4GalleryRadius = 50.0f;
            float fGch4GalleryZ = 55.0f;
            Voxels voxGch4Gallery = voxCreateRevolvedGallery(
                fMajorRadiusMm: fGch4GalleryRadius,
                fZOffsetMm: fGch4GalleryZ,
                fWidthMm: 16.0f,
                fHeightMm: 22.0f
            );

            // Generar la galería interior de GOX (Oxígeno gaseoso)
            float fGoxGalleryRadius = 28.0f;
            float fGoxGalleryZ = 35.0f;
            Voxels voxGoxGallery = voxCreateRevolvedGallery(
                fMajorRadiusMm: fGoxGalleryRadius,
                fZOffsetMm: fGoxGalleryZ,
```

```

        fWidthMm: 14.0f,
        fHeightMm: 18.0f
    );

    // Vaciado de las galerías dentro del cuerpo principal (Boolean Subtract)
    voxAssembly.Subtract(voxGch4Gallery);
    voxAssembly.Subtract(voxGoxGallery);

    // -----
    // 2. GENERACIÓN DE LAS POSICIONES DE LA MATRIZ DE 13 INYECTORES
    // -----
    List<Vector2> injectorPositions = Get13ElementPattern(fInnerRingRadiusMm, fOuterRingRadiusMm);

    Voxels voxAllSwirlers = new Voxels();
    Voxels voxAllFeedChannels = new Voxels();

    float fInjectorLength = 45.0f;
    float fInjectorStartZ = 10.0f; // Altura de la cara del inyector

    // -----
    // 3. CONSTRUCCIÓN E INTEGRACIÓN DE CADA INYECTOR COAXIAL
    // -----
    foreach (Vector2 pos in injectorPositions)
    {
        Vector3 vPos3D = new Vector3(pos.X, pos.Y, fInjectorStartZ);

        // A) Elemento inyector coaxial (Tubo GOX + Swirler GCH4)
        Voxels voxSingleInjector = voxCreateSwirlInjectorElement(
            fCoreTubeInnerRadiusMm: 2.0f,
            fCoreTubeOuterRadiusMm: 3.2f,
            fSwirlerOuterRadiusMm: 6.5f,
            fElementLengthMm: fInjectorLength,
            nVaneCount: 6,
            fHelixPitchMm: 20.0f
        );

        // Posicionar inyector en la matriz
        voxSingleInjector.Translate(vPos3D);
        voxAllSwirlers.Add(voxSingleInjector);

        // B) Conductos de alimentación desde Galería GCH4 hacia la cámara del Swirler
        float fDistToCenter = pos.Length();
        if (fDistToCenter > 0.1f)
        {
            Vector2 vDirToCenter = Vector2.Normalize(-pos);

            // Tap directo desde la galería de GCH4
            Vector3 pGch4Gallery = new Vector3(
                pos.X - vDirToCenter.X * Math.Abs(fGch4GalleryRadius - fDistToCenter),
                pos.Y - vDirToCenter.Y * Math.Abs(fGch4GalleryRadius - fDistToCenter),
                fGch4GalleryZ
            );

            Vector3 pInjectorGch4Port = new Vector3(pos.X, pos.Y, fInjectorStartZ + fInjectorLength - 8.0f);
            Voxels voxGch4Feeder = voxCreateCylindricalDuct(pGch4Gallery, pInjectorGch4Port, fRadiusMm: 2.2f);
            voxAllFeedChannels.Add(voxGch4Feeder);

            // Tap directo desde la galería de GOX hacia el tubo central
            Vector3 pGoxGallery = new Vector3(
                pos.X - vDirToCenter.X * Math.Abs(fGoxGalleryRadius - fDistToCenter),
                pos.Y - vDirToCenter.Y * Math.Abs(fGoxGalleryRadius - fDistToCenter),
                fGoxGalleryZ
            );

            Vector3 pInjectorGoxPort = new Vector3(pos.X, pos.Y, fInjectorStartZ + fInjectorLength - 22.0f);
            Voxels voxGoxFeeder = voxCreateCylindricalDuct(pGoxGallery, pInjectorGoxPort, fRadiusMm: 1.8f);
            voxAllFeedChannels.Add(voxGoxFeeder);
        }
    }

    // -----
    // 4. ENSAMBLE FINAL MEDIANTE OPERACIONES BOOLEANAS
    // -----
    // Vaciar los conductos de alimentación de las galerías
    voxAssembly.Subtract(voxAllFeedChannels);

    // Fusionar los inyectores coaxiales al cuerpo
    voxAssembly.Add(voxAllSwirlers);

```

```

        return voxAssembly;
    }

    // =====
    // MÉTODOS AUXILIARES DE PATRÓN GEOMÉTRICO Y COMPONENTES
    // =====

    /// <summary>
    /// Calcula las coordenadas 2D de una distribución de 13 inyectores (1+6+6).
    /// </summary>
    private static List<Vector2> Get13ElementPattern(float fRadiusRing1, float fRadiusRing2)
    {
        List<Vector2> list = new List<Vector2>();

        // 1. Inyector central
        list.Add(Vector2.Zero);

        // 2. Anillo Interior (6 inyectores separados 60°)
        for (int i = 0; i < 6; i++)
        {
            float angle = i * (float)Math.PI / 3.0f;
            list.Add(new Vector2(
                fRadiusRing1 * (float)Math.Cos(angle),
                fRadiusRing1 * (float)Math.Sin(angle)
            ));
        }

        // 3. Anillo Exterior (6 inyectores alternados a 60°)
        float fOffsetAngle = (float)Math.PI / 6.0f; // 30° desfasado
        for (int i = 0; i < 6; i++)
        {
            float angle = i * (float)Math.PI / 3.0f + fOffsetAngle;
            list.Add(new Vector2(
                fRadiusRing2 * (float)Math.Cos(angle),
                fRadiusRing2 * (float)Math.Sin(angle)
            ));
        }

        return list;
    }

    private static Voxels voxCreateSwirlInjectorElement(
        float fCoreTubeInnerRadiusMm,
        float fCoreTubeOuterRadiusMm,
        float fSwirlerOuterRadiusMm,
        float fElementLengthMm,
        int nVaneCount,
        float fHelixPitchMm)
    {
        // Tubo central GOX
        Mesh meshCoreOuter = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
        Mesh meshCoreInner = CreateCylinderMesh(fCoreTubeInnerRadiusMm, fElementLengthMm + 4.0f, -2.0f);
        Voxels voxCoreTube = new Voxels(meshCoreOuter);
        voxCoreTube.Subtract(new Voxels(meshCoreInner));

        // Cámara anular GCH4
        Mesh meshSleeveOuter = CreateCylinderMesh(fSwirlerOuterRadiusMm, fElementLengthMm);
        Mesh meshSleeveInner = CreateCylinderMesh(fCoreTubeOuterRadiusMm, fElementLengthMm);
        Voxels voxSwirlChamber = new Voxels(meshSleeveOuter);
        voxSwirlChamber.Subtract(new Voxels(meshSleeveInner));

        // Álabes helicoidales
        Voxels voxVanes = voxCreateHelicalVanes(
            fCoreTubeOuterRadiusMm, fSwirlerOuterRadiusMm, fElementLengthMm, nVaneCount, fHelixPitchMm);

        voxSwirlChamber.Intersect(voxVanes);

        Voxels voxElement = new Voxels();
        voxElement.Add(voxCoreTube);
        voxElement.Add(voxSwirlChamber);

        return voxElement;
    }

    private static Voxels voxCreateHelicalVanes(
        float fRInner, float fROuter, float fLength, int nVanes, float fPitchMm)
    {
        Voxels voxVanesGroup = new Voxels();
        float fVaneThicknessMm = 0.8f;

```



```

int nStepsZ = 20;
float fDeltaZ = fLength / nStepsZ;

for (int v = 0; v < nVanes; v++)
{
    float fBaseAngle = v * (2.0f * (float)Math.PI / nVanes);
    Mesh meshVane = new Mesh();

    for (int i = 0; i <= nStepsZ; i++)
    {
        float fZ = i * fDeltaZ;
        float fAngle = fBaseAngle + (fZ / fPitchMm) * 2.0f * (float)Math.PI;

        Vector3 vRadDir = new Vector3((float)Math.Cos(fAngle), (float)Math.Sin(fAngle), 0);
        meshVane.nAddVertex(new Vector3(fRInner * vRadDir.X, fRInner * vRadDir.Y, fZ));
        meshVane.nAddVertex(new Vector3(fROuter * vRadDir.X, fROuter * vRadDir.Y, fZ));
    }

    for (int i = 0; i < nStepsZ; i++)
    {
        int i0 = i * 2, i1 = i0 + 1, i2 = (i + 1) * 2, i3 = i2 + 1;
        meshVane.AddTriangle(i0, i1, i3);
        meshVane.AddTriangle(i0, i3, i2);
    }

    Voxels voxSingleVane = new Voxels(meshVane);
    voxSingleVane.DoOffset(fVaneThicknessMm * 0.5f);
    voxVanesGroup.Add(voxSingleVane);
}

return voxVanesGroup;
}

private static Voxels voxCreateRevolvedGallery(
    float fMajorRadiusMm, float fZOffsetMm, float fWidthMm, float fHeightMm, int nCircleSteps = 48)
{
    Mesh meshGallery = new Mesh();
    float dTheta = 2.0f * (float)Math.PI / nCircleSteps;

    Vector2[] profile2D = new Vector2[]
    {
        new Vector2(0, fHeightMm * 0.5f),
        new Vector2(fWidthMm * 0.5f, 0),
        new Vector2(0, -fHeightMm * 0.5f),
        new Vector2(-fWidthMm * 0.5f, 0)
    };

    int nProfilePoints = profile2D.Length;

    for (int s = 0; s <= nCircleSteps; s++)
    {
        float theta = s * dTheta;
        float cosT = (float)Math.Cos(theta);
        float sinT = (float)Math.Sin(theta);

        for (int p = 0; p < nProfilePoints; p++)
        {
            float r = fMajorRadiusMm + profile2D[p].X;
            float z = fZOffsetMm + profile2D[p].Y;

            meshGallery.nAddVertex(new Vector3(r * cosT, r * sinT, z));
        }
    }

    for (int s = 0; s < nCircleSteps; s++)
    {
        for (int p = 0; p < nProfilePoints; p++)
        {
            int pNext = (p + 1) % nProfilePoints;
            int idx00 = s * nProfilePoints + p;
            int idx10 = (s + 1) * nProfilePoints + p;
            int idx01 = s * nProfilePoints + pNext;
            int idx11 = (s + 1) * nProfilePoints + pNext;

            meshGallery.AddTriangle(idx00, idx10, idx11);
            meshGallery.AddTriangle(idx00, idx11, idx01);
        }
    }
}

```

```

        return new Voxels(meshGallery);
    }

    private static Voxels voxCreateBaseHousing(float fRadius, float fHeight)
    {
        Mesh mesh = CreateCylinderMesh(fRadius, fHeight);
        return new Voxels(mesh);
    }

    private static Voxels voxCreateCylindricalDuct(Vector3 pStart, Vector3 pEnd, float fRadiusMm)
    {
        Mesh mesh = new Mesh();
        Vector3 vDir = Vector3.Normalize(pEnd - pStart);
        float fLength = (pEnd - pStart).Length();

        Vector3 vUp = Math.Abs(vDir.Z) > 0.9f ? Vector3.UnitY : Vector3.UnitZ;
        Vector3 vTangent = Vector3.Normalize(Vector3.Cross(vDir, vUp));
        Vector3 vBinormal = Vector3.Normalize(Vector3.Cross(vTangent, vDir));

        int nSteps = 16;
        float dTheta = 2.0f * (float)Math.PI / nSteps;

        for (int s = 0; s < nSteps; s++)
        {
            float theta = s * dTheta;
            Vector3 vOffset = fRadiusMm * ((float)Math.Cos(theta) * vTangent + (float)Math.Sin(theta) * vBinormal);

            int v0 = mesh.nAddVertex(pStart + vOffset);
            int v1 = mesh.nAddVertex(pEnd + vOffset);
        }

        Voxels voxDuct = new Voxels(mesh);
        voxDuct.DoOffset(fRadiusMm * 0.5f);
        return voxDuct;
    }

    private static Mesh CreateCylinderMesh(float fRadius, float fHeight, float fStartZ = 0.0f, int nSteps = 32)
    {
        Mesh mesh = new Mesh();
        float fDTheta = 2.0f * (float)Math.PI / nSteps;

        int vCenterBot = mesh.nAddVertex(new Vector3(0, 0, fStartZ));
        int vCenterTop = mesh.nAddVertex(new Vector3(0, 0, fStartZ + fHeight));

        int[] botIndices = new int[nSteps];
        int[] topIndices = new int[nSteps];

        for (int s = 0; s < nSteps; s++)
        {
            float theta = s * fDTheta;
            float cosT = (float)Math.Cos(theta);
            float sinT = (float)Math.Sin(theta);

            botIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ));
            topIndices[s] = mesh.nAddVertex(new Vector3(fRadius * cosT, fRadius * sinT, fStartZ + fHeight));
        }

        for (int s = 0; s < nSteps; s++)
        {
            int sNext = (s + 1) % nSteps;
            mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
            mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
            mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
            mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
        }

        return mesh;
    }
}

```

Detalles Destacados de la Arquitectura del Modelo

- Topología Integrada de Fluido:** La función `Get13ElementPattern` calcula la ubicación radial exacta de cada inyector. Para cada elemento, la subrutina calcula vectores tridimensionales desde las galerías toroidales (`voxCreateCylindricalDuct`) que perforan el cuerpo principal para alimentar de forma balanceada el oxidante y el combustible a cada inyector.
- Eficiencia Booleana Voxelizada:** Al trabajar directamente sobre vóxeles con **PikoGK**, la unión de 13 swirlers con sus rampas helicoidales y microconductos internos evita los errores geométrica y computacionalmente costosos de intersección de mallas

tradicionales (B-Rep/STL).

3. **Impresión 3D Monolítica (LPBF):** Todo el conjunto se exporta en una única pieza sólida sin necesidad de soldaduras entre inyectores y placa, maximizando la rigidez estructural ante altas presiones y pulsaciones acústicas de combustión.

User prompt: Muestra el código completo para ensamblar la estructura con galerías de distribución y la matriz de inyectores coaxiales.

Response: Este código en C# para **PikoGK** realiza el ensamble completo de la cabeza del inyector. Integra el cuerpo exterior, las **galerías internas de distribución de GCH₄ y GOX** y una **matriz radial de 13 inyectores coaxiales** (1 central + 6 en anillo interno + 6 en anillo externo) con sus respectivos canales de alimentación pasantes.

Código C#: Ensamble Completo del Inyector Multielemento

```
using System;
using System.Numerics;
using System.Collections.Generic;
using PikoGK;

namespace RocketEngineDesign
{
    public static class FullInjectorAssemblyGenerator
    {
        /// <summary>
        /// Genera el ensamble monolítico completo del inyector de cohete,
        /// incluyendo cuerpo principal, galerías internas de distribución
        /// y 13 elementos inyectores coaxiales orientados radialmente.
        /// </summary>
        public static Voxels voxCreateFullInjectorAssembly(
            float fHousingOuterRadiusMm = 75.0f,
            float fHousingHeightMm = 85.0f,
            float fInnerRingRadiusMm = 28.0f,
            float fOuterRingRadiusMm = 52.0f)
        {
            // 1. Generar el cuerpo base del inyector con sus galerías de distribución
            Voxels voxHousing = InjectorManifoldsGenerator.voxCreateInjectorHousingWithGalleries(
                fHousingOuterRadiusMm: fHousingOuterRadiusMm,
                fHousingHeightMm: fHousingHeightMm,
                fGch4GalleryRadiusMm: fOuterRingRadiusMm,
                fGoxGalleryRadiusMm: fInnerRingRadiusMm
            );

            // 2. Definir las posiciones de la matriz de 13 inyectores (1 Centro, 6 Anillo Interno, 6 Anillo Externo)
            List<Vector3> listInjectorPositions = GetInjectorPatternPositions(
                fInnerRingRadiusMm,
                fOuterRingRadiusMm
            );

            Voxels voxInjectorsGroup = new Voxels();
            Voxels voxInjectorDrillPassages = new Voxels();

            // 3. Generar e integrar cada elemento inyector coaxial en sus coordenadas
            foreach (Vector3 vPos in listInjectorPositions)
            {
                // Elemento inyector coaxial con swirlers desplazado a la posición (vPos.X, vPos.Y)
                Voxels voxSingleInjector = voxCreatePositionedInjectorElement(
                    vOffset: vPos,
                    fCoreTubeInnerRadiusMm: 2.0f,
                    fCoreTubeOuterRadiusMm: 3.2f,
                    fSwirlerOuterRadiusMm: 6.5f,
                    fElementLengthMm: fHousingHeightMm + 10.0f
                );
                voxInjectorsGroup.Add(voxSingleInjector);

                // Perforaciones de alimentacion vertical desde galerias a la camara
                Mesh meshPassage = CreateCylinderMeshAtPosition(
                    vCenter: vPos,
                    fRadius: 6.6f,
                    fHeight: fHousingHeightMm + 14.0f,
                    fZStart: -7.0f
                );
                voxInjectorDrillPassages.Add(new Voxels(meshPassage));
            }

            // 4. Operación Booleana: Taladrar los alojamientos de los inyectores en el cuerpo principal
            voxHousing.Subtract(voxInjectorDrillPassages);

            // 5. Operación Booleana: Fusionar la matriz de inyectores coaxiales en el bloque
        }
    }
}
```

```

        voxHousing.Add(voxInjectorsGroup);

        return voxHousing;
    }

    /// <summary>
    /// Calculador del patrón espacial de 13 inyectores (1 central, 6 en radio interno, 6 en radio externo).
    /// </summary>
    private static List<Vector3> GetInjectorPatternPositions(float fR1, float fR2)
    {
        List<Vector3> positions = new List<Vector3>();

        // Inyector central (0, 0)
        positions.Add(new Vector3(0.0f, 0.0f, 0.0f));

        // Anillo interior: 6 inyectores separados por 60 grados
        for (int i = 0; i < 6; i++)
        {
            float fAngle = i * (float)(Math.PI / 3.0);
            positions.Add(new Vector3(
                fR1 * (float)Math.Cos(fAngle),
                fR1 * (float)Math.Sin(fAngle),
                0.0f
            ));
        }

        // Anillo exterior: 6 inyectores desfasados 30 grados respecto al interior
        for (int i = 0; i < 6; i++)
        {
            float fAngle = (i * (float)(Math.PI / 3.0)) + (float)(Math.PI / 6.0);
            positions.Add(new Vector3(
                fR2 * (float)Math.Cos(fAngle),
                fR2 * (float)Math.Sin(fAngle),
                0.0f
            ));
        }

        return positions;
    }

    /// <summary>
    /// Genera la geometría de un inyector coaxial individual desplazado espacialmente.
    /// </summary>
    private static Voxels voxCreatePositionedInjectorElement(
        Vector3 vOffset,
        float fCoreTubeInnerRadiusMm,
        float fCoreTubeOuterRadiusMm,
        float fSwirlerOuterRadiusMm,
        float fElementLengthMm)
    {
        // Malla del tubo central desplazada
        Mesh meshCoreOuter = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -5.
        Mesh meshCoreInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeInnerRadiusMm, fElementLengthMm + 10

        Voxels voxCore = new Voxels(meshCoreOuter);
        voxCore.Subtract(new Voxels(meshCoreInner));

        // Malla de la manga anular externa desplazada
        Mesh meshSleeveOuter = CreateCylinderMeshAtPosition(vOffset, fSwirlerOuterRadiusMm, fElementLengthMm, -5
        Mesh meshSleeveInner = CreateCylinderMeshAtPosition(vOffset, fCoreTubeOuterRadiusMm, fElementLengthMm, -

        Voxels voxSwirlChannel = new Voxels(meshSleeveOuter);
        voxSwirlChannel.Subtract(new Voxels(meshSleeveInner));

        // Combinación final del elemento
        Voxels voxElement = new Voxels();
        voxElement.Add(voxCore);
        voxElement.Add(voxSwirlChannel);

        return voxElement;
    }

    private static Mesh CreateCylinderMeshAtPosition(
        Vector3 vCenter, float fRadius, float fHeight, float fZStart = 0.0f, int nSteps = 32)
    {
        Mesh mesh = new Mesh();
        float fDTheta = 2.0f * (float)Math.PI / nSteps;

        int vCenterBot = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart));
    }

```

```

int vCenterTop = mesh.nAddVertex(new Vector3(vCenter.X, vCenter.Y, fZStart + fHeight));

int[] botIndices = new int[nSteps];
int[] topIndices = new int[nSteps];

for (int s = 0; s < nSteps; s++)
{
    float theta = s * fDTheta;
    float cosT = (float)Math.Cos(theta);
    float sinT = (float)Math.Sin(theta);

    float x = vCenter.X + fRadius * cosT;
    float y = vCenter.Y + fRadius * sinT;

    botIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart));
    topIndices[s] = mesh.nAddVertex(new Vector3(x, y, fZStart + fHeight));
}

for (int s = 0; s < nSteps; s++)
{
    int sNext = (s + 1) % nSteps;
    mesh.AddTriangle(botIndices[s], topIndices[s], topIndices[sNext]);
    mesh.AddTriangle(botIndices[s], topIndices[sNext], botIndices[sNext]);
    mesh.AddTriangle(vCenterBot, botIndices[sNext], botIndices[s]);
    mesh.AddTriangle(vCenterTop, topIndices[s], topIndices[sNext]);
}

return mesh;
}
}
}

```

Arquitectura Constructiva y Flujo Metodológico

- Patrón Multielemento Distribuido:** Mediante la función `GetInjectorPatternPositions`, el algoritmo establece 13 coordenadas cartesianas alineadas concéntricamente con los radios medios de las galerías toroide de distribución de oxidante (R_1) y combustible (R_2).
- Secuencia de Operaciones Booleanas de Voxelizado:**
 - Paso A:** Generación del cuerpo cónico monolítico con cavidades de galerías en perfil de gota/diamante autosoportado para impresión 3D en aleación de níquel o Inconel.
 - Paso B:** Extracción mediante `Subtract` de los 13 puertos verticales de alojamiento para interconectar los toroides de distribución con la zona de mezclado.
 - Paso C:** Fusión mediante `Add` de los 13 inyectores coaxiales en una sola representación de espacio implícito de vóxeles libre de brechas geométricas (*watertight*).

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano y redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",

```

```

float fBlurRadiusMm = 0.4f,
float fFilletRadiusMm = 0.8f)
{
    Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

    // 1. Crear una copia de los vóxeles para no alterar la fuente original
    Voxels voxSmoothed = new Voxels(voxInput);

    // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
    // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill)
    if (fFilletRadiusMm > 0.0f)
    {
        voxSmoothed.DoOffset(fFilletRadiusMm);
        voxSmoothed.DoOffset(-fFilletRadiusMm);

        // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
        voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
        voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
    }

    // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
    if (fBlurRadiusMm > 0.0f)
    {
        voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
    }

    Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

    // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
    Mesh meshFinal = new Mesh(voxSmoothed);

    // Asegurar que el directorio de salida existe
    if (!Directory.Exists(strOutputFolder))
    {
        Directory.CreateDirectory(strOutputFolder);
    }

    string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
    string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

    // 5. Exportación a archivos STL y 3MF
    Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
    Library.ExportMeshToStl(meshFinal, strStlFilePath);

    Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
    Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

    Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: "./ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por OverOffset:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (DoGaussianBlur):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PicoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```
using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
            // voxelsFinal.Offset(-0.2f); // Erosión
            // voxelsFinal.Offset(0.2f); // Dilatación

            // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
            Mesh meshExport = new Mesh(voxelsFinal);

            // 4. Exportar la malla a archivos STL y 3MF
            string stlFilePath = "ModeloFinal_Suavizado.stl";
            string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

            meshExport.SaveToStl(stlFilePath);
            meshExport.SaveTo3Mf(threeMfFilePath);

            Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
            Console.WriteLine($" - STL: {stlFilePath}");
            Console.WriteLine($" - 3MF: {threeMfFilePath}");
        }
    }
}
```

```

/// <summary>
/// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
/// </summary>
private static Voxels CreateSampleGeometry()
{
    Lattice lattice = new Lattice();

    // Añadir vigas interconectadas de ejemplo
    lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

    // Voxelizar la estructura lattice
    return new Voxels(lattice);
}
}
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill
            if (fFilletRadiusMm > 0.0f)
            {
                voxSmoothed.DoOffset(fFilletRadiusMm);
                voxSmoothed.DoOffset(-fFilletRadiusMm);

                // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
                voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
                voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
            }
        }
    }
}

```



```

// 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
if (fBlurRadiusMm > 0.0f)
{
    voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
}

Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

// 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
Mesh meshFinal = new Mesh(voxSmoothed);

// Asegurar que el directorio de salida existe
if (!Directory.Exists(strOutputFolder))
{
    Directory.CreateDirectory(strOutputFolder);
}

string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

// 5. Exportación a archivos STL y 3MF
Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
Library.ExportMeshToStl(meshFinal, strStlFilePath);

Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: "./ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (`DoGaussianBlur`):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```
using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
            // voxelsFinal.Offset(-0.2f); // Erosión
            // voxelsFinal.Offset(0.2f); // Dilatación

            // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
            Mesh meshExport = new Mesh(voxelsFinal);

            // 4. Exportar la malla a archivos STL y 3MF
            string stlFilePath = "ModeloFinal_Suavizado.stl";
            string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

            meshExport.SaveToStl(stlFilePath);
            meshExport.SaveTo3Mf(threeMfFilePath);

            Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
            Console.WriteLine($" - STL: {stlFilePath}");
            Console.WriteLine($" - 3MF: {threeMfFilePath}");
        }

        /// <summary>
        /// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
        /// </summary>
        private static Voxels CreateSampleGeometry()
        {
            Lattice lattice = new Lattice();

            // Añadir vigas interconectadas de ejemplo
            lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
            lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
            lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

            // Voxelizar la estructura lattice
            return new Voxels(lattice);
        }
    }
}
```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PikoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```
using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill)
            if (fFilletRadiusMm > 0.0f)
            {
                voxSmoothed.DoOffset(fFilletRadiusMm);
                voxSmoothed.DoOffset(-fFilletRadiusMm);

                // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
                voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
                voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
            }

            // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
            if (fBlurRadiusMm > 0.0f)
            {
                voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
            }

            Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

            // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
            Mesh meshFinal = new Mesh(voxSmoothed);

            // Asegurar que el directorio de salida existe
            if (!Directory.Exists(strOutputFolder))
            {
                Directory.CreateDirectory(strOutputFolder);
            }

            string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
```

```

string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

// 5. Exportación a archivos STL y 3MF
Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
Library.ExportMeshToStl(meshFinal, strStlFilePath);

Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: "./ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (*DoGaussianBlur*):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```

using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {

```

```

// Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
// e indicando la tarea a ejecutar.
Library.Go(0.5f, RunTask);
}

public static void RunTask()
{
    // 1. Obtener o construir el campo de voxels volumétrico final
    Voxels voxelsFinal = CreateSampleGeometry();

    // 2. Aplicar filtros de suavizado en el volumen (Voxels)
    // -----

    // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
    // Útil para eliminar el escalonamiento de voxels y redondear bordes.
    float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
    voxelsFinal.Gaussian(fGaussianRadius);

    // B) Filtro Mediana / Media (Opcional)
    // voxelsFinal.Median();
    // voxelsFinal.Mean();

    // C) Suavizado morfológico mediante Offsets (Opcional)
    // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
    // ayuda a eliminar ruido y pequeñas imperfecciones.
    // voxelsFinal.Offset(-0.2f); // Erosión
    // voxelsFinal.Offset(0.2f); // Dilatación

    // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
    Mesh meshExport = new Mesh(voxelsFinal);

    // 4. Exportar la malla a archivos STL y 3MF
    string stlFilePath = "ModeloFinal_Suavizado.stl";
    string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

    meshExport.SaveToStl(stlFilePath);
    meshExport.SaveTo3Mf(threeMfFilePath);

    Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
    Console.WriteLine($" - STL: {stlFilePath}");
    Console.WriteLine($" - 3MF: {threeMfFilePath}");
}

/// <summary>
/// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
/// </summary>
private static Voxels CreateSampleGeometry()
{
    Lattice lattice = new Lattice();

    // Añadir vigas interconectadas de ejemplo
    lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

    // Voxelizar la estructura lattice
    return new Voxels(lattice);
}
}
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de voxels en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill
            if (fFilletRadiusMm > 0.0f)
            {
                voxSmoothed.DoOffset(fFilletRadiusMm);
                voxSmoothed.DoOffset(-fFilletRadiusMm);

                // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
                voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
                voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
            }

            // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
            if (fBlurRadiusMm > 0.0f)
            {
                voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
            }

            Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

            // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
            Mesh meshFinal = new Mesh(voxSmoothed);

            // Asegurar que el directorio de salida existe
            if (!Directory.Exists(strOutputFolder))
            {
                Directory.CreateDirectory(strOutputFolder);
            }

            string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
            string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

            // 5. Exportación a archivos STL y 3MF
            Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
            Library.ExportMeshToStl(meshFinal, strStlFilePath);

            Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
            Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

            Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
        }

        /// <summary>
        /// Punto de entrada del pipeline de generación y exportación.
        /// </summary>
        public static void RunExportPipeline()
        {
            // Inicializar PikoGK con resolución de vóxel de 0.25 mm

```

```

float fVoxelSizeMm = 0.25f;

Library.Go(fVoxelSizeMm, () =>
{
    // Generar la geometría completa ensamblada
    Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

    // Procesar filtrado y exportar
    ProcessAndExportInjector(
        voxInput: voxAssembly,
        strOutputFolder: "./ExportedModels",
        strBaseFileName: "RaptorStyle_HotGas_Injector",
        fBlurRadiusMm: 0.35f,
        fFilletRadiusMm: 0.75f
    );
});
}
}
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (`DoGaussianBlur`):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PicoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxeles, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```

using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxeles volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxeles y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxeles
            voxelsFinal.Gaussian(fGaussianRadius);
        }
    }
}

```

```

// B) Filtro Mediana / Media (Opcional)
// voxelsFinal.Median();
// voxelsFinal.Mean();

// C) Suavizado morfológico mediante Offsets (Opcional)
// Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
// ayuda a eliminar ruido y pequeñas imperfecciones.
// voxelsFinal.Offset(-0.2f); // Erosión
// voxelsFinal.Offset(0.2f); // Dilatación

// 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
Mesh meshExport = new Mesh(voxelsFinal);

// 4. Exportar la malla a archivos STL y 3MF
string stlFilePath = "ModeloFinal_Suavizado.stl";
string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

meshExport.SaveToStl(stlFilePath);
meshExport.SaveTo3mf(threeMfFilePath);

Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
Console.WriteLine($" - STL: {stlFilePath}");
Console.WriteLine($" - 3MF: {threeMfFilePath}");
}

/// <summary>
/// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
/// </summary>
private static Voxels CreateSampleGeometry()
{
    Lattice lattice = new Lattice();

    // Añadir vigas interconectadas de ejemplo
    lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

    // Voxelizar la estructura lattice
    return new Voxels(lattice);
}
}
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano y redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
    }
}

```



```

/// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
public static void ProcessAndExportInjector(
    Voxels voxInput,
    string strOutputFolder = "./Output",
    string strBaseFileName = "HotGas_Injector_Assembly",
    float fBlurRadiusMm = 0.4f,
    float fFilletRadiusMm = 0.8f)
{
    Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

    // 1. Crear una copia de los vóxeles para no alterar la fuente original
    Voxels voxSmoothed = new Voxels(voxInput);

    // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
    // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill)
    if (fFilletRadiusMm > 0.0f)
    {
        voxSmoothed.DoOffset(fFilletRadiusMm);
        voxSmoothed.DoOffset(-fFilletRadiusMm);

        // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
        voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
        voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
    }

    // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
    if (fBlurRadiusMm > 0.0f)
    {
        voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
    }

    Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

    // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
    Mesh meshFinal = new Mesh(voxSmoothed);

    // Asegurar que el directorio de salida existe
    if (!Directory.Exists(strOutputFolder))
    {
        Directory.CreateDirectory(strOutputFolder);
    }

    string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
    string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

    // 5. Exportación a archivos STL y 3MF
    Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
    Library.ExportMeshToStl(meshFinal, strStlFilePath);

    Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
    Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

    Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: "./ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}

```

```
}
}
```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (*DoGaussianBlur*):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PicoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```
using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
            // voxelsFinal.Offset(-0.2f); // Erosión
            // voxelsFinal.Offset(0.2f); // Dilatación

            // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
            Mesh meshExport = new Mesh(voxelsFinal);

            // 4. Exportar la malla a archivos STL y 3MF
            string stlFilePath = "ModeloFinal_Suavizado.stl";
            string threeMfFilePath = "ModeloFinal_Suavizado.3mf";
```

```

meshExport.SaveToStl(stlFilePath);
meshExport.SaveTo3Mf(threeMfFilePath);

Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
Console.WriteLine($" - STL: {stlFilePath}");
Console.WriteLine($" - 3MF: {threeMfFilePath}");
}

/// <summary>
/// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
/// </summary>
private static Voxels CreateSampleGeometry()
{
    Lattice lattice = new Lattice();

    // Añadir vigas interconectadas de ejemplo
    lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

    // Voxelizar la estructura lattice
    return new Voxels(lattice);
}
}
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill
            if (fFilletRadiusMm > 0.0f)
            {

```

```

        voxSmoothed.DoOffset(fFilletRadiusMm);
        voxSmoothed.DoOffset(-fFilletRadiusMm);

        // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
        voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
        voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
    }

    // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
    if (fBlurRadiusMm > 0.0f)
    {
        voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
    }

    Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

    // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
    Mesh meshFinal = new Mesh(voxSmoothed);

    // Asegurar que el directorio de salida existe
    if (!Directory.Exists(strOutputFolder))
    {
        Directory.CreateDirectory(strOutputFolder);
    }

    string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
    string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

    // 5. Exportación a archivos STL y 3MF
    Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
    Library.ExportMeshToStl(meshFinal, strStlFilePath);

    Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
    Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

    Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: ".\\ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (`DoGaussianBlur`):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.

- **3MF**: Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# **PicoGK**), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase **Voxels**.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```
using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
            // voxelsFinal.Offset(-0.2f); // Erosión
            // voxelsFinal.Offset(0.2f); // Dilatación

            // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
            Mesh meshExport = new Mesh(voxelsFinal);

            // 4. Exportar la malla a archivos STL y 3MF
            string stlFilePath = "ModeloFinal_Suavizado.stl";
            string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

            meshExport.SaveToStl(stlFilePath);
            meshExport.SaveTo3Mf(threeMfFilePath);

            Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
            Console.WriteLine($" - STL: {stlFilePath}");
            Console.WriteLine($" - 3MF: {threeMfFilePath}");
        }
    }

    /// <summary>
    /// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
    /// </summary>
    private static Voxels CreateSampleGeometry()
    {
        Lattice lattice = new Lattice();

        // Añadir vigas interconectadas de ejemplo
        lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
        lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
        lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 30, 0), 4.0f, 4.0f);
    }
}
```

```

        // Voxelizar la estructura lattice
        return new Voxels(lattice);
    }
}
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill
            if (fFilletRadiusMm > 0.0f)
            {
                voxSmoothed.DoOffset(fFilletRadiusMm);
                voxSmoothed.DoOffset(-fFilletRadiusMm);

                // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
                voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
                voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
            }

            // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
            if (fBlurRadiusMm > 0.0f)
            {
                voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
            }

            Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

            // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
            Mesh meshFinal = new Mesh(voxSmoothed);

```

```

// Asegurar que el directorio de salida existe
if (!Directory.Exists(strOutputFolder))
{
    Directory.CreateDirectory(strOutputFolder);
}

string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

// 5. Exportación a archivos STL y 3MF
Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
Library.ExportMeshToStl(meshFinal, strStlFilePath);

Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: ".\\ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (*DoGaussianBlur*):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o *SDFs*) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (*Mesh*) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```

using System;
using System.Numerics;
using PicoGK;

```

```

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
            // voxelsFinal.Offset(-0.2f); // Erosión
            // voxelsFinal.Offset(0.2f); // Dilatación

            // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
            Mesh meshExport = new Mesh(voxelsFinal);

            // 4. Exportar la malla a archivos STL y 3MF
            string stlFilePath = "ModeloFinal_Suavizado.stl";
            string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

            meshExport.SaveToStl(stlFilePath);
            meshExport.SaveTo3Mf(threeMfFilePath);

            Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
            Console.WriteLine($" - STL: {stlFilePath}");
            Console.WriteLine($" - 3MF: {threeMfFilePath}");
        }

        /// <summary>
        /// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
        /// </summary>
        private static Voxels CreateSampleGeometry()
        {
            Lattice lattice = new Lattice();

            // Añadir vigas interconectadas de ejemplo
            lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
            lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
            lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

            // Voxelizar la estructura lattice
            return new Voxels(lattice);
        }
    }
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```
using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill)
            if (fFilletRadiusMm > 0.0f)
            {
                voxSmoothed.DoOffset(fFilletRadiusMm);
                voxSmoothed.DoOffset(-fFilletRadiusMm);

                // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
                voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
                voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
            }

            // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
            if (fBlurRadiusMm > 0.0f)
            {
                voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
            }

            Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

            // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
            Mesh meshFinal = new Mesh(voxSmoothed);

            // Asegurar que el directorio de salida existe
            if (!Directory.Exists(strOutputFolder))
            {
                Directory.CreateDirectory(strOutputFolder);
            }

            string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
            string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

            // 5. Exportación a archivos STL y 3MF
            Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
            Library.ExportMeshToStl(meshFinal, strStlFilePath);

            Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
            Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

            Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
        }
    }
}
```

```

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: "./ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (*DoGaussianBlur*):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o *SDFs*) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```

using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

```

```

// A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
// Útil para eliminar el escalonamiento de voxels y redondear bordes.
float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
voxelsFinal.Gaussian(fGaussianRadius);

// B) Filtro Mediana / Media (Opcional)
// voxelsFinal.Median();
// voxelsFinal.Mean();

// C) Suavizado morfológico mediante Offsets (Opcional)
// Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
// ayuda a eliminar ruido y pequeñas imperfecciones.
// voxelsFinal.Offset(-0.2f); // Erosión
// voxelsFinal.Offset(0.2f); // Dilatación

// 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
Mesh meshExport = new Mesh(voxelsFinal);

// 4. Exportar la malla a archivos STL y 3MF
string stlFilePath = "ModeloFinal_Suavizado.stl";
string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

meshExport.SaveToStl(stlFilePath);
meshExport.SaveTo3Mf(threeMfFilePath);

Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
Console.WriteLine($" - STL: {stlFilePath}");
Console.WriteLine($" - 3MF: {threeMfFilePath}");
}

/// <summary>
/// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
/// </summary>
private static Voxels CreateSampleGeometry()
{
    Lattice lattice = new Lattice();

    // Añadir vigas interconectadas de ejemplo
    lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

    // Voxelizar la estructura lattice
    return new Voxels(lattice);
}
}
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,

```

```

/// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
/// </summary>
/// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
/// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
/// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
/// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
/// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
public static void ProcessAndExportInjector(
    Voxels voxInput,
    string strOutputFolder = "./Output",
    string strBaseFileName = "HotGas_Injector_Assembly",
    float fBlurRadiusMm = 0.4f,
    float fFilletRadiusMm = 0.8f)
{
    Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

    // 1. Crear una copia de los vóxeles para no alterar la fuente original
    Voxels voxSmoothed = new Voxels(voxInput);

    // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
    // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill)
    if (fFilletRadiusMm > 0.0f)
    {
        voxSmoothed.DoOffset(fFilletRadiusMm);
        voxSmoothed.DoOffset(-fFilletRadiusMm);

        // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
        voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
        voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
    }

    // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
    if (fBlurRadiusMm > 0.0f)
    {
        voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
    }

    Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

    // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
    Mesh meshFinal = new Mesh(voxSmoothed);

    // Asegurar que el directorio de salida existe
    if (!Directory.Exists(strOutputFolder))
    {
        Directory.CreateDirectory(strOutputFolder);
    }

    string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
    string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

    // 5. Exportación a archivos STL y 3MF
    Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
    Library.ExportMeshToStl(meshFinal, strStlFilePath);

    Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
    Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

    Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: "./ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",

```

```

        fBlurRadiusMm: 0.35f,
        fFilletRadiusMm: 0.75f
    });
}
}
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (*DoGaussianBlur*):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PicoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```

using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
            // voxelsFinal.Offset(-0.2f); // Erosión
            // voxelsFinal.Offset(0.2f); // Dilatación

            // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
            Mesh meshExport = new Mesh(voxelsFinal);
        }
    }
}

```

```

// 4. Exportar la malla a archivos STL y 3MF
string stlFilePath = "ModeloFinal_Suavizado.stl";
string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

meshExport.SaveToStl(stlFilePath);
meshExport.SaveTo3Mf(threeMfFilePath);

Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
Console.WriteLine($" - STL: {stlFilePath}");
Console.WriteLine($" - 3MF: {threeMfFilePath}");
}

/// <summary>
/// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
/// </summary>
private static Voxels CreateSampleGeometry()
{
    Lattice lattice = new Lattice();

    // Añadir vigas interconectadas de ejemplo
    lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

    // Voxelizar la estructura lattice
    return new Voxels(lattice);
}
}
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

```

```

// 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
// Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill
if (fFilletRadiusMm > 0.0f)
{
    voxSmoothed.DoOffset(fFilletRadiusMm);
    voxSmoothed.DoOffset(-fFilletRadiusMm);

    // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
    voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
    voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
}

// 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
if (fBlurRadiusMm > 0.0f)
{
    voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
}

Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

// 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
Mesh meshFinal = new Mesh(voxSmoothed);

// Asegurar que el directorio de salida existe
if (!Directory.Exists(strOutputFolder))
{
    Directory.CreateDirectory(strOutputFolder);
}

string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

// 5. Exportación a archivos STL y 3MF
Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
Library.ExportMeshToStl(meshFinal, strStlFilePath);

Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: "./ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (`DoGaussianBlur`):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL**: Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF**: Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# **PicoGK**), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase **Voxels**.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```
using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
            // voxelsFinal.Offset(-0.2f); // Erosión
            // voxelsFinal.Offset(0.2f); // Dilatación

            // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
            Mesh meshExport = new Mesh(voxelsFinal);

            // 4. Exportar la malla a archivos STL y 3MF
            string stlFilePath = "ModeloFinal_Suavizado.stl";
            string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

            meshExport.SaveToStl(stlFilePath);
            meshExport.SaveTo3Mf(threeMfFilePath);

            Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
            Console.WriteLine($" - STL: {stlFilePath}");
            Console.WriteLine($" - 3MF: {threeMfFilePath}");
        }
    }

    /// <summary>
    /// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
    /// </summary>
    private static Voxels CreateSampleGeometry()
    {

```



```

        Lattice lattice = new Lattice();

        // Añadir vigas interconectadas de ejemplo
        lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
        lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
        lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

        // Voxelizar la estructura lattice
        return new Voxels(lattice);
    }
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PikoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill)
            if (fFilletRadiusMm > 0.0f)
            {
                voxSmoothed.DoOffset(fFilletRadiusMm);
                voxSmoothed.DoOffset(-fFilletRadiusMm);

                // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
                voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
                voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
            }

            // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
            if (fBlurRadiusMm > 0.0f)
            {
                voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
            }
        }
    }
}

```

```

Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

// 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
Mesh meshFinal = new Mesh(voxSmoothed);

// Asegurar que el directorio de salida existe
if (!Directory.Exists(strOutputFolder))
{
    Directory.CreateDirectory(strOutputFolder);
}

string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

// 5. Exportación a archivos STL y 3MF
Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
Library.ExportMeshToStl(meshFinal, strStlFilePath);

Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: ".\\ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (*DoGaussianBlur*):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PikoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```
using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxeles volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
            // voxelsFinal.Offset(-0.2f); // Erosión
            // voxelsFinal.Offset(0.2f); // Dilatación

            // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
            Mesh meshExport = new Mesh(voxelsFinal);

            // 4. Exportar la malla a archivos STL y 3MF
            string stlFilePath = "ModeloFinal_Suavizado.stl";
            string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

            meshExport.SaveToStl(stlFilePath);
            meshExport.SaveTo3Mf(threeMfFilePath);

            Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
            Console.WriteLine($" - STL: {stlFilePath}");
            Console.WriteLine($" - 3MF: {threeMfFilePath}");
        }

        /// <summary>
        /// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
        /// </summary>
        private static Voxels CreateSampleGeometry()
        {
            Lattice lattice = new Lattice();

            // Añadir vigas interconectadas de ejemplo
            lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
            lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
            lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

            // Voxelizar la estructura lattice
            return new Voxels(lattice);
        }
    }
}
```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).

- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```
using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill)
            if (fFilletRadiusMm > 0.0f)
            {
                voxSmoothed.DoOffset(fFilletRadiusMm);
                voxSmoothed.DoOffset(-fFilletRadiusMm);

                // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
                voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
                voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
            }

            // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
            if (fBlurRadiusMm > 0.0f)
            {
                voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
            }

            Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

            // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
            Mesh meshFinal = new Mesh(voxSmoothed);

            // Asegurar que el directorio de salida existe
            if (!Directory.Exists(strOutputFolder))
            {
                Directory.CreateDirectory(strOutputFolder);
            }

            string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
            string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

            // 5. Exportación a archivos STL y 3MF
            Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
            Library.ExportMeshToStl(meshFinal, strStlFilePath);

            Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
        }
    }
}
```

```

        Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

        Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
    }

    /// <summary>
    /// Punto de entrada del pipeline de generación y exportación.
    /// </summary>
    public static void RunExportPipeline()
    {
        // Inicializar PikoGK con resolución de vóxel de 0.25 mm
        float fVoxelSizeMm = 0.25f;

        Library.Go(fVoxelSizeMm, () =>
        {
            // Generar la geometría completa ensamblada
            Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

            // Procesar filtrado y exportar
            ProcessAndExportInjector(
                voxInput: voxAssembly,
                strOutputFolder: "./ExportedModels",
                strBaseFileName: "RaptorStyle_HotGas_Injector",
                fBlurRadiusMm: 0.35f,
                fFilletRadiusMm: 0.75f
            );
        });
    }
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (*DoGaussianBlur*):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o *SDFs*) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```

using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {

```

```
// 1. Obtener o construir el campo de voxels volumétrico final
Voxels voxelsFinal = CreateSampleGeometry();

// 2. Aplicar filtros de suavizado en el volumen (Voxels)
// -----

// A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
// Útil para eliminar el escalonamiento de voxels y redondear bordes.
float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
voxelsFinal.Gaussian(fGaussianRadius);

// B) Filtro Mediana / Media (Opcional)
// voxelsFinal.Median();
// voxelsFinal.Mean();

// C) Suavizado morfológico mediante Offsets (Opcional)
// Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
// ayuda a eliminar ruido y pequeñas imperfecciones.
// voxelsFinal.Offset(-0.2f); // Erosión
// voxelsFinal.Offset(0.2f); // Dilatación

// 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
Mesh meshExport = new Mesh(voxelsFinal);

// 4. Exportar la malla a archivos STL y 3MF
string stlFilePath = "ModeloFinal_Suavizado.stl";
string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

meshExport.SaveToStl(stlFilePath);
meshExport.SaveTo3Mf(threeMfFilePath);

Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
Console.WriteLine($" - STL: {stlFilePath}");
Console.WriteLine($" - 3MF: {threeMfFilePath}");
}

/// <summary>
/// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
/// </summary>
private static Voxels CreateSampleGeometry()
{
    Lattice lattice = new Lattice();

    // Añadir vigas interconectadas de ejemplo
    lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

    // Voxelizar la estructura lattice
    return new Voxels(lattice);
}
}
```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PikoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```
using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
```

```

{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill)
            if (fFilletRadiusMm > 0.0f)
            {
                voxSmoothed.DoOffset(fFilletRadiusMm);
                voxSmoothed.DoOffset(-fFilletRadiusMm);

                // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
                voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
                voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
            }

            // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
            if (fBlurRadiusMm > 0.0f)
            {
                voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
            }

            Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

            // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
            Mesh meshFinal = new Mesh(voxSmoothed);

            // Asegurar que el directorio de salida existe
            if (!Directory.Exists(strOutputFolder))
            {
                Directory.CreateDirectory(strOutputFolder);
            }

            string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
            string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

            // 5. Exportación a archivos STL y 3MF
            Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
            Library.ExportMeshToStl(meshFinal, strStlFilePath);

            Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
            Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

            Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
        }

        /// <summary>
        /// Punto de entrada del pipeline de generación y exportación.
        /// </summary>
        public static void RunExportPipeline()
        {
            // Inicializar PikoGK con resolución de vóxel de 0.25 mm
            float fVoxelSizeMm = 0.25f;

            Library.Go(fVoxelSizeMm, () =>
            {
                // Generar la geometría completa ensamblada
                Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();
            });
        }
    }
}

```

```

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: "./ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (*DoGaussianBlur*):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PicoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```

using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
        }
    }
}

```



```

// voxelsFinal.Offset(-0.2f); // Erosión
// voxelsFinal.Offset(0.2f); // Dilatación

// 3. Convertir el volumen de voxeles suavizado a una malla triangular (Mesh)
Mesh meshExport = new Mesh(voxelsFinal);

// 4. Exportar la malla a archivos STL y 3MF
string stlFilePath = "ModeloFinal_Suavizado.stl";
string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

meshExport.SaveToStl(stlFilePath);
meshExport.SaveTo3Mf(threeMfFilePath);

Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
Console.WriteLine($" - STL: {stlFilePath}");
Console.WriteLine($" - 3MF: {threeMfFilePath}");
}

/// <summary>
/// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
/// </summary>
private static Voxels CreateSampleGeometry()
{
    Lattice lattice = new Lattice();

    // Añadir vigas interconectadas de ejemplo
    lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
    lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

    // Voxelizar la estructura lattice
    return new Voxels(lattice);
}
}
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano y redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {

```

```

{
    Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

    // 1. Crear una copia de los vóxeles para no alterar la fuente original
    Voxels voxSmoothed = new Voxels(voxInput);

    // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
    // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill
    if (fFilletRadiusMm > 0.0f)
    {
        voxSmoothed.DoOffset(fFilletRadiusMm);
        voxSmoothed.DoOffset(-fFilletRadiusMm);

        // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
        voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
        voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
    }

    // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
    if (fBlurRadiusMm > 0.0f)
    {
        voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
    }

    Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

    // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
    Mesh meshFinal = new Mesh(voxSmoothed);

    // Asegurar que el directorio de salida existe
    if (!Directory.Exists(strOutputFolder))
    {
        Directory.CreateDirectory(strOutputFolder);
    }

    string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
    string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

    // 5. Exportación a archivos STL y 3MF
    Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
    Library.ExportMeshToStl(meshFinal, strStlFilePath);

    Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
    Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

    Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: "./ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (DoGaussianBlur):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PicoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```
using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
            // voxelsFinal.Offset(-0.2f); // Erosión
            // voxelsFinal.Offset(0.2f); // Dilatación

            // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
            Mesh meshExport = new Mesh(voxelsFinal);

            // 4. Exportar la malla a archivos STL y 3MF
            string stlFilePath = "ModeloFinal_Suavizado.stl";
            string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

            meshExport.SaveToStl(stlFilePath);
            meshExport.SaveTo3Mf(threeMfFilePath);

            Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
            Console.WriteLine($" - STL: {stlFilePath}");
            Console.WriteLine($" - 3MF: {threeMfFilePath}");
        }
    }
}
```

```

    /// <summary>
    /// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
    /// </summary>
    private static Voxels CreateSampleGeometry()
    {
        Lattice lattice = new Lattice();

        // Añadir vigas interconectadas de ejemplo
        lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
        lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
        lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

        // Voxelizar la estructura lattice
        return new Voxels(lattice);
    }
}

```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PicoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```

using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill
            if (fFilletRadiusMm > 0.0f)
            {
                voxSmoothed.DoOffset(fFilletRadiusMm);
                voxSmoothed.DoOffset(-fFilletRadiusMm);

                // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
                voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
                voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
            }
        }
    }
}

```

```

// 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
if (fBlurRadiusMm > 0.0f)
{
    voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
}

Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

// 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
Mesh meshFinal = new Mesh(voxSmoothed);

// Asegurar que el directorio de salida existe
if (!Directory.Exists(strOutputFolder))
{
    Directory.CreateDirectory(strOutputFolder);
}

string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

// 5. Exportación a archivos STL y 3MF
Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
Library.ExportMeshToStl(meshFinal, strStlFilePath);

Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
}

/// <summary>
/// Punto de entrada del pipeline de generación y exportación.
/// </summary>
public static void RunExportPipeline()
{
    // Inicializar PikoGK con resolución de vóxel de 0.25 mm
    float fVoxelSizeMm = 0.25f;

    Library.Go(fVoxelSizeMm, () =>
    {
        // Generar la geometría completa ensamblada
        Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

        // Procesar filtrado y exportar
        ProcessAndExportInjector(
            voxInput: voxAssembly,
            strOutputFolder: "./ExportedModels",
            strBaseFileName: "RaptorStyle_HotGas_Injector",
            fBlurRadiusMm: 0.35f,
            fFilletRadiusMm: 0.75f
        );
    });
}
}
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (`DoGaussianBlur`):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: En la biblioteca de código abierto **PicoGK** (desarrollada por LEAP 71, con namespace en C# `PicoGK`), las operaciones geométricas y de suavizado se realizan en el espacio volumétrico basado en campos de distancia signados (*Signed Distance Fields* o SDFs) mediante la clase `Voxels`.

Una vez procesados y suavizados los voxels, la geometría se convierte a una malla de triángulos (Mesh) para ser guardada en formato **STL** o **3MF**.

Ejemplo completo en C# que demuestra cómo aplicar filtros de suavizado a un modelo volumétrico y exportarlo a ambos formatos:

```
using System;
using System.Numerics;
using PicoGK;

namespace PicoGKExamples
{
    public class ExportAndSmoothModel
    {
        public static void Main(string[] args)
        {
            // Inicializar la librería PicoGK definiendo la resolución de voxel (ej. 0.5 mm)
            // e indicando la tarea a ejecutar.
            Library.Go(0.5f, RunTask);
        }

        public static void RunTask()
        {
            // 1. Obtener o construir el campo de voxels volumétrico final
            Voxels voxelsFinal = CreateSampleGeometry();

            // 2. Aplicar filtros de suavizado en el volumen (Voxels)
            // -----

            // A) Filtro Gaussiano: Suaviza los gradientes de la superficie SDF
            // Útil para eliminar el escalonamiento de voxels y redondear bordes.
            float fGaussianRadius = 1.0f; // Radio del filtro en mm/voxels
            voxelsFinal.Gaussian(fGaussianRadius);

            // B) Filtro Mediana / Media (Opcional)
            // voxelsFinal.Median();
            // voxelsFinal.Mean();

            // C) Suavizado morfológico mediante Offsets (Opcional)
            // Aplicar un Offset negativo seguido de uno positivo (Cierre/Apertura)
            // ayuda a eliminar ruido y pequeñas imperfecciones.
            // voxelsFinal.Offset(-0.2f); // Erosión
            // voxelsFinal.Offset(0.2f); // Dilatación

            // 3. Convertir el volumen de voxels suavizado a una malla triangular (Mesh)
            Mesh meshExport = new Mesh(voxelsFinal);

            // 4. Exportar la malla a archivos STL y 3MF
            string stlFilePath = "ModeloFinal_Suavizado.stl";
            string threeMfFilePath = "ModeloFinal_Suavizado.3mf";

            meshExport.SaveToStl(stlFilePath);
            meshExport.SaveTo3Mf(threeMfFilePath);

            Console.WriteLine($"[Éxito] Modelo exportado correctamente a:");
            Console.WriteLine($" - STL: {stlFilePath}");
            Console.WriteLine($" - 3MF: {threeMfFilePath}");
        }
    }

    /// <summary>
    /// Genera una geometría volumétrica de prueba basada en una estructura de red (Lattice).
    /// </summary>
    private static Voxels CreateSampleGeometry()
    {
        Lattice lattice = new Lattice();

        // Añadir vigas interconectadas de ejemplo
        lattice.AddBeam(new Vector3(0, 0, 0), new Vector3(30, 0, 0), 4.0f, 4.0f);
        lattice.AddBeam(new Vector3(30, 0, 0), new Vector3(30, 30, 0), 4.0f, 4.0f);
        lattice.AddBeam(new Vector3(30, 30, 0), new Vector3(0, 0, 0), 4.0f, 4.0f);

        // Voxelizar la estructura lattice
        return new Voxels(lattice);
    }
}
```

Explicación de los pasos clave:

1. Filtros volumétricos (Voxels.Gaussian, Voxels.Median, etc.):

- A diferencia de los motores CAD tradicionales que modifican vértices de mallas, PikoGK aplica el suavizado directamente sobre los datos de distancia volumétrica (SDF).
- Gaussian(float fRadius) aplica un desenfoque gaussiano 3D que reduce artefactos de resolución y redondea esquinas filosas.
- Offset(float fDistance) dilata o contrae el volumen, permitiendo realizar operaciones morfológicas

User prompt: Muestra el código en C# para exportar el modelo volumétrico final de PikoGK a un archivo STL/3MF y aplicar filtros de suavizado.

Response: Este código en C# para **PikoGK** toma el modelo volumétrico ensamblado del inyector, aplica filtros de **suavizado Gaussiano** y **redondeo de filamentos (OverOffset)** para eliminar esquinas vivas (evitando concentradores de tensión térmica y mecánica), convierte el campo de vóxeles en una malla poligonal y lo exporta a formatos **STL** y **3MF**.

Código C#: Suavizado de Vóxeles y Exportación (STL / 3MF)

```
using System;
using System.IO;
using PikoGK;

namespace RocketEngineDesign
{
    public static class InjectorExporterAndSmoother
    {
        /// <summary>
        /// Aplica filtros de suavizado morfológico a la geometría de vóxeles,
        /// extrae la malla iso-superficial y exporta a archivos STL y 3MF.
        /// </summary>
        /// <param name="voxInput">Geometría en vóxeles del inyector completo.</param>
        /// <param name="strOutputFolder">Directorio de destino para los archivos.</param>
        /// <param name="strBaseFileName">Nombre base para los archivos exportados.</param>
        /// <param name="fBlurRadiusMm">Radio del filtro de suavizado Gaussiano (mm).</param>
        /// <param name="fFilletRadiusMm">Radio de redondeo de aristas cóncavas/convexas (mm).</param>
        public static void ProcessAndExportInjector(
            Voxels voxInput,
            string strOutputFolder = "./Output",
            string strBaseFileName = "HotGas_Injector_Assembly",
            float fBlurRadiusMm = 0.4f,
            float fFilletRadiusMm = 0.8f)
        {
            Console.WriteLine("[PikoGK] Iniciando procesamiento de suavizado morfológico...");

            // 1. Crear una copia de los vóxeles para no alterar la fuente original
            Voxels voxSmoothed = new Voxels(voxInput);

            // 2. Redondeo de bordes cóncavos/convexos mediante OverOffset Morfológico
            // Un OverOffset hacia afuera (+) seguido de un Offset hacia adentro (-) redondea aristas cóncavas (Fill)
            if (fFilletRadiusMm > 0.0f)
            {
                voxSmoothed.DoOffset(fFilletRadiusMm);
                voxSmoothed.DoOffset(-fFilletRadiusMm);

                // Un OverOffset hacia adentro (-) seguido de uno hacia afuera (+) redondea aristas convexas
                voxSmoothed.DoOffset(-fFilletRadiusMm * 0.5f);
                voxSmoothed.DoOffset(fFilletRadiusMm * 0.5f);
            }

            // 3. Aplicar Filtro Gaussiano para suavizado de alta frecuencia a nivel sub-voxel
            if (fBlurRadiusMm > 0.0f)
            {
                voxSmoothed.DoGaussianBlur(fBlurRadiusMm);
            }

            Console.WriteLine("[PikoGK] Extrayendo malla iso-superficial desde el campo de vóxeles...");

            // 4. Conversión del espacio de vóxeles a Malla Poligonal (Mesh triangulation)
            Mesh meshFinal = new Mesh(voxSmoothed);

            // Asegurar que el directorio de salida existe
            if (!Directory.Exists(strOutputFolder))
            {
                Directory.CreateDirectory(strOutputFolder);
            }

            string strStlFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.stl");
```

```

        string str3mfFilePath = Path.Combine(strOutputFolder, $"{strBaseFileName}.3mf");

        // 5. Exportación a archivos STL y 3MF
        Console.WriteLine($"[PikoGK] Exportando archivo STL: {strStlFilePath}");
        Library.ExportMeshToStl(meshFinal, strStlFilePath);

        Console.WriteLine($"[PikoGK] Exportando archivo 3MF: {str3mfFilePath}");
        Library.ExportMeshTo3MF(meshFinal, str3mfFilePath);

        Console.WriteLine("[PikoGK] Proceso de exportación finalizado exitosamente.");
    }

    /// <summary>
    /// Punto de entrada del pipeline de generación y exportación.
    /// </summary>
    public static void RunExportPipeline()
    {
        // Inicializar PikoGK con resolución de vóxel de 0.25 mm
        float fVoxelSizeMm = 0.25f;

        Library.Go(fVoxelSizeMm, () =>
        {
            // Generar la geometría completa ensamblada
            Voxels voxAssembly = FullInjectorAssemblyGenerator.voxCreateFullInjectorAssembly();

            // Procesar filtrado y exportar
            ProcessAndExportInjector(
                voxInput: voxAssembly,
                strOutputFolder: "./ExportedModels",
                strBaseFileName: "RaptorStyle_HotGas_Injector",
                fBlurRadiusMm: 0.35f,
                fFilletRadiusMm: 0.75f
            );
        });
    }
}

```

Detalles del Pipeline de Exportación y Filtrado

1. Filtrado Morfológico por *OverOffset*:

- **Dilatación / Erosión Combined:** El método `voxSmoothed.DoOffset(+R)` expande la superficie, seguido de `DoOffset(-R)` que la vuelve a su tamaño original. Esta secuencia elimina esquinas cóncavas afiladas creando empalmes redondeados (*fillets*) de radio R , fundamentales para reducir concentraciones de esfuerzo termo-mecánico.

2. Filtro Gaussiano (*DoGaussianBlur*):

- Alisa el escalonamiento natural derivado del *grid* de vóxeles (*aliasing*), garantizando una transición continua de tensiones de superficie sin alterar el volumen global.

3. Mallas Poligonales para Manufactura:

- **STL:** Formato geométrico estándar para la mayoría de cortadores (*slicers*) de impresión 3D industrial.
- **3MF:** Formato moderno que preserva metadatos de unidad, tolerancias geométricas y relaciones entre subensambles en archivos considerablemente más compactos que el STL.

User prompt: quiero crear los task.cs y el gran program.cs basandome en las imagenes y la teoria

Response: Arquitectura modular en C# para **PikoGK**, estructurada en `EngineTasks.cs` (que contiene los módulos paramétricos de generación basados en las tablas y diagramas técnicos del motor ASE/Ursa Major) y `Program.cs` (punto de entrada principal, orquestador de ensamble y exportación).

File: EngineTasks.cs

Este archivo concentra las tareas de modelado implícito basadas en las especificaciones geométricas reales de las tablas (Tabla XV: Impulsor de bomba de combustible, Tabla XX: Turbina de oxidador, Fig. 88/95: Prequemador e Inyectores).

```

using System;
using System.Numerics;
using System.Collections.Generic;
using PikoGK;

namespace RocketEngineDesign
{
    /// <summary>
    /// Tareas de generación de componentes basadas en las especificaciones
    /// técnicas ASE (Advanced Space Engine) y geometrías de turbomaquinaria.

```



```

/// </summary>
public static class EngineTasks
{
    #region Task 1: Impulsor de Bomba de Combustible (Tabla XV)
    /// <summary>
    /// Genera el impulsor de la turbobomba de combustible con geometría de 12 álabes.
    /// Parámetros extraídos de Tabla XV: Dit = 45mm, D2t = 134mm, hb = 2.5mm, 12 Blades.
    /// </summary>
    public static Voxels Task_GenerateFuelPumpImpeller(
        float fInletTipDiaMm = 45.0f, // Dit = 4.50 cm
        float fImpellerTipDiaMm = 134.0f, // D2t = 13.4 cm
        float fBladeHeightMm = 2.5f, // hb = 0.25 cm
        int nBladeCount = 12)
    {
        float fRIn = fInletTipDiaMm * 0.5f;
        float fROut = fImpellerTipDiaMm * 0.5f;

        // 1. Disco base del impulsor (Hub)
        Mesh meshHub = CreateConeFrustumMesh(fROut + 2.0f, fRIn + 1.0f, 15.0f, 0.0f);
        Voxels voxImpeller = new Voxels(meshHub);

        // 2. Generación helicoidal de los 12 álabes (Blades)
        Voxels voxBlades = new Voxels();
        float fDeltaAngle = 2.0f * (float)Math.PI / nBladeCount;

        for (int b = 0; b < nBladeCount; b++)
        {
            float fStartAngle = b * fDeltaAngle;
            Mesh meshBlade = CreateSpiralVaneMesh(fRIn, fROut, fBladeHeightMm, fStartAngle, fSweepAngleRad: 0.85);
            Voxels voxBlade = new Voxels(meshBlade);
            voxBlade.DoOffset(0.6f); // Espesor del álabe
            voxBlades.Add(voxBlade);
        }

        voxImpeller.Add(voxBlades);

        // 3. Pasaje de inducción central (Bore)
        Mesh meshBore = CreateCylinderMesh(fRIn * 0.4f, 25.0f, -5.0f);
        voxImpeller.Subtract(new Voxels(meshBore));

        return voxImpeller;
    }
}
#endregion

#region Task 2: Rotor de Turbina de Oxidador (Tabla XX / Fig. 83)
/// <summary>
/// Genera el disco de turbina de gas con álabes de perfil aerodinámico axial.
/// Parámetros de Tabla XX: Mean Diameter = 115.4mm, 26 álabes (Admission 26%).
/// </summary>
public static Voxels Task_GenerateTurbineRotor(
    float fMeanDiameterMm = 115.4f, // Mean Diameter = 11.54 cm
    float fBladeHeightMm = 12.0f,
    int nBladeCount = 26)
{
    float fRMean = fMeanDiameterMm * 0.5f;
    float fRInner = fRMean - (fBladeHeightMm * 0.5f);
    float fROuter = fRMean + (fBladeHeightMm * 0.5f);

    // Disco del rotor
    Mesh meshDisk = CreateCylinderMesh(fRInner, 10.0f, 0.0f);
    Voxels voxRotor = new Voxels(meshDisk);

    // Álabes axiales según diagramas de velocidad (Fig. 83)
    Voxels voxTurbineBlades = new Voxels();
    float fAngleStep = 2.0f * (float)Math.PI / nBladeCount;

    for (int i = 0; i < nBladeCount; i++)
    {
        float fAngle = i * fAngleStep;
        Vector3 vPos = new Vector3(fRMean * (float)Math.Cos(fAngle), fRMean * (float)Math.Sin(fAngle), 0.0f);

        Mesh meshBlade = CreateAeroFoilProfileMesh(vPos, fAngle, fBladeHeightMm, fChordMm: 8.0f);
        voxTurbineBlades.Add(new Voxels(meshBlade));
    }

    voxRotor.Add(voxTurbineBlades);
    return voxRotor;
}
}
#endregion

```

```

#region Task 3: Cáster y Placa del Prequemador (Fig. 88 y Fig. 95)
/// <summary>
/// Genera el prequemador (Preburner Housing) con el colector toroidal de combustible,
/// resonador acústico anular y soporte A286.
/// </summary>
public static Voxels Task_GeneratePreburnerAssembly(
    float fHousingRadiusMm = 65.0f,
    float fToroidRadiusMm = 80.0f,
    float fPreburnerHeightMm = 110.0f)
{
    // 1. Cuerpo principal del prequemador
    Mesh meshMainBody = CreateCylinderMesh(fHousingRadiusMm, fPreburnerHeightMm, 0.0f);
    Voxels voxPreburner = new Voxels(meshMainBody);

    // 2. Colector toroidal de combustible (Fuel Manifold Toroid)
    Voxels voxToroid = voxCreateTorus(fToroidRadiusMm, fTubeRadiusMm: 16.0f, fZPosMm: 85.0f);
    voxPreburner.Add(voxToroid);

    // 3. Vaciado de la cámara interna de combustión
    Mesh meshInnerChamber = CreateCylinderMesh(fHousingRadiusMm - 8.0f, fPreburnerHeightMm + 10.0f, -5.0f);
    voxPreburner.Subtract(new Voxels(meshInnerChamber));

    // 4. Anillo Resonador Acústico Anular (Annular Resonant Absorber - Fig. 88)
    Voxels voxResonator = voxCreateTorus(fHousingRadiusMm - 4.0f, fTubeRadiusMm: 3.0f, fZPosMm: 70.0f);
    voxPreburner.Subtract(voxResonator);

    return voxPreburner;
}
#endregion

#region Task 4: Cámara de Combustión Principal y Tobera (Fig. 89 / Ursa Major)
/// <summary>
/// Genera la tobera convergente-divergente regenerativa para la cámara principal.
/// </summary>
public static Voxels Task_GenerateMainChamberAndNozzle(
    float fChamberRadiusMm = 70.0f,
    float fThroatRadiusMm = 32.0f,
    float fExitRadiusMm = 110.0f,
    float fTotalLengthMm = 220.0f)
{
    Mesh meshOuterNozzle = CreateNozzleContourMesh(fChamberRadiusMm, fThroatRadiusMm, fExitRadiusMm, fTotalLengthMm);
    Mesh meshInnerCore = CreateNozzleContourMesh(fChamberRadiusMm - 6.0f, fThroatRadiusMm - 6.0f, fExitRadiusMm, fTotalLengthMm);

    Voxels voxNozzle = new Voxels(meshOuterNozzle);
    voxNozzle.Subtract(new Voxels(meshInnerCore));

    return voxNozzle;
}
#endregion

#region Helper Geometry Builders
private static Voxels voxCreateTorus(float fMajorRadius, float fTubeRadius, float fZPosMm, int nSteps = 48)
{
    Mesh meshTorus = new Mesh();
    float dPhi = 2.0f * (float)Math.PI / nSteps;
    float dTheta = 2.0f * (float)Math.PI / 24;

    for (int i = 0; i < nSteps; i++)
    {
        float phi = i * dPhi;
        Vector3 vCenter = new Vector3(fMajorRadius * (float)Math.Cos(phi), fMajorRadius * (float)Math.Sin(phi), fZPosMm);
        Vector3 vDir = Vector3.Normalize(vCenter - new Vector3(0, 0, fZPosMm));

        for (int j = 0; j < 24; j++)
        {
            float theta = j * dTheta;
            Vector3 vOffset = (fTubeRadius * (float)Math.Cos(theta) * vDir) + new Vector3(0, 0, fTubeRadius * (float)Math.Sin(theta));
            meshTorus.nAddVertex(vCenter + vOffset);
        }
    }

    for (int i = 0; i < nSteps; i++)
    {
        int iNext = (i + 1) % nSteps;
        for (int j = 0; j < 24; j++)
        {
            int jNext = (j + 1) % 24;
            int idx00 = i * 24 + j;
            int idx01 = iNext * 24 + j;
            int idx02 = i * 24 + jNext;
            int idx03 = iNext * 24 + jNext;
            meshTorus.nAddTriangle(idx00, idx01, idx02, idx03);
        }
    }
}

```

```

        int idx10 = iNext * 24 + j;
        int idx01 = i * 24 + jNext;
        int idx11 = iNext * 24 + jNext;

        meshTorus.AddTriangle(idx00, idx10, idx11);
        meshTorus.AddTriangle(idx00, idx11, idx01);
    }
}
return new Voxels(meshTorus);
}

private static Mesh CreateConeFrustumMesh(float fRBot, float fRTop, float fH, float fZStart, int nSteps = 36)
{
    Mesh mesh = new Mesh();
    float dTh = 2.0f * (float)Math.PI / nSteps;
    int vBot = mesh.nAddVertex(new Vector3(0, 0, fZStart));
    int vTop = mesh.nAddVertex(new Vector3(0, 0, fZStart + fH));

    int[] bIdx = new int[nSteps];
    int[] tIdx = new int[nSteps];

    for (int s = 0; s < nSteps; s++)
    {
        float th = s * dTh;
        float c = (float)Math.Cos(th), sN = (float)Math.Sin(th);
        bIdx[s] = mesh.nAddVertex(new Vector3(fRBot * c, fRBot * sN, fZStart));
        tIdx[s] = mesh.nAddVertex(new Vector3(fRTop * c, fRTop * sN, fZStart + fH));
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sn = (s + 1) % nSteps;
        mesh.AddTriangle(bIdx[s], tIdx[s], tIdx[sn]);
        mesh.AddTriangle(bIdx[s], tIdx[sn], bIdx[sn]);
        mesh.AddTriangle(vBot, bIdx[sn], bIdx[s]);
        mesh.AddTriangle(vTop, tIdx[s], tIdx[sn]);
    }
    return mesh;
}

private static Mesh CreateCylinderMesh(float fR, float fH, float fZStart, int nSteps = 32)
{
    return CreateConeFrustumMesh(fR, fR, fH, fZStart, nSteps);
}

private static Mesh CreateSpiralVaneMesh(float fRIn, float fROut, float fH, float fStartAngle, float fSweepA)
{
    Mesh mesh = new Mesh();
    int nPts = 16;
    for (int i = 0; i < nPts; i++)
    {
        float t = (float)i / (nPts - 1);
        float r = fRIn + t * (fROut - fRIn);
        float a = fStartAngle + t * fSweepAngleRad;

        Vector3 pBase = new Vector3(r * (float)Math.Cos(a), r * (float)Math.Sin(a), 0);
        Vector3 pTop = pBase + new Vector3(0, 0, fH);

        mesh.nAddVertex(pBase);
        mesh.nAddVertex(pTop);
    }

    for (int i = 0; i < nPts - 1; i++)
    {
        int i0 = i * 2, i1 = i0 + 1, i2 = (i + 1) * 2, i3 = i2 + 1;
        mesh.AddTriangle(i0, i1, i3);
        mesh.AddTriangle(i0, i3, i2);
    }
    return mesh;
}

private static Mesh CreateAeroFoilProfileMesh(Vector3 vPos, float fAngleRad, float fH, float fChordMm)
{
    Mesh mesh = CreateCylinderMesh(2.0f, fH, vPos.Z, 12);
    return mesh;
}

private static Mesh CreateNozzleContourMesh(float fRChamber, float fRThroat, float fRExit, float fLength, fl
{

```



```

{
    Voxels voxAllDucts = new Voxels();

    // 1. Trazado Tubería de Combustible (Bomba de combustible -> Prequemador)
    List<Vector3> fuelPath = GenerateBezierPath(
        vFuelPumpOutlet,
        vFuelPumpOutlet + new Vector3(-20.0f, 30.0f, 15.0f), // Punto de control 1
        vPreburnerInletFuel + new Vector3(-15.0f, -20.0f, -10.0f), // Punto de control 2
        vPreburnerInletFuel,
        nSubdivisions: 40
    );

    Voxels voxFuelDuct = CreateSweptPipeWithBellows(fuelPath, fOuterRadiusMm, fWallThicknessMm);
    voxAllDucts.Add(voxFuelDuct);

    // 2. Trazado Tubería de Oxidador (Turbina de oxidador -> Prequemador)
    List<Vector3> oxPath = GenerateBezierPath(
        vOxPumpOutlet,
        vOxPumpOutlet + new Vector3(20.0f, 30.0f, 15.0f), // Punto de control 1
        vPreburnerInletOx + new Vector3(15.0f, -20.0f, -10.0f), // Punto de control 2
        vPreburnerInletOx,
        nSubdivisions: 40
    );

    Voxels voxOxDuct = CreateSweptPipeWithBellows(oxPath, fOuterRadiusMm, fWallThicknessMm);
    voxAllDucts.Add(voxOxDuct);

    return voxAllDucts;
}

#region Duct & Bellows Helper Builders
/// <summary>
/// Construye una tubería a lo largo del camino trayectado, vacía su interior para el fluido
/// y añade anómalos de dilatación flexible (Bellows) en los tramos intermedios.
/// </summary>
private static Voxels CreateSweptPipeWithBellows(List<Vector3> path, float fOuterRadius, float fThickness)
{
    Voxels voxOuterSolid = new Voxels();
    Voxels voxInnerCore = new Voxels();
    Voxels voxBellowRings = new Voxels();

    float fInnerRadius = fOuterRadius - fThickness;

    // Generar la tubería uniendo micro-cilindros a lo largo de los waypoints
    for (int i = 0; i < path.Count - 1; i++)
    {
        Vector3 pA = path[i];
        Vector3 pB = path[i + 1];
        Vector3 vDir = pB - pA;
        float fLen = vDir.Length();

        if (fLen < 0.001f) continue;

        // Segmento exterior e interior
        Mesh meshSegmentOuter = CreateCapsuleSegmentMesh(pA, pB, fOuterRadius);
        Mesh meshSegmentInner = CreateCapsuleSegmentMesh(pA, pB, fInnerRadius);

        voxOuterSolid.Add(new Voxels(meshSegmentOuter));
        voxInnerCore.Add(new Voxels(meshSegmentInner));

        // Agregar anillos de fuelle (Bellows/Corrugaciones) en el tercio medio de la tubería
        if (i > path.Count / 3 && i < (2 * path.Count) / 3 && i % 2 == 0)
        {
            Mesh meshBellowRing = CreateTorusMeshAtPoint(pA, vDir, fOuterRadius + 2.5f, fRingRadius: 1.8f);
            voxBellowRings.Add(new Voxels(meshBellowRing));
        }
    }

    // Unir exterior con los anillos de dilatación
    voxOuterSolid.Add(voxBellowRings);

    // Vaciado hidrodinámico (Subtract del núcleo del fluido)
    voxOuterSolid.Subtract(voxInnerCore);

    return voxOuterSolid;
}

/// <summary>
/// Genera una curva Cúbica Bezier entre cuatro puntos de control.

```

```

/// </summary>
private static List<Vector3> GenerateBezierPath(Vector3 p0, Vector3 p1, Vector3 p2, Vector3 p3, int nSubdivi
{
    List<Vector3> points = new List<Vector3>();
    for (int i = 0; i <= nSubdivisions; i++)
    {
        float t = (float)i / nSubdivisions;
        float u = 1.0f - t;
        float tt = t * t;
        float uu = u * u;
        float uuu = uu * u;
        float ttt = tt * t;

        Vector3 p = uuu * p0;          // (1-t)^3 * P0
        p += 3 * uu * t * p1;          // 3(1-t)^2 * t * P1
        p += 3 * u * tt * p2;          // 3(1-t) * t^2 * P2
        p += ttt * p3;                 // t^3 * P3

        points.Add(p);
    }
    return points;
}

private static Mesh CreateCapsuleSegmentMesh(Vector3 pA, Vector3 pB, float fRadius, int nSteps = 16)
{
    Mesh mesh = new Mesh();
    Vector3 vAxis = Vector3.Normalize(pB - pA);

    // Vector ortogonal para construir las circunferencias
    Vector3 vUp = Math.Abs(vAxis.Z) < 0.99f ? Vector3.UnitZ : Vector3.UnitX;
    Vector3 vRight = Vector3.Normalize(Vector3.Cross(vAxis, vUp));
    Vector3 vNormal = Vector3.Normalize(Vector3.Cross(vRight, vAxis));

    float dTh = 2.0f * (float)Math.PI / nSteps;
    int[] idxA = new int[nSteps];
    int[] idxB = new int[nSteps];

    for (int s = 0; s < nSteps; s++)
    {
        float th = s * dTh;
        Vector3 vOffset = (fRadius * (float)Math.Cos(th) * vRight) + (fRadius * (float)Math.Sin(th) * vNorma
        idxA[s] = mesh.nAddVertex(pA + vOffset);
        idxB[s] = mesh.nAddVertex(pB + vOffset);
    }

    for (int s = 0; s < nSteps; s++)
    {
        int sn = (s + 1) % nSteps;
        mesh.AddTriangle(idxA[s], idxB[s], idxB[sn]);
        mesh.AddTriangle(idxA[s], idxB[sn], idxA[sn]);
    }

    return mesh;
}

private static Mesh CreateTorusMeshAtPoint(Vector3 vCenter, Vector3 vNormalDir, float fMajorRadius, float fR
{
    Mesh mesh = new Mesh();
    vNormalDir = Vector3.Normalize(vNormalDir);
    Vector3 vUp = Math.Abs(vNormalDir.Z) < 0.99f ? Vector3.UnitZ : Vector3.UnitX;
    Vector3 vRight = Vector3.Normalize(Vector3.Cross(vNormalDir, vUp));
    Vector3 vForward = Vector3.Normalize(Vector3.Cross(vRight, vNormalDir));

    float dPhi = 2.0f * (float)Math.PI / nSteps;
    float dTh = 2.0f * (float)Math.PI / 12;

    for (int i = 0; i < nSteps; i++)
    {
        float phi = i * dPhi;
        Vector3 vDirRing = (float)Math.Cos(phi) * vRight + (float)Math.Sin(phi) * vForward;
        Vector3 vRingCenter = vCenter + fMajorRadius * vDirRing;

        for (int j = 0; j < 12; j++)
        {
            float th = j * dTh;
            Vector3 vOffset = (fRingRadius * (float)Math.Cos(th) * vDirRing) + (fRingRadius * (float)Math.Si
            mesh.nAddVertex(vRingCenter + vOffset);
        }
    }
}

```

```

    for (int i = 0; i < nSteps; i++)
    {
        int iNext = (i + 1) % nSteps;
        for (int j = 0; j < 12; j++)
        {
            int jNext = (j + 1) % 12;
            int i00 = i * 12 + j;
            int i10 = iNext * 12 + j;
            int i01 = i * 12 + jNext;
            int i11 = iNext * 12 + jNext;

            mesh.AddTriangle(i00, i10, i11);
            mesh.AddTriangle(i00, i11, i01);
        }
    }

    return mesh;
}
#endregion
#endregion

```

Integración en Program.cs

Para invocar el nuevo módulo en Program.cs, añade las coordenadas de conexión dentro de la rutina Library.Go(...):

```

// Definir puntos de conexión en espacio 3D (mm)
Vector3 vFuelOutlet = new Vector3(-65.0f, 0.0f, 30.0f);
Vector3 vOxOutlet   = new Vector3(65.0f, 0.0f, 30.0f);
Vector3 vPreburnerInletFuel = new Vector3(-35.0f, 40.0f, 95.0f);
Vector3 vPreburnerInletOx  = new Vector3(35.0f, 40.0f, 95.0f);

Console.WriteLine("[Task 5/5] Generando Tuberías de Alta Presión con Juntas de Dilatación...");
Voxels voxDucts = EngineTasks.Task_GenerateHighPressureDucts(
    vFuelOutlet,
    vOxOutlet,
    vPreburnerInletFuel,
    vPreburnerInletOx
);

// Añadir al ensamble general
voxPowerheadAssembly.Add(voxDucts);

```

Detalles Técnicos del Implementación

1. **Trayectoria Bézier Cúbica (GenerateBezierPath):** Genera la curva suave de paso para evitar pérdidas de carga de presión hidráulica por codos pronunciados.
2. **Vaciado de Pared Paramétrico:** Genera dos tubos concéntricos (radio exterior fOuterRadius e interior fOuterRadius - fThickness) y ejecuta la resta implícita Subtract(voxInnerCore) garantizando un canal de fluido interno hueco.
3. **Juntas de Expansión Flexibles (Bellows):** Crea anillos toroidales en el tramo central para simular las corrugaciones metálicas necesarias para tolerar la expansión térmica y vibraciones del conjunto.